

JVM in the Age of AI

2026 Edition

Artur Skowroński





Fimbulvetr is here



Fimbulvetr is here

Artur Skowronski

Head of Application Development



SCALA 3.3.0 RELEASED!

TUESDAY 30 MAY 2023

Paweł Marks, VirtusLab



Contents

[What does "LTS" mean?](#)

[What's new in Scala 3.3.0](#)

[Linting](#)

→ [Checking for unused values](#)

→ [Checking for discarded values](#)

[More consistent braceless](#)

[syntax](#)

[boundary and break](#)

[The new default](#)

[implementation of lazy vals](#)

[Should I update to Scala 3.3.0?](#)

[Known incompatibility:](#)

[Stability of inline parameters](#)

[Contributors](#)

[Problem with this page?](#)

[Please help us fix it!](#)

Java's Plans for 2026 and new curiosities from JDK Mailing Lists - JVM Weekly vol. 159

Let's have a sneak peak into what will happen 2026!



ARTUR SKOWRONSKI

JAN 15, 2026



6

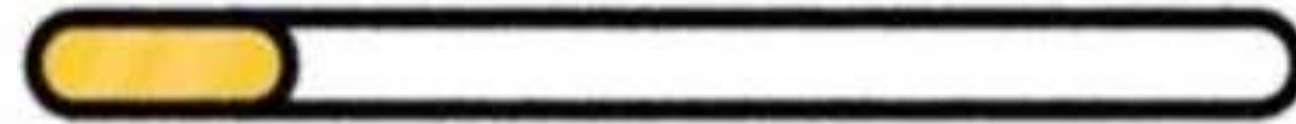


Share

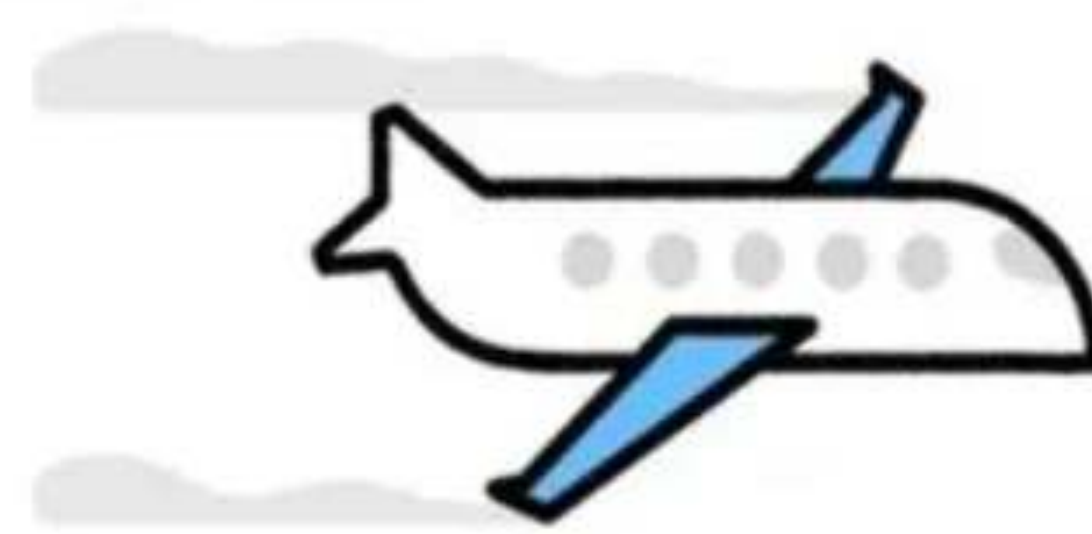
...



THE FASTEST THINGS ON EARTH



CHEETAH



AIRPLANE



SPEED OF LIGHT



PEOPLE BECOMING
EXPERTS IN AI



What we won't be talking about today...

- **Using LLM through API** (Langchain4j, Semantic Kernel, Spring AI etc.)
- **Data Engineering** (data collection, data cleaning, etc.)
- **Model Development** (MLOps, observability etc.)



What we won't be talking about today..

- The **usage** and **inference** from existing models
- **Mainly about GPU programming** 😊

What we will be talking about today!

- The **usage** and **inference** from existing models

- **Mainly about GPU programming** 😊

- ...but if I had told you about it, then probably no one would have shown up 😊



What we will be talking about today!



Helicopter View





JVM in the Age of AI: Deep Dive

11 demos

6 different JDKs...

...including 3 compiled from Source



OTHER PEOPLE WHEN THERE IS
HYPE ON THE MARKET:



ME:



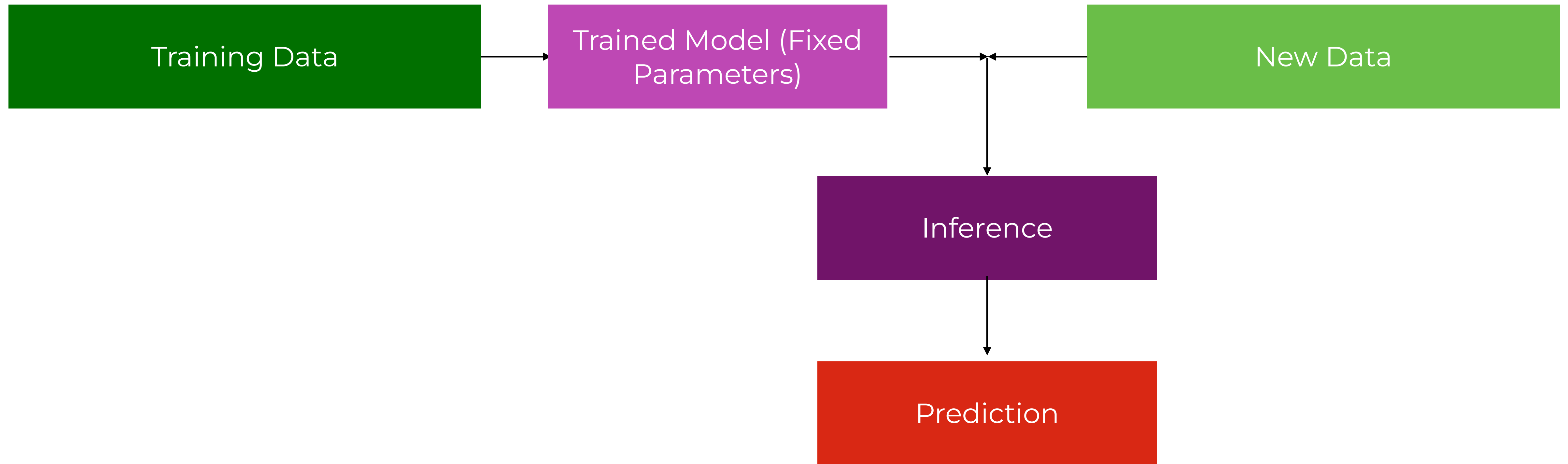
**I DON'T KNOW ANYTHING, BUT I DO KNOW
THAT EVERYTHING IS INTERESTING IF YOU
GO INTO IT DEEPLY ENOUGH.**

- RICHARD FEYNMAN -

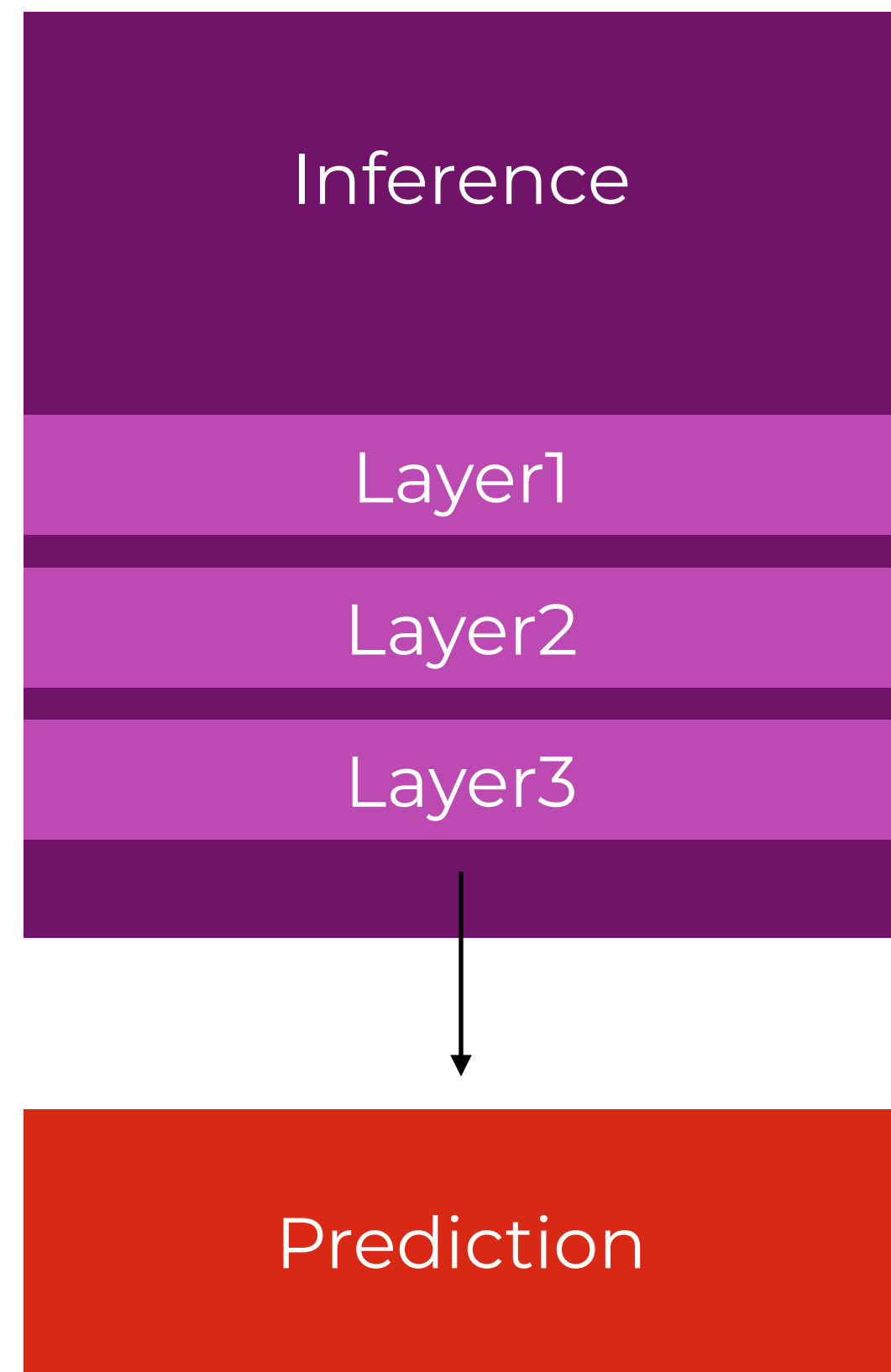


A bit of theory



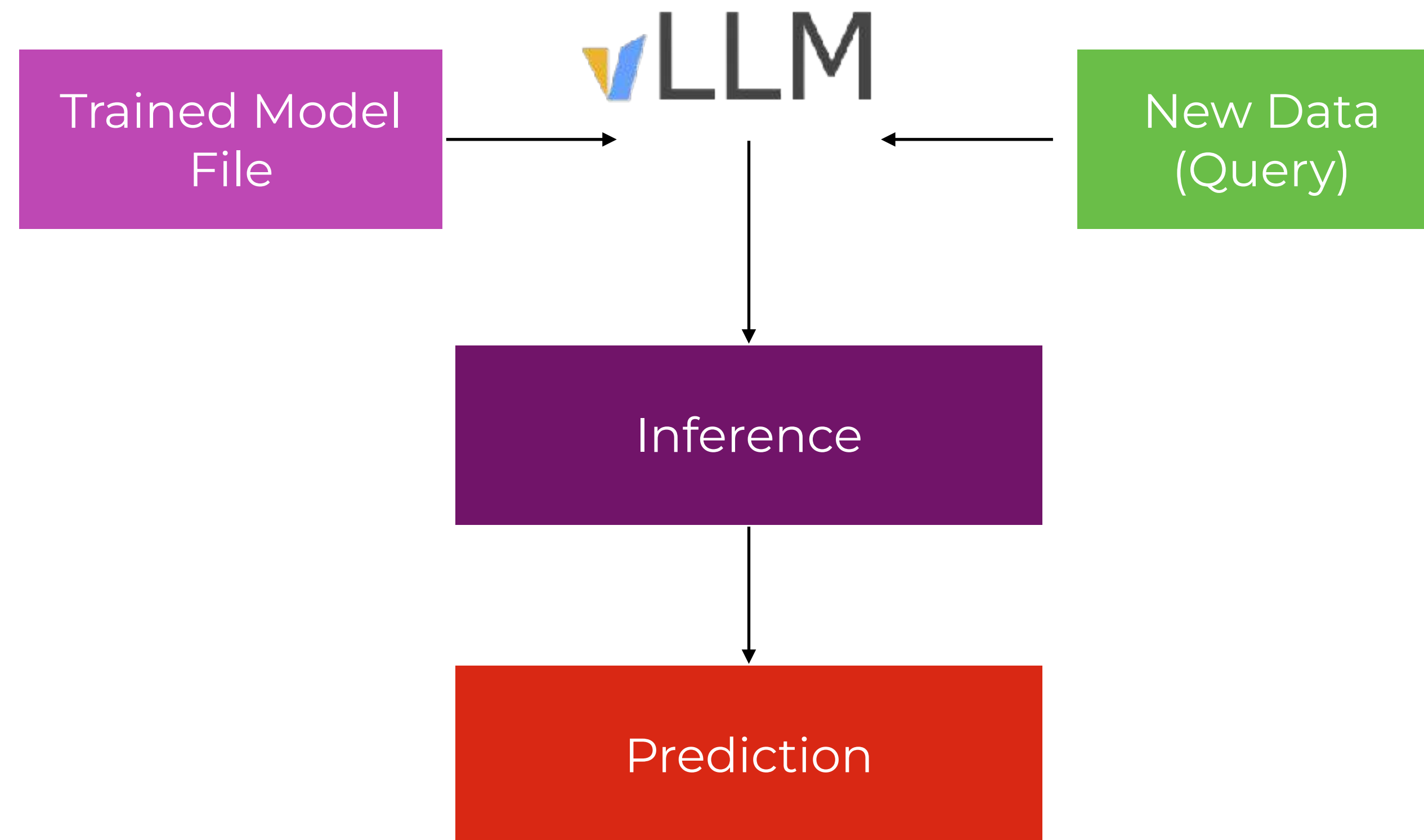


Model Inference

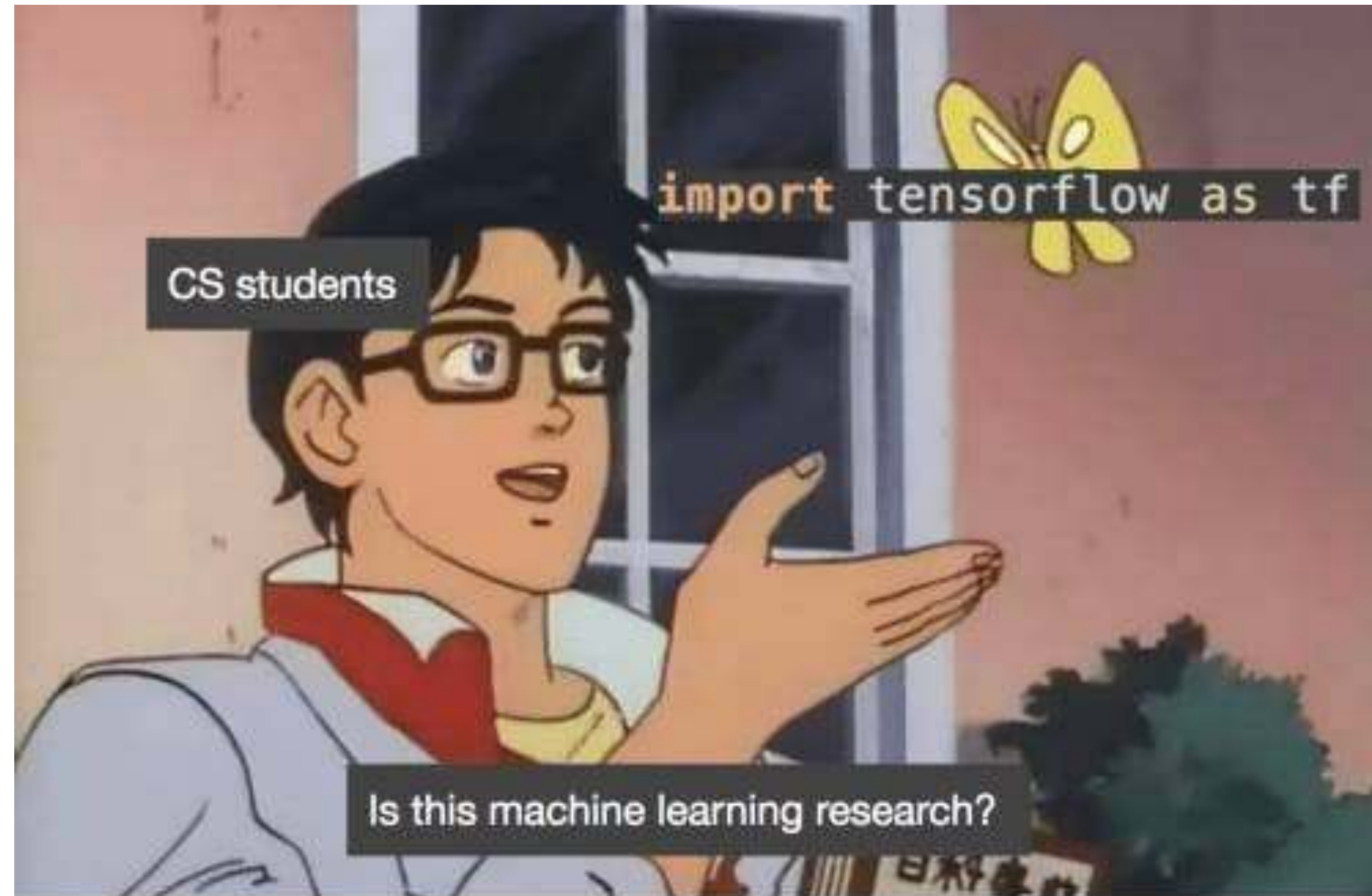


Next Token of Text
Classification of Image

Model Inference



Model Inference



Programmers Nowadays

Model Inference

ML in the past



Careful adding these features to our classifier as they are trivially related to the output class.



We don't want data leakage!



Yup, we'd be fooling ourselves and our clients.

AI today



Let's load up the entire Internet to the damn thing.

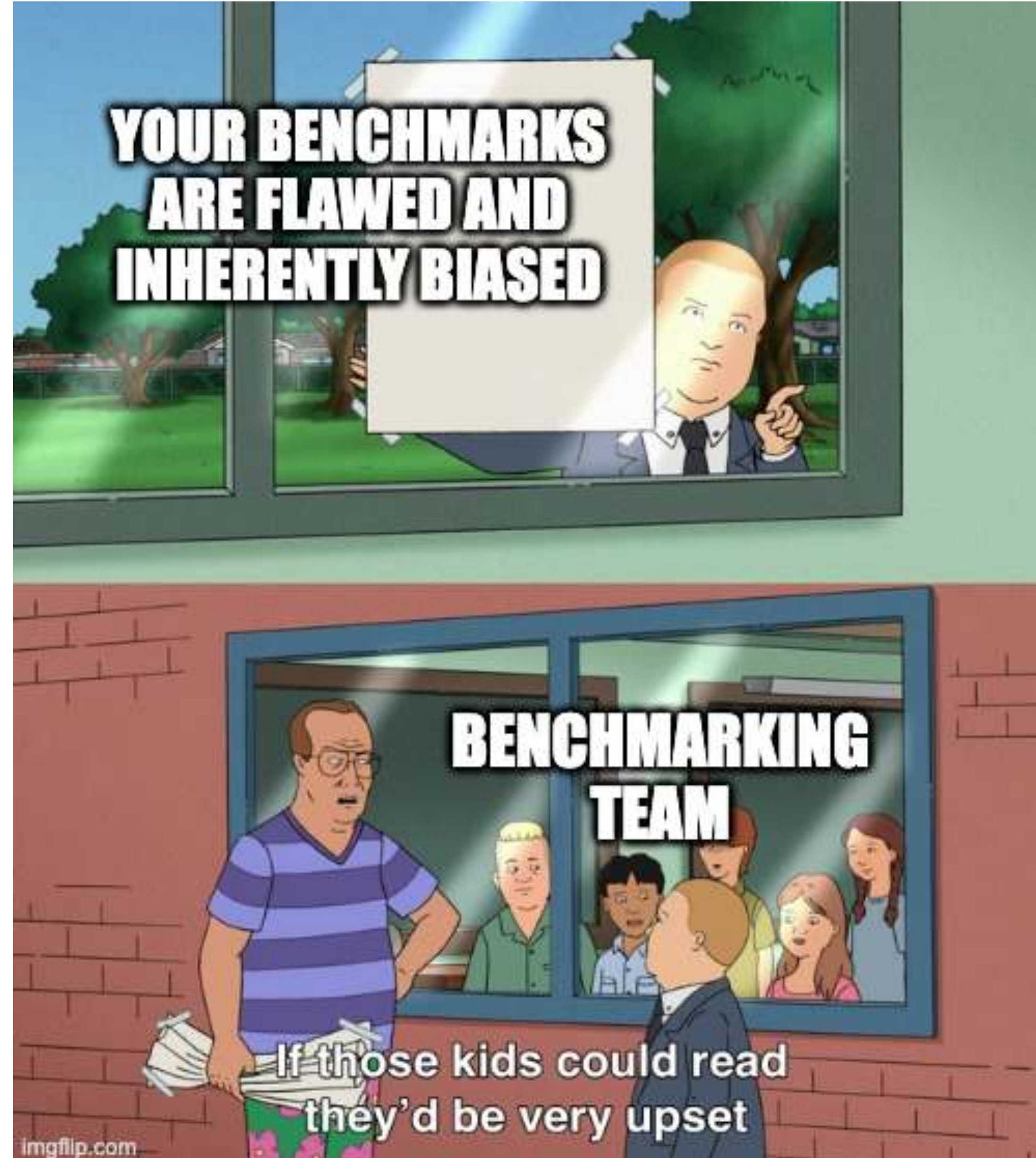


F*** yeah!

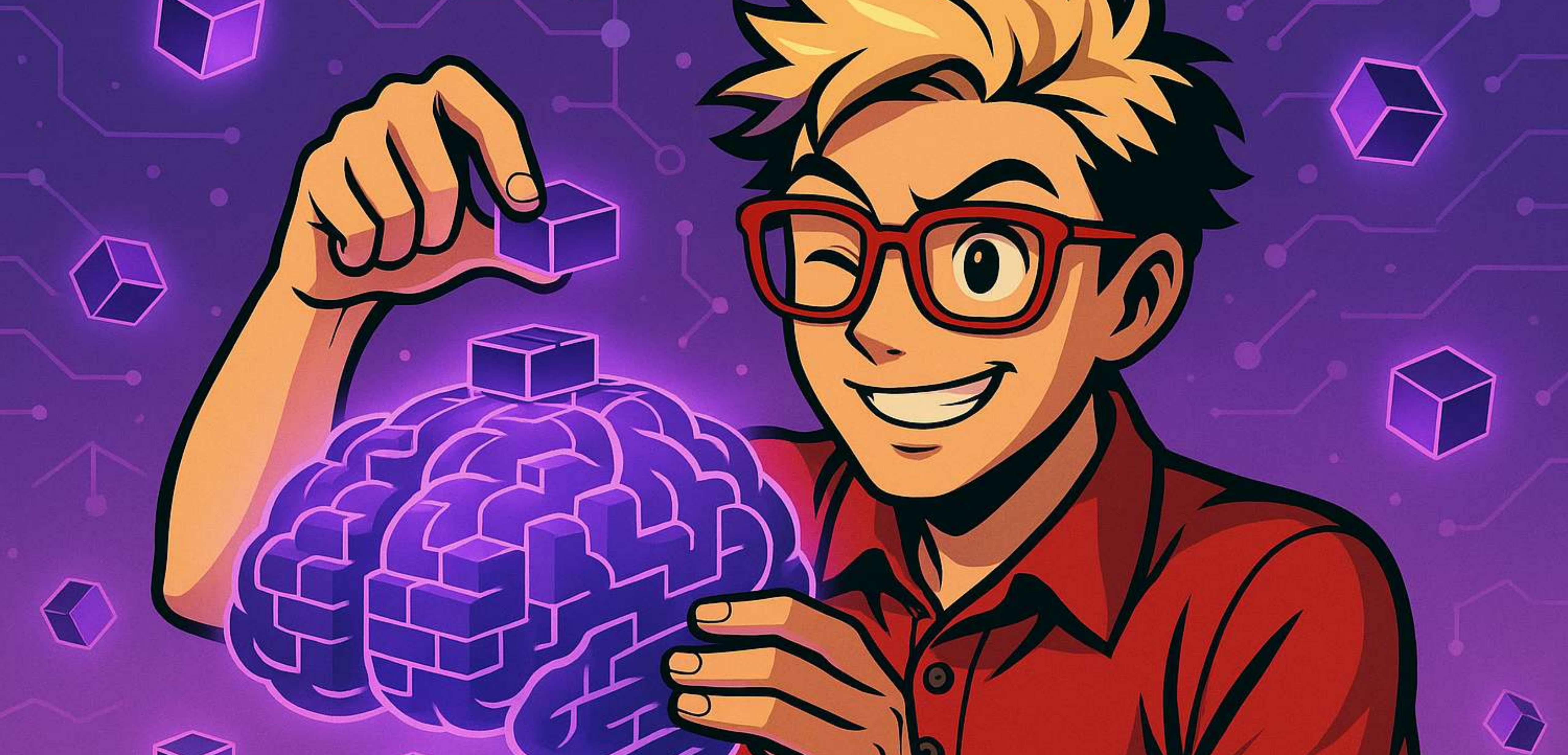


Excellent idea!

And as we are in 2026: AI = LLM



Discussing benchmark is freaking hard



So I want you to help build mental model what is achievable



NVIDIA TensorRT LLM



LLaMA C++

Serving the models

Llama 4

Llama 4 release

- [meta-llama/Llama-4-Scout-17B-16E-Instruct](#)
Image-to-Text • 109B • Updated May 23, 2025 • 201k • 1.21k
- [meta-llama/Llama-4-Scout-17B-16E](#)
Image-to-Text • 109B • Updated Apr 9, 2025 • 7.24k • 227
- [meta-llama/Llama-4-Maverick-17B-128E-Instruct](#)
Image-to-Text • 402B • Updated May 23, 2025 • 28.9k • 455
- [meta-llama/Llama-4-Maverick-17B-128E-Instruct-FP8](#)

Llama 3.3 Evals

This collection provides detailed information on how we derived the reported...

- [meta-llama/Llama-3.3-70B-Instruct-evals](#)
Viewer • Updated Dec 6, 2024 • 41.3k • 118 • 44

Llama 3.2 Evals

This collection provides detailed information on how we derived the reported...

- [meta-llama/Llama-3.2-1B-Instruct-evals](#)
Viewer • Updated Sep 25, 2024 • 142k • 247 • 26
- [meta-llama/Llama-3.2-1B-evals](#)
Viewer • Updated Sep 25, 2024 • 48.6k • 81 • 8
- [meta-llama/Llama-3.2-3B-Instruct-evals](#)
Viewer • Updated Sep 25, 2024 • 142k • 150 • 18
- [meta-llama/Llama-3.2-3B-evals](#)

Llama 3.1 Evals

This collection provides detailed information on how we derived the reported...

- [meta-llama/Llama-3.1-8B-evals](#)
Viewer • Updated Oct 2, 2024 • 79.7k • 55 • 24
- [meta-llama/Llama-3.1-8B-Instruct-evals](#)
Viewer • Updated Oct 2, 2024 • 158k • 804 • 32
- [meta-llama/Llama-3.1-70B-evals](#)
Viewer • Updated Oct 2, 2024 • 79.7k • 14 • 12

Llama 3.3

This collection hosts the transformers and original repos of the Llama 3.3

- [meta-llama/Llama-3.3-70B-Instruct](#)
Text Generation • 71B • Updated Dec 21, 2024 • 781k • 2.64k

Llama 3.2

This collection hosts the transformers and original repos of the Llama 3.2 and...

- [meta-llama/Llama-3.2-1B](#)
Text Generation • 1B • Updated Oct 24, 2024 • 2.42M • 2.28k
- [meta-llama/Llama-3.2-3B](#)
Text Generation • 3B • Updated Oct 24, 2024 • 797k • 689
- [meta-llama/Llama-3.2-1B-Instruct](#)
Text Generation • 1B • Updated Oct 24, 2024 • 2.71M • 1.28k
- [meta-llama/Llama-3.2-3B-Instruct](#)

Llama 3.1

This collection hosts the transformers and original repos of the Llama 3.1, Lla...

- [meta-llama/Llama-3.1-8B](#)
Text Generation • 8B • Updated Oct 17, 2024 • 2.66M • 2.04k
- [meta-llama/Llama-3.1-8B-Instruct](#)
Text Generation • 8B • Updated Sep 25, 2024 • 9.95M • 5.36k
- [meta-llama/Llama-3.1-70B](#)
Text Generation • 71B • Updated Sep 25, 2024 • 76.2k • 399
- [meta-llama/Llama-3.1-70B-Instruct](#)

Meta Llama 3

This collection hosts the transformers and original repos of the Meta Llama 3 ...

- [meta-llama/Meta-Llama-3-8B](#)
Text Generation • 8B • Updated Sep 27, 2024 • 1.69M • 6.44k
- [meta-llama/Meta-Llama-3-8B-Instruct](#)
Text Generation • 8B • Updated Jun 19, 2025 • 1.51M • 4.37k
- [meta-llama/Meta-Llama-3-70B-Instruct](#)
Text Generation • 71B • Updated Jun 19, 2025 • 49.2k • 1.5k

Llama Family



meta-llama/Llama-3.2-1B-Instruct



like

1.28k

Follow

Meta Llama

73.7k



Text Generation



Transformers



Safetensors



PyTorch



8 languages

llama

facebook

meta

llama-3

conversational



text-generation-inference



arxiv:2204.05149



arxiv:2405.16406



License: llama3.2



Model card



Files and versions



Community

144



Deploy



Use this model

You need to agree to share your contact information to access this model

The information you provide will be collected, stored, processed and shared in accordance with the [Meta Privacy Policy](#).

LLAMA 3.2 COMMUNITY LICENSE AGREEMENT

Llama 3.2 Version Release Date: September 25, 2024

"Agreement" means the terms and conditions for use, reproduction, distribution and modification of the Llama Materials set forth herein.

"Documentation" means the specifications, manuals and documentation accompanying Llama 3.2 distributed by Meta at <https://llama.meta.com/doc/overview>.

"Licensee" or "you" means you, or your employer or any other person or entity (if you are entering into this Agreement on such person or...

Log in

or

Sign Up

to review the conditions and access this model content.

Downloads last month

2,714,557



Safetensors

Model size

1B params

Tensor type

BF16



Chat template



Files info

Inference Providers

NEW

Novita



Text Generation

Examples



Run 15,000+ Models Instantly

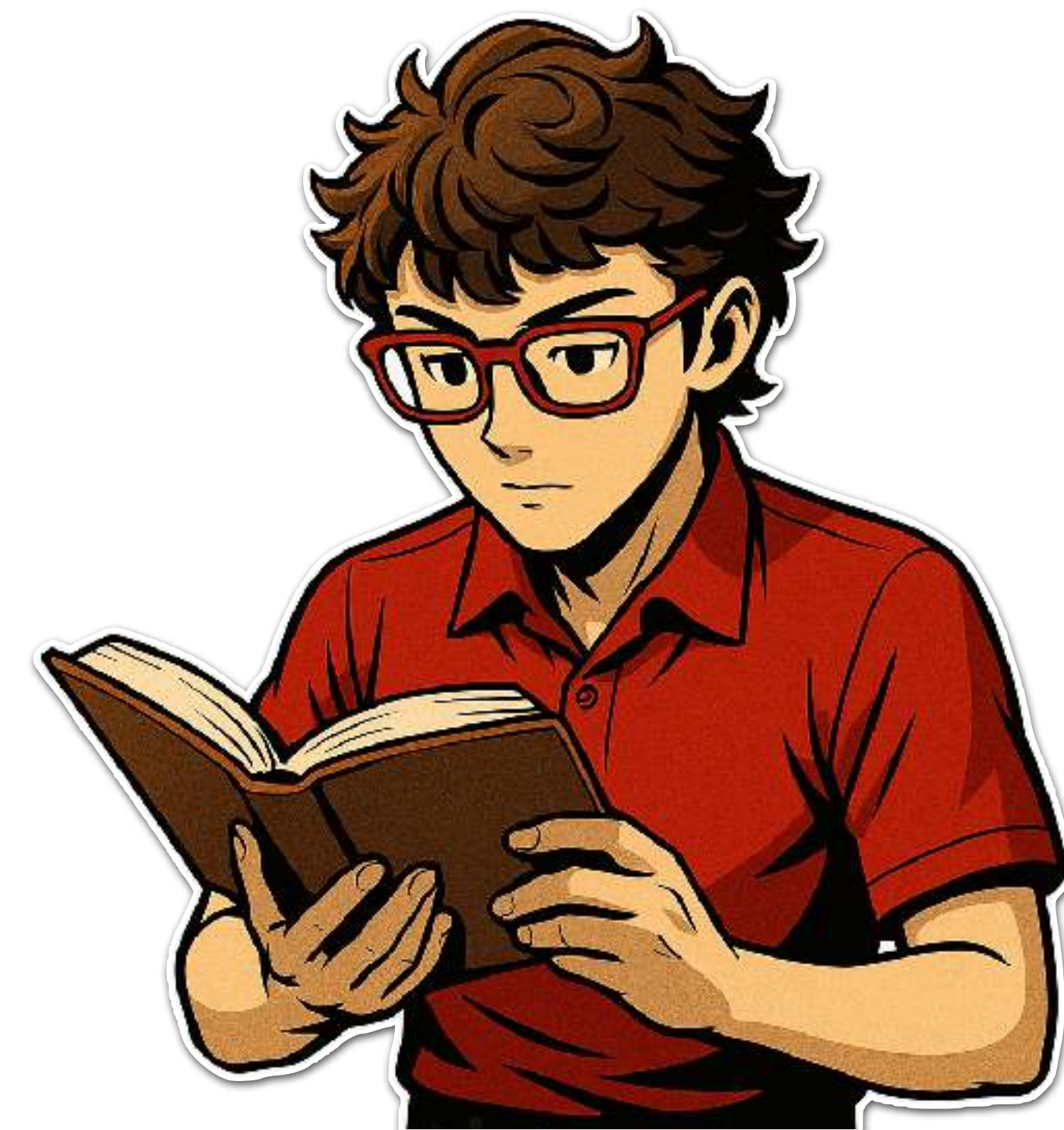
Inference Providers let you run inference on thousands of models served by our partners using a simple, unified, OpenAI-compatible serverless API ([Learn more](#)).

meta-llama/Llama-3.2-1B-Instruct.

Llama-3.2-1B-Instruct-f16.gguf

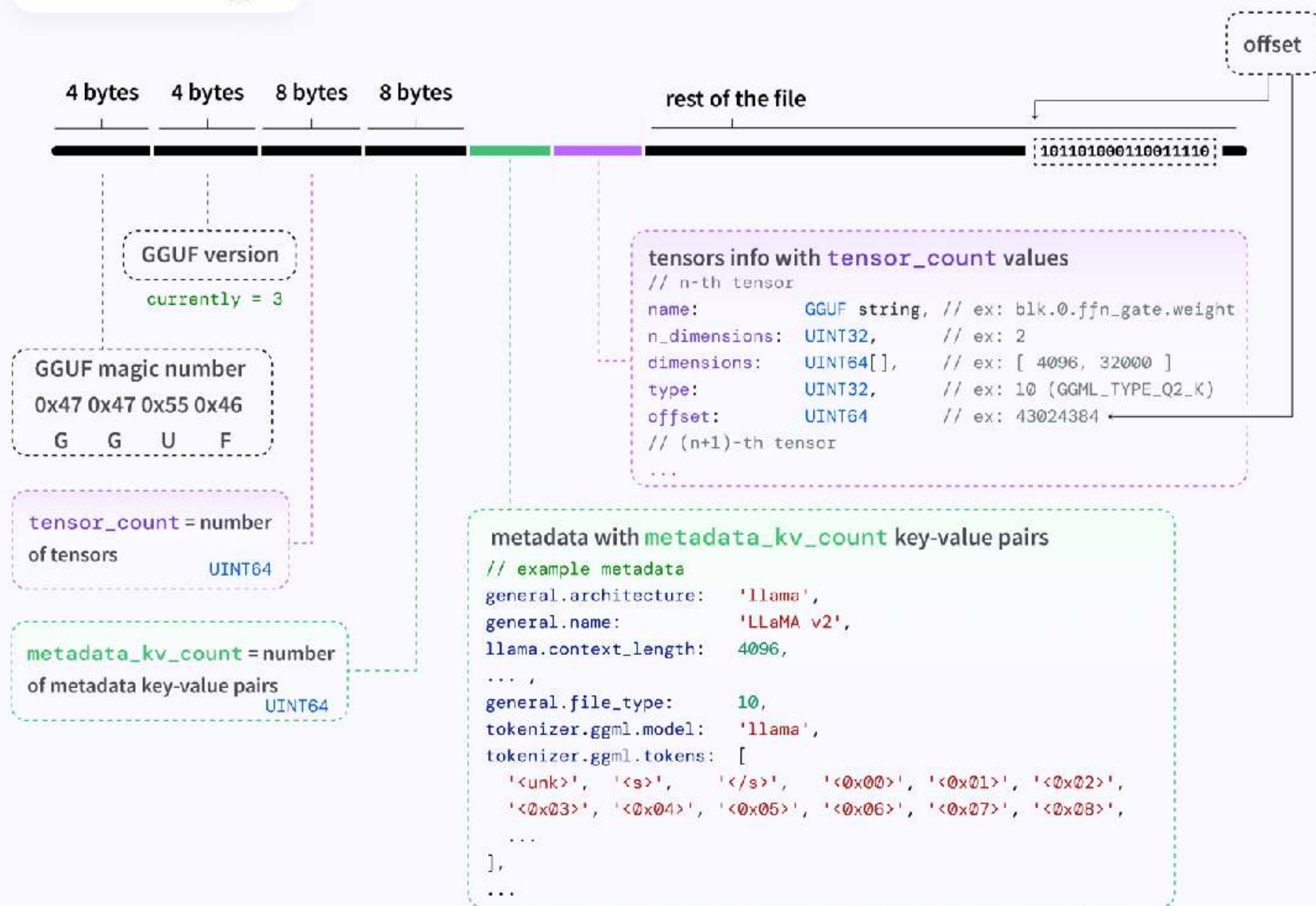
Model is ideal for local benchmarks

- Ultra-small size (~2GB): Runs on any hardware
- F16 precision: High accuracy without quantization artifacts
- Fast inference: 1B params = high tokens/second even on consumer CPUs/GPUs
- Instruction-tuned: Realistic benchmark of chat/agent performance (not raw pretraining)
- Standardized GGUF: Works with all major tools (llama.cpp, Ollama, LM Studio, etc.) for apples-to-apples comparison



Llama-3.2-1B-Instruct-f16.gguf

File name **model.gguf**



GGUF

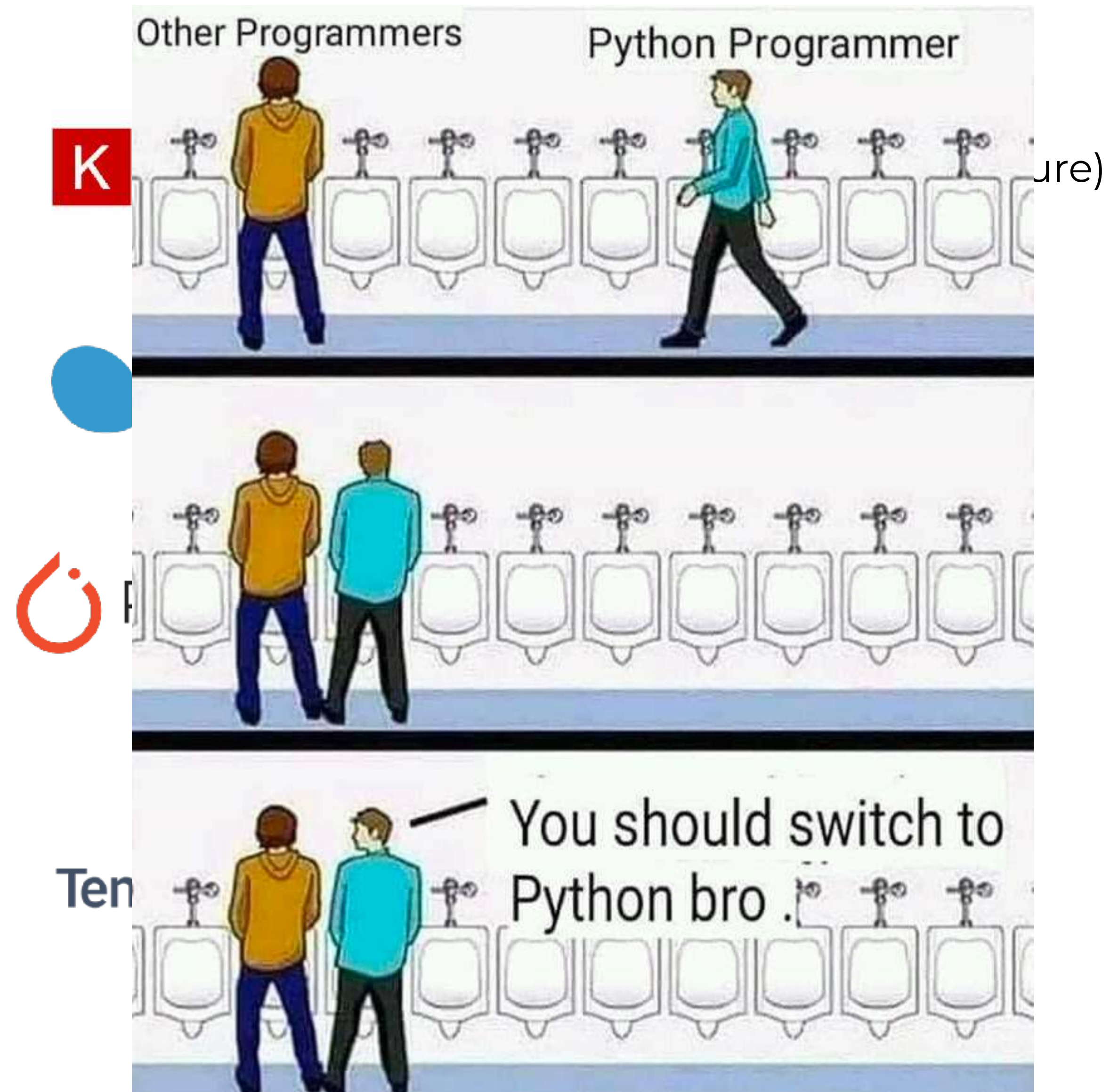
 Keras → .h5 (weights & architecture)

 → .pkl (Pickle)

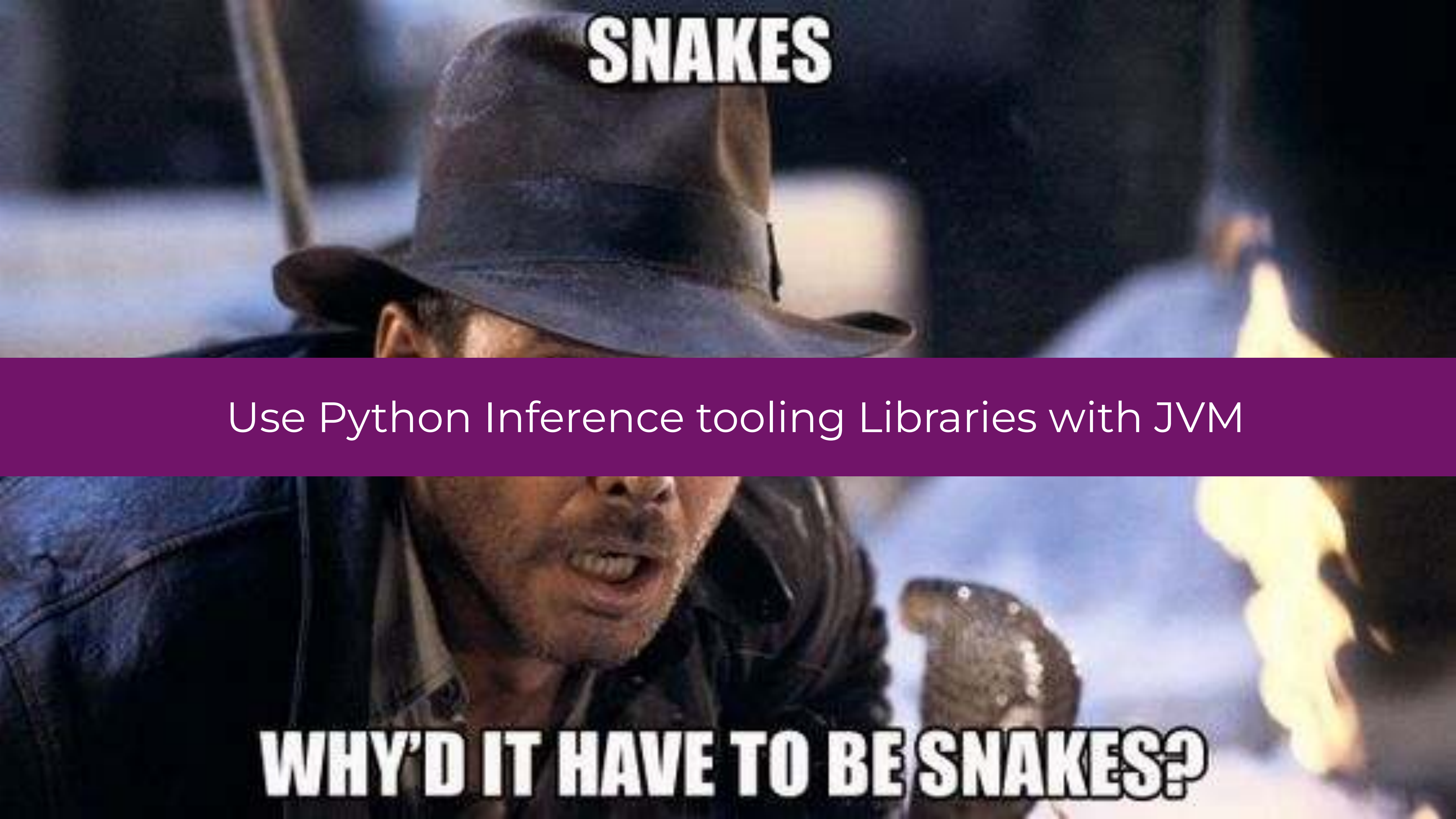
 PyTorch → .pt


TensorFlow → .pb

Different Model Formats



All these frameworks are Python based



SNAKES

Use Python Inference tooling Libraries with JVM

WHY'D IT HAVE TO BE SNAKES?



GraalVM

Build faster, smaller, leaner applications

An advanced JDK with ahead-of-time Native Image compilation



Download

Get Started

GraalVM

One VM to Rule Them All

Thomas Würthinger* Christian Wimmer* Andreas Wöß† Lukas Stadler†
Gilles Duboscq† Christian Humer† Gregor Richards§ Doug Simon* Mario Wolczko*

*Oracle Labs †Institute for System Software, Johannes Kepler University Linz, Austria §S³ Lab, Purdue University
{thomas.wuerthinger, christian.wimmer, doug.simon, mario.wolczko}@oracle.com
{woess, stadler, duboscq, christian.humer}@ssw.jku.at gr@purdue.edu

Abstract

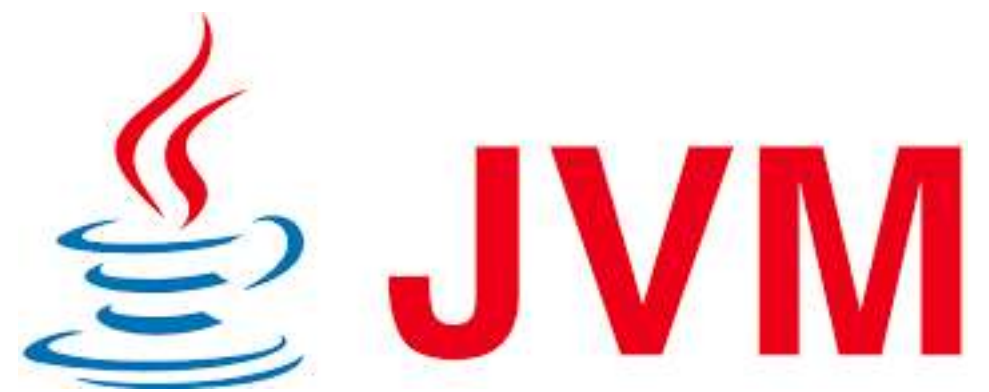
Building high-performance virtual machines is a complex and expensive undertaking; many popular languages still have low-performance implementations. We describe a new approach to virtual machine (VM) construction that amortizes much of the effort in initial construction by allowing new languages to be implemented with modest additional effort. The approach relies on abstract syntax tree (AST) interpretation where a node can rewrite itself to a more specialized or more general node, together with an optimizing compiler that exploits the structure of the interpreter. The compiler uses speculative assumptions and deoptimization in order to produce efficient machine code. Our initial experience suggests that high performance is attainable while preserving a modular and layered architecture and that new high-

These implementations can be characterized in the following way:

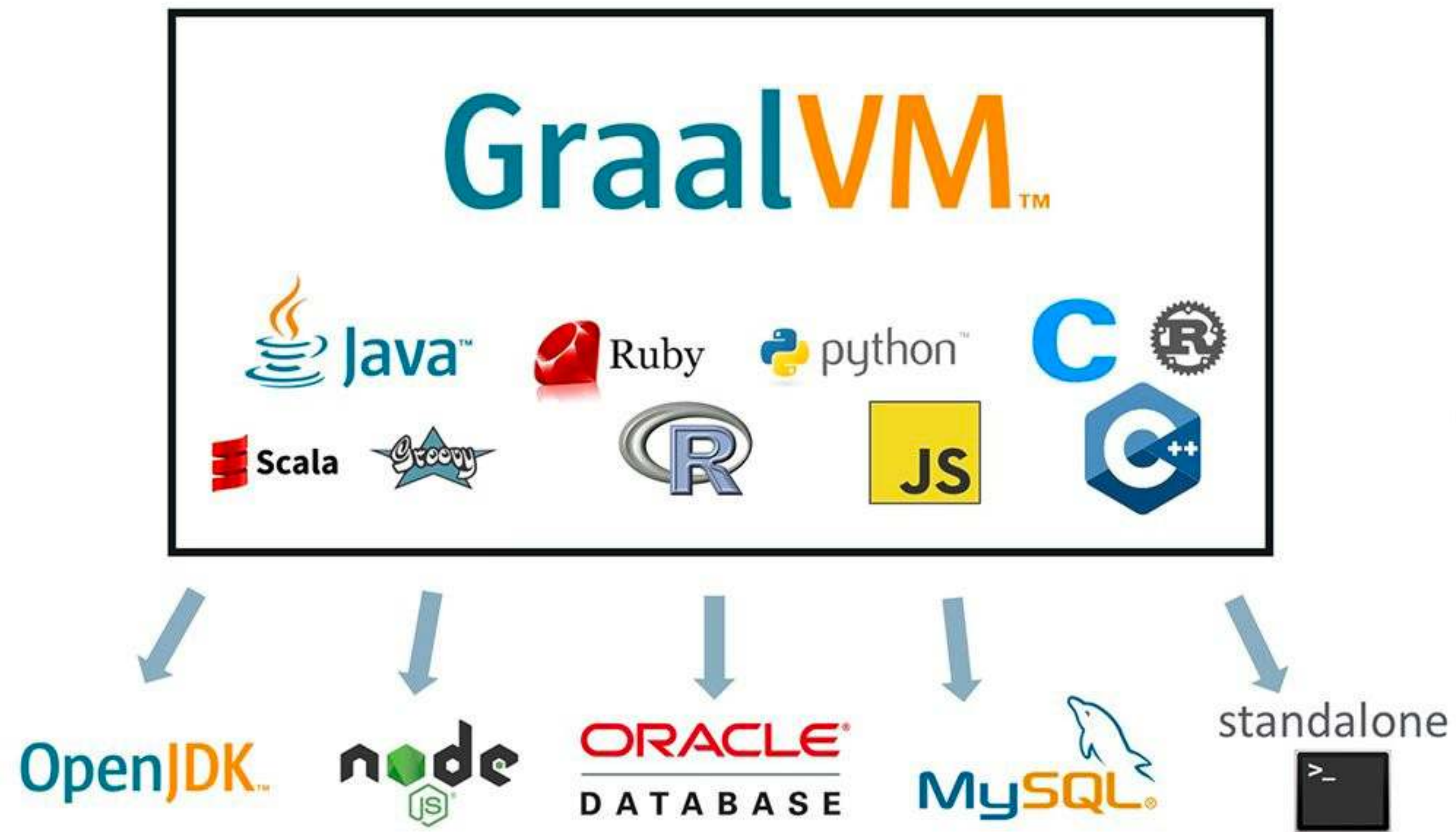
- Their performance on typical applications is within a small multiple (1-3x) of the best statically compiled code for most equivalent programs written in an unsafe language such as C.
- They are usually written in an unsafe, systems programming language (C or C++).
- Their implementation is highly complex.
- They implement a single language, or provide a bytecode interface that advantages a narrow set of languages to the detriment of other languages.

In contrast, there are numerous languages that are nonu-





GraalVM



GraalVM

Self-Optimizing AST Interpreters

Thomas Würthinger^{*} Andreas Wöß[†] Lukas Stadler[†] Gilles Duboscq[†]
Doug Simon^{*} Christian Wimmer^{*}

^{*}Oracle Labs [†]Institute for System Software, Johannes Kepler University Linz, Austria
thomas.wuerthinger@oracle.com woess@ssw.jku.at stadler@ssw.jku.at duboscq@ssw.jku.at
doug.simon@oracle.com christian.wimmer@oracle.com

Abstract

An abstract syntax tree (AST) interpreter is a simple and natural way to implement a programming language. However, it is also considered the slowest approach because of the high overhead of virtual method dispatch. Language implementers therefore define bytecodes to speed up interpretation, at the cost of introducing inflexible and hard to maintain bytecode formats. We present a novel approach to implementing AST interpreters in which the AST is modified during interpretation to incorporate type feedback. This tree rewriting is a general and powerful mechanism to optimize many constructs common in dynamic programming languages. Our system is implemented in JavaTM and uses the static typing and primitive data types of Java elegantly to avoid the cost of boxed representations of primitive values in dynamic programming languages.

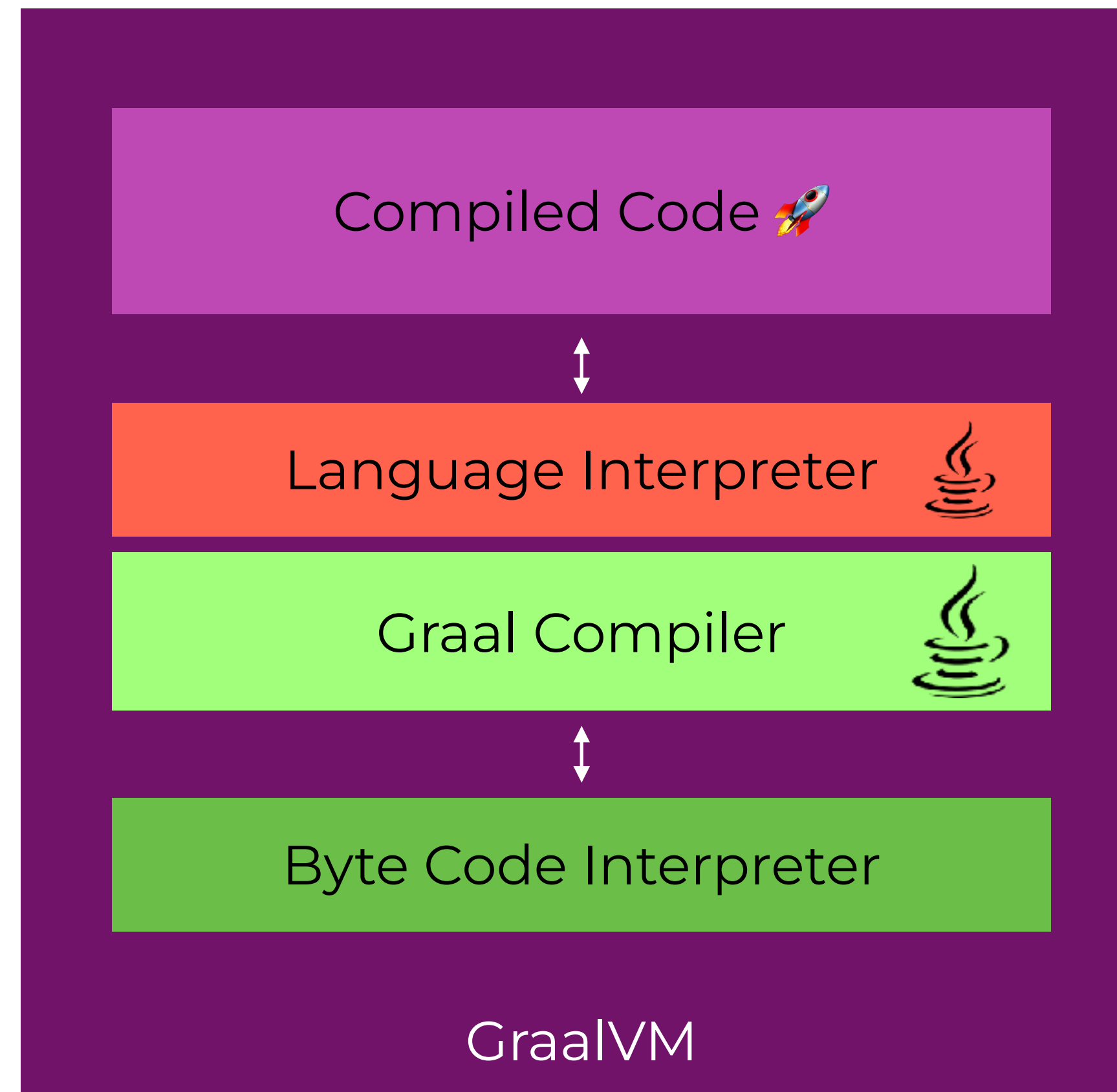
Categories and Subject Descriptors D.3.4 [Programming Languages]: Processors—Run-time environments, Optimization

General Terms Algorithms, Languages, Performance

Keywords Java, JavaScript, dynamic languages, virtual machine, language implementation, optimization

studied [9, 22]. Bytecodes are a virtual instruction set, i.e., they can be defined freely by language implementers without worrying about hardware constraints. There are two different strategies when defining bytecodes. We believe that both are cumbersome when implementing a new language:

- Language-independent bytecodes aim to provide an instruction set that is suitable for many languages. JavaTM bytecodes [17], originally designed primarily for the Java programming language, are now used for many languages. The Common Intermediate Language (CIL) [8], which is used by the .NET system, was specifically designed for multiple languages. Language-independent bytecodes provide only general instructions whose semantics do not match the language semantics exactly. Long bytecode sequences are potentially required to achieve the desired language behavior. For example, Java bytecodes are statically typed, so compiling dynamically typed JavaScript requires frequent type checks. A simple addition in JavaScript requires checking whether the operands are numbers, strings, or arbitrary objects, which all require different handling. Therefore, implementations of JavaScript that run on a Java VM often call a method for a seemingly simple addition.



GraalVM in Truffle Mode

Language Interpreter

GraalPy

GraalJS

GraalRuby

Espresso

GraalVM in Truffle Mode

```

public final class GraalPyFromJava {  @ Artur Skowronski
    public static void main(String[] args) {  @ Artur Skowronski
        System.out.println("[GraalPy] =====");
        System.out.println("[GraalPy] GraalPy Java Host Demo");
        System.out.println("[GraalPy] =====");
        System.out.println("[GraalPy] Java: " + System.getProperty("java.version"));
        System.out.println("[GraalPy] VM: " + System.getProperty("java.vm.name"));
        System.out.println("[GraalPy] OS: " + System.getProperty("os.name") + " " + System.getProperty("os.arch"));
        System.out.println("[GraalPy] =====");
        System.out.println();

        System.out.println("[GraalPy] Creating GraalPy context...");
        long startTime = System.currentTimeMillis();

        try (Context ctx = Context.newBuilder( ...permittedLanguages: "python").build()) {
            long contextTime = System.currentTimeMillis() - startTime;
            System.out.println("[GraalPy] Context created in " + contextTime + "ms");

            System.out.println("[GraalPy] Evaluating: import sys; sys.version");
            Value version = ctx.eval( languageId: "python", source: "import sys; sys.version");
            System.out.println("[GraalPy] python.version=" + version.asString());

            System.out.println("[GraalPy] Evaluating: 1.5 + 2.25");
            Value result = ctx.eval( languageId: "python", source: "1.5 + 2.25");
            System.out.println("[GraalPy] Result: 1.5 + 2.25 = " + result.asDouble());

            long totalTime = System.currentTimeMillis() - startTime;
            System.out.println();
            System.out.println("[GraalPy] Demo completed successfully in " + totalTime + "ms");
        } catch (PolyglotException e) { ... }
    }
}

```

GraalVM in Truffle Mode

```
./gradlew :demos:graalpy-java-host:runLlama
```

```
> Task :demos:graalpy-java-host:run
```

```
[GaalPy] =====
```

```
[GaalPy] GaalPy Java Host Demo
```

```
[GaalPy] =====
```

```
[GaalPy] Java: 25
```

```
[GaalPy] VM: OpenJDK 64-Bit Server VM
```

```
[GaalPy] OS: Mac OS X aarch64
```

```
[GaalPy] =====
```

```
[GaalPy] Creating GaalPy context...
```

```
WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
```

```
WARNING: sun.misc.Unsafe::objectFieldOffset has been called by com.oracle.truffle.api.strings.TStringUnsafe (file:/Users/askowronski/.gradle/caches/modules-2/files-2.1/org.graalvm.truffle/truffle-api/25.0.1/7b3f32fd4b7f0c24607328435652f758791eab56/truffle-api-25.0.1.jar)
```

```
WARNING: Please consider reporting this to the maintainers of class com.oracle.truffle.api.strings.TStringUnsafe
```

DEMO 1

```
[GaalPy] Context created in 235ms
```

```
[GaalPy] Evaluating: import sys; sys.version
```

```
[GaalPy] python.version=3.12.8 (Thu Jan 15 11:43:30 CET 2026)
```

```
[Gaal, Interpreted, Java 25 (aarch64)]
```

```
[GaalPy] Evaluating: 1.5 + 2.25
```

```
[GaalPy] Result: 1.5 + 2.25 = 3.75
```

```
[GaalPy] Demo completed successfully in 1589ms
```

```
BUILD SUCCESSFUL in 2s
```

```
2 actionable tasks: 1 executed, 1 up-to-date
```

```
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.3.0/userguide/configuration\_cache\_enabling.html
```

```
./gradlew :demos:graalpy-java-host:run
```

```
String userInput = args.length > 0 ? args[0] : "Say this is a test";
try (Context context = GraalPyResources.createContext()) {
    CreateChatCompletionFunction createChatCompletion = context.eval("python",
        """
        import os from openai import OpenAI

        client = OpenAI(
            # This is the default and can be omitted
            api_key=os.environ.get("OPENAI_API_KEY"),
        )

        def create_chat_completion(user_input):
            return client.chat.completions.create(
                messages=[
                    {
                        "role": "user",
                        "content": user_input,
                    }
                ],
                model="gpt-3.5-turbo",
            )

        create_chat_completion
        """).as(CreateChatCompletionFunction.class);
    ChatCompletion chatCompletion = createChatCompletion.apply(userInput);
    for (Choice choice : chatCompletion.choices()) {
        System.out.println(choice.message().content());
    }
}
```

GraalVM in Truffle Mode

What's new in Truffle 24.0 and Graal Languages



Alina Yurenko · Following

Published in [graalvm](#) · 5 min read · Mar 19, 2024

GraalPy 🐍

Compatibility ✅

We are working hard to make more and more libraries available on GraalPy. In this release, we expanded support for the following: `tensorflow`, `tensorflow-io`, `protobuf`, `llvmlite`, `pydantic-core`, `catboost`, `ray`, `readme-renderer`, `safetensors`, `keras`, `pybind11`, `grpcio`, `Py03`, `cryptography`, `bcrypt`, `cramjam`, `libcst`, `orjson`, `rpds_py`. Try them out, provide feedback, and let us know what other libraries you would like to run on GraalPy.

Go a Bit More Complex: GraalPy

GraalPy Compatibility

- GraalPy supports the C extensions API
- One can pip install and run many well-known packages such as NumPy, Pandas, PyTorch, TensorFlow, and ML models from Huggingface.co
- We run the test suites of the top 600 PyPI packages every day
- To best of our knowledge we are the most compatible alternative implementation of Python to date
- GraalPy is included in manylinux images
- GraalPy is available in GitHub action setup-python

8

Copyright © 2024, Oracle and/or its affiliates

10/07/2024



Compatibility

Compatibility per GraalPy Release

GraalPy 25.0

GraalPy 24.2

GraalPy 24.1

Over 600 Python packages tested for compatibility with GraalPy



✓ Compatible: 480 (61.15%) ⓘ

✗ Currently Incompatible: 140 (17.83%) ⓘ

ⓘ Currently Untested: 165 (21.02%) ⓘ

● Not Supported: 0 (0%) ⓘ



Many more Python packages work on GraalPy than are listed here. If there is a package you are interested in that is not included, chances are that it might just work. If it does not, feel free to create an issue for us on [GitHub](#).

Go a Bit More Complex: GraalPy



Numeric Computing

We test **NumPy** across multiple versions and know of multiple deployments where it brings numeric computing to Java.



Scientific Computing

SciPy's rich library for scientific computing is just a package download away.



Data Processing

Thanks to Arrow, **Pandas** on GraalPy can run multi-threaded while avoiding unnecessary data copies.



Models for any Task

The **Huggingface** transformers library works on GraalPy with its huge library of language, vision, and audio models.



Training and Inference

Train models and run inference on GraalPy with **PyTorch**, taking full advantage of the latest techniques and accelerator hardware.



Agentic Workflows

With **Autogen** and GraalPy you can write agentic workflows and use Java code to create tools for AI Agents.

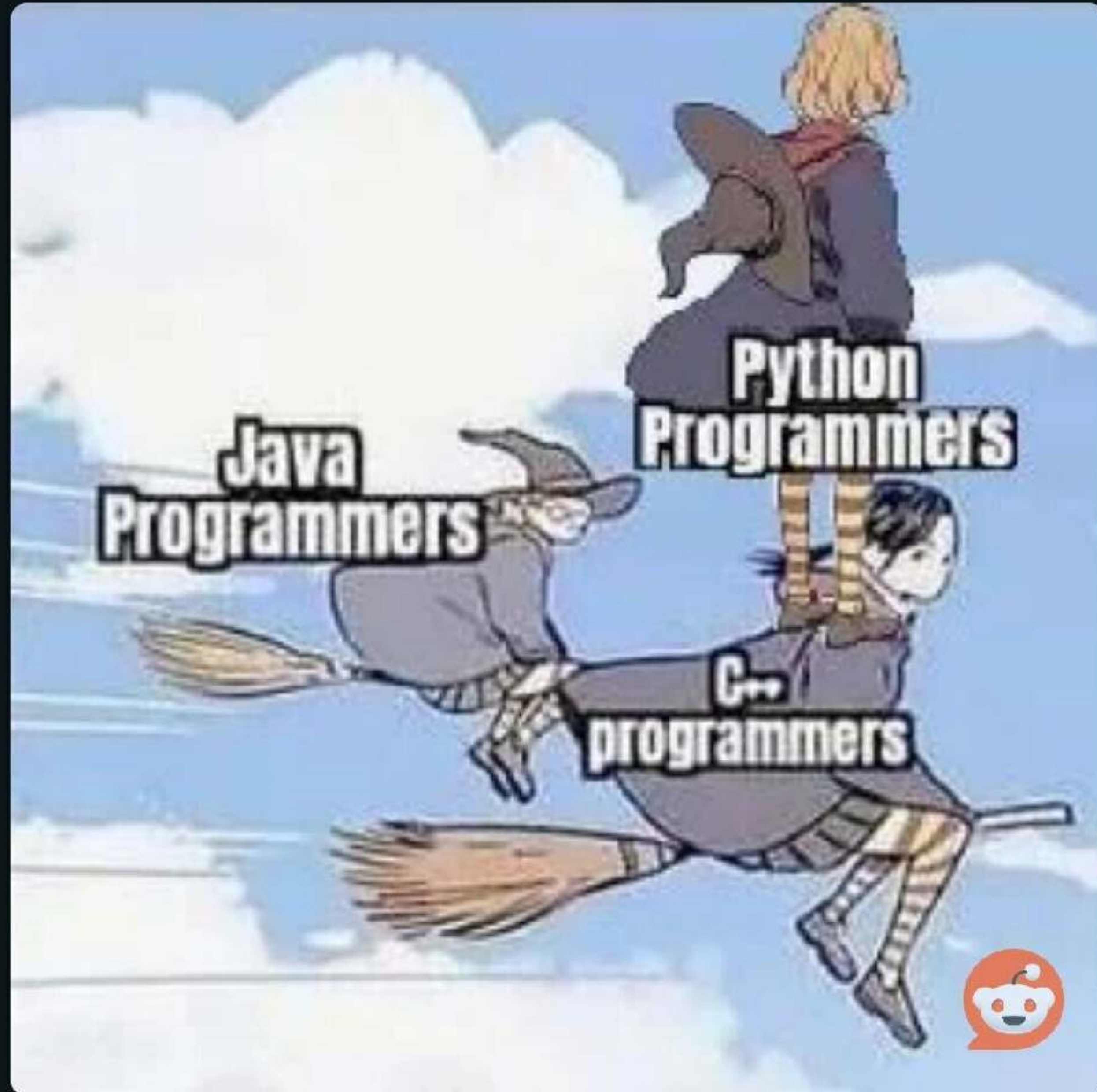
Compatibility



Truffle & GraalPy



From **ProgrammerHumor** community on **Reddit**



Native Extensions Support

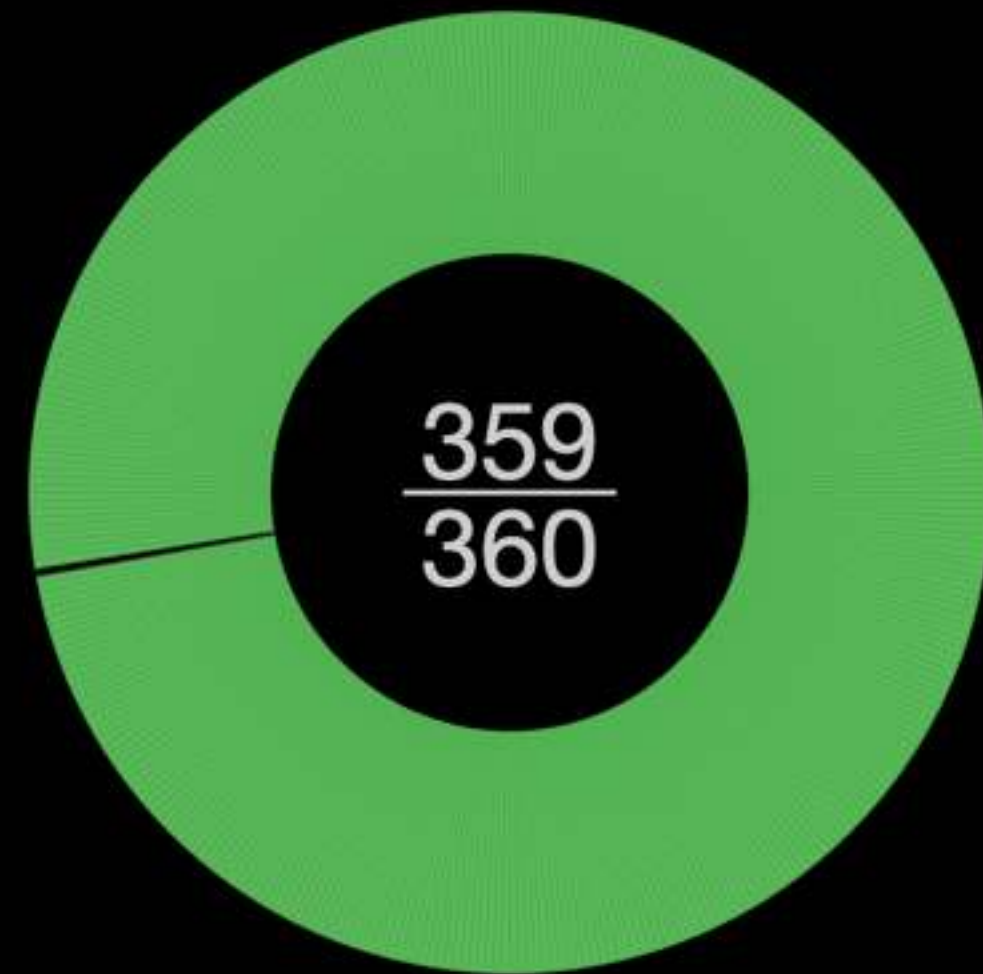
CPython provides a [native extensions API](#) for writing Python extensions in C/C++. GraalPy provides experimental support for this API, which allows many packages like NumPy and PyTorch to work well for many use cases. The support extends only to the API, not the binary interface (ABI), so extensions built for CPython are not binary compatible with GraalPy. Packages that use the native API must be built and installed with GraalPy, and the prebuilt wheels for CPython from pypi.org cannot be used. For best results, it is crucial that you only use the `pip` command that comes preinstalled in GraalPy virtualenvs to install packages. The version of `pip` shipped with GraalPy applies additional patches to packages upon installation to fix known compatibility issues and it is preconfigured to use an additional repository from [graalvm.org](#) where we publish a selection of prebuilt wheels for GraalPy. Please do not update `pip` or use alternative tools such as `uv`.

Embedding limitations

Python native extensions run by default as native binaries, with full access to the underlying system. Native code is entirely unrestricted and can circumvent any security protections Truffle or the JVM may provide. Native data structures are not subject to the Java GC and the combination of them with Java data structures may lead to memory leaks. Native libraries generally cannot be loaded multiple times into the same process, and they may contain global state that cannot be safely reset. Thus, it is not possible to create multiple GraalPy contexts that access native modules within the same JVM. This includes the case when you create a context, close it, and then create another context. The second context will not be able to access native extensions.

Go a Bit More Complex: GraalPy

Python Wheels



What are wheels?

Wheels are the new standard of Python distribution and are intended to replace eggs. Support is offered in `pip >= 1.4` and `setuptools >= 0.8`.

Advantages of wheels

1. Faster installation for pure Python and native C extension packages.
2. Avoids arbitrary code execution for installation. (Avoids `setup.py`)
3. Installation of a C extension does not require a compiler on Linux, Windows or macOS.
4. Allows better caching for testing and continuous integration.
5. Creates `.pyc` files as part of installation to ensure they match the Python interpreter used.
6. More consistent installs across platforms and machines.

What is this list?

This site shows the top 360 most-downloaded packages on [PyPI](#) showing which have been uploaded as wheel archives.

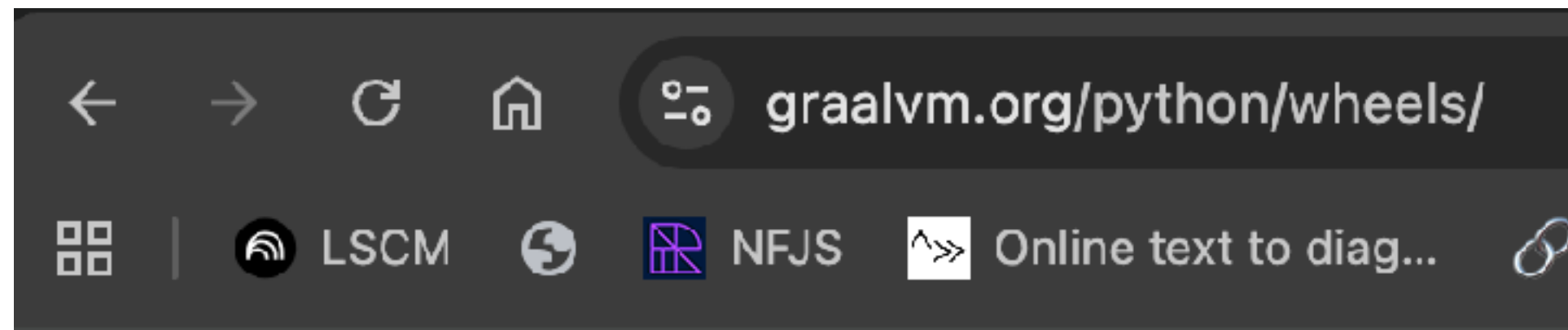
- Green packages offer wheels,
- Uncoloured packages have no wheel archives uploaded (yet!).

boto3 ✓
urllib3 ✓
botocore ✓
packaging ✓
certifi ✓
typing-extensions ✓
requests ✓
setuptools ✓
idna ✓
charset-normalizer ✓
aiobotocore ✓

Python wheels are pre-built binary package files (.whl) containing compiled C extensions, Python bytecode, and metadata for fast installation without source compilation.

pydantic ✓
fsspec ✓
s3fs ✓
pycparser ✓
pluggy ✓
protobuf ✓
click ✓
pygments ✓
pandas ✓

Python Wheels



[contourpy](#)
[httptools](#)
[jiter](#)
[kiwisolver](#)
[numpy](#)
[oracledb](#)
[psutil](#)
[pydantic-core](#)
[ujson](#)
[uvloop](#)
[watchfiles](#)
[xxhash](#)
[polyleven](#)

```
python -m pip install \  
  --no-cache-dir \  
  --extra-index-url https://www.graalvm.org/python/wheels/ \  
  numpy|
```



Python Wheels

```

=====
GraalPy Llama Inference Demo
=====
Python: 3.13.0 (main, Jul  9 2025, 07:20:15) [Clang 17.0.0 (clang-1700.0.13.5)]
Executable: /Users/askowronski/.pyenv/versions/3.13.0/bin/python3
Platform: macOS-26.2-arm64-arm-64bit-Mach-0
=====

Loading model: /Users/askowronski/.llama/models/Llama-3.2-1B-Instruct-f16.gguf
Using 10 CPU threads
llama_context: n_ctx_per_seq (2048) < n_ctx_train (131072) -- the full capacity of the model will not be utilized
ggml_metal_init: skipping kernel_get_rows_bf16 (not supported)
ggml_metal_init: skipping kernel_set_rows_bf16 (not supported)
ggml_metal_init: skipping kernel_mul_mv_bf16_f32 (not supported)
ggml_metal_init: skipping kernel_mul_mv_bf16_f32_c4 (not supported)
ggml_metal_init: skipping kernel_mul_mv_bf16_f32_1row (not supported)
ggml_metal_init: skipping kernel_mul_mv_bf16_f32_14 (not supported)
ggml_metal_init: skipping kernel_mul_mv_bf16_bf16 (not supported)
ggml_metal_init: skipping kernel_mul_mv_id_bf16_f32 (not supported)
ggml_metal_init: skipping kernel_mul_mm_bf16_f32 (not supported)
ggml_metal_init: skipping kernel_mul_mm_id_bf16_f16 (not supported)
ggml_metal_init: skipping kernel_flash_attn_ext_bf16_h64 (not supported)
ggml_metal_init: skipping kernel_flash_attn_ext_bf16_h80 (not supported)
ggml_metal_init: skipping kernel_flash_attn_ext_bf16_h96 (not supported)
ggml_metal_init: skipping kernel_flash_attn_ext_bf16_h112 (not supported)
ggml_metal_init: skipping kernel_flash_attn_ext_bf16_h128 (not supported)
ggml_metal_init: skipping kernel_flash_attn_ext_bf16_h192 (not supported)
ggml_metal_init: skipping kernel_flash_attn_ext_bf16_hk192_hv128 (not supported)

```

DEMO 2

```

ggml_metal_init: skipping kernel_cpy_bf16_f32 (not supported)
ggml_metal_init: skipping kernel_cpy_bf16_f32 (not supported)
ggml_metal_init: skipping kernel_cpy_bf16_bf16 (not supported)
Model loaded in 2.95s

Prompt: Tell me a short joke about programming.
-----
Response:
/Users/askowronski/.pyenv/versions/3.13.0/lib/python3.13/site-packages/llama_cpp/llama.py:1242: RuntimeWarning: Detected duplicate leading "<|begin_of_text|>" in prompt, this will likely reduce response quality, consider removing it...
  warnings.warn(
Why do programmers prefer dark mode? Because light attracts bugs.
-----

Stats:
Inference time: 0.72s
Prompt tokens: 30
Completion tokens: 12
Total tokens: 42
Tokens/sec: 16.70

```

python3 llama_inference.py --prompt "Tell me a short joke about programming."

```
try (Context ctx = Context.newBuilder("python")
    .allowIO(IOAccess.ALL) // Allow file access
    .allowNativeAccess(true) // Allow C extensions (llama.cpp)
    .option("python.PythonPath", venvSitePackages) // Add venv to path
    .build()) {

    System.out.println("[GraalPy] Context initialized. Importing llama_inference...");

    ctx.eval("python", "import sys; sys.path.append('" +
        Paths.get(projectRoot, "demos", "graalpy-llama").toString() + "')");

    Value llamaModule = ctx.eval("python", "import llama_inference; llama_inference");

    System.out.println("[GraalPy] Module imported. Running inference...");
    Value runInference = llamaModule.getMember("run_inference");
    runInference.execute(modelPath, "Tell me a short joke about Java.", 64, 0.7);

} catch (PolyglotException e) {
    System.err.println("[GraalPy] Polyglot Exception: " + e.getMessage());
    e.printStackTrace();
}
```



CTypes Limitations

```
~/Priv/talks/conference-jvm-in-age-ai-2026 git:(main) 14 files changed, 227 insertions(+), 128 deletions(-) (16.415s)
```

```
cd /Users/askowronski/Priv/talks/conference-jvm-in-age-ai-2026/demos/graalpy-llama
```

```
./scripts/run-llama.sh --prompt "Tell me a short joke about programming."
```

```
Using GraalPy: /Users/askowronski/.graalpy/graalpy-jvm-25.0.1-macos-aarch64/bin/graalpy
```

```
GraalPy 3.12.8 (Oracle GraalVM JVM 25.0.1)
```

```
=====
GraalPy Llama Inference Demo
=====
```

```
Python: 3.12.8 (Thu Oct 09 19:52:16 CEST 2025)
```

```
Executable: /Users/askowronski/Priv/talks/conference-jvm-in-age-ai-2026/demos/graalpy-llama/.venv/bin/python
```

```
Platform: macOS-26.2-arm64-arm-64bit
=====
```

DEMO 3

```
run_inference(args.model, args.prompt, args.max_tokens, args.temperature)
```

```
File "/Users/askowronski/Priv/talks/conference-jvm-in-age-ai-2026/demos/graalpy-llama/llama_inference.py", line 52, in run_inference
```

```
    llm = Llama(
```

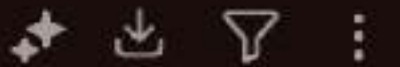
```
File "/Users/askowronski/Priv/talks/conference-jvm-in-age-ai-2026/demos/graalpy-llama/.venv/lib/python3.12/site-packages/llama_cpp/llama.py", line 225, in __init__
```

```
    self.model_params = llama_cpp.llama_model_default_params()
```

```
SystemError: ctypes: returning struct by value is not supported.
```

CTypes Limitations

```
~/Priv/talks/conference-jvm-in-age-ai-2026 git:(main) 14 files changed, 227 insertions(+), 128 deletions(-) (16.415s)
```



```
cd /Users/askowronski/Priv/talks/conference-jvm-in-age-ai-2026/demos/graalpy-llama
```

```
./scripts/run-llama.sh --prompt "Tell me a short joke about programming."
```

```
Using GraalPy: /Users/askowronski/.graalpy/graalpy-jvm-25.0.1-macos-aarch64/bin/graalpy
```

```
GraalPy 3.12.8 (Oracle GraalVM JVM 25.0.1)
```

```
=====
GraalPy Llama Inference Demo
=====
```

```
Python: 3.12.8 (Thu Oct 09 19:52:16 CEST 2025)
```

```
Executable: /Users/askowronski/Priv/talks/conference-jvm-in-age-ai-2026/demos/graalpy-llama/.venv/bin/python
```

```
Platform: macOS-26.2-arm64-arm-64bit
=====
```

```
Loading model: /Users/askowronski/.llama/models/Llama-3.2-1B-Instruct-f16.gguf
```

```
Using 10 CPU threads
```

```
Traceback (most recent call last):
```

```
File "/Users/askowronski/Priv/talks/conference-jvm-in-age-ai-2026/demos/graalpy-llama/llama_inference.py", line 152, in <module>
    main()
```

```
File "/Users/askowronski/Priv/talks/conference-jvm-in-age-ai-2026/demos/graalpy-llama/llama_inference.py", line 148, in main
    run_inference(args.model, args.prompt, args.max_tokens, args.temperature)
```

```
File "/Users/askowronski/Priv/talks/conference-jvm-in-age-ai-2026/demos/graalpy-llama/llama_inference.py", line 52, in run_inference
    llm = Llama()
```

```
File "/Users/askowronski/Priv/talks/conference-jvm-in-age-ai-2026/demos/graalpy-llama/.venv/lib/python3.12/site-packages/llama_cpp/llama.py", line 225, in __init__
    self.model_params = llama_cpp.llama_model_default_params()
```

```
SystemError: ctypes: returning struct by value is not supported.
```

CTypes Limitations



```
File "llama_inference.py", line 52, in run_inference
  llm = Llama(
File ".venv/lib/python3.12/site-packages/llama_cpp/llama.py", line 225, in init
  self.model_params = llama_cpp.llama_model_default_params()
SystemError: ctypes: returning struct by value is not supported.
```

What is happening technically

llama_cpp uses Python's ctypes module to call functions from the native llama.cpp C library.

The C function `llama_model_default_params()` returns a C struct by value (not a pointer).

GraalPy's implementation of ctypes does not support "struct returned by value", so the call fails with `SystemError`.

CTypes Limitations



Artur Skowroński Jan 16th at 10:49 PM

Hi team!

I'm trying to run llama-cpp-python on GraalPy and hitting a ctypes limitation. Would appreciate any guidance on workarounds or if this is on the roadmap.

Environment:

- GraalPy JVM 25.0.1 (macOS ARM64)
- Python 3.12.8
- llama-cpp-python 0.3.16

Error:

```
File "llama_inference.py", line 52, in run_inference
    llm = Llama(
File ".venv/lib/python3.12/site-packages/llama_cpp/llama.py", line 225, in init
    self.model_params = llama_cpp.llama_model_default_params()
SystemError: ctypes: returning struct by value is not supported.
```

The `llama_model_default_params()` function in `llama.cpp` returns a `llama_model_params` struct by value. This works fine in CPython but fails in GraalPy.

I noticed the Truffle [NFI docs](#) list supported types (VOID, SINT*, UINT*, FLOAT, DOUBLE, POINTER, STRING, etc.) but structs aren't mentioned. Is there a workaround for this limitation?

Thanks for any help! 🙏 (edited)

🔗 graalvm.org

Truffle Native Function Interface

GraalVM is an advanced JDK with ahead-of-time Native Image compilation. (74 kB) ▾



Michael Šimáček Jan 19th at 9:16 AM

Hi. The limitation should be fixed in the upcoming feature release. You can try a snapshot build at <https://github.com/graalvm/graalvm-ce-dev-builds/releases/>

CTypes Limitations

Support Hugging Face Transformers #276

New issue



Closed



conker84 opened on Aug 10, 2022

...

Is there any plan to support Hugging Face Transformers? They are the next (maybe current) big thing in Python. I'm trying to make it work right now but I'm getting the various errors and one of them is:

com.oracle.truffle.api.CompilerDirectives\$ShouldNotReachHere: writeByte not implemented

```
Traceback (most recent call last):
  File "pip", line 8, in <module 'pip'>
  File "main.py", line 70, in main
  File "base_command.py", line 101, in main
  File "base_command.py", line 221, in _main
  File "base_command.py", line 167, in exc_logging_wrapper
  File "req_command.py", line 247, in wrapper
  File "install.py", line 369, in run
  File "resolver.py", line 92, in resolve
  File "resolvers.py", line 481, in resolve
  File "resolvers.py", line 348, in resolve
  File "resolvers.py", line 172, in _add_to_criteria
  File "structs.py", line 151, in __bool__
  File "functools", line 590, in wrapper
  File "found_candidates.py", line 155, in __bool__
  File "found_candidates.py", line 143, in <genexpr>
  File "found_candidates.py", line 44, in _iter_built
  File "factory.py", line 279, in iter_index_candidate_infos
  File "functools", line 566, in wrapper
  File "package_finder.py", line 889, in find_best_candidate
  File "functools", line 566, in wrapper
  File "package_finder.py", line 830, in find_all_candidates
  File "sources.py", line 134, in page_candidates
  File "package_finder.py", line 790, in process_project_url
  File "collector.py", line 577, in fetch_response
  File "collector.py", line 481, in _get_index_content
  File "collector.py", line 138, in _get_simple_response
  File "sessions.py", line 600, in get
  File "sessions.py", line 518, in request
```



Assignees

No one assigned

Labels

No labels

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

Code with agent mode

No branches or pull requests

Notifications

Customize

Subscribe

You're not receiving notifications from this thread.

CTypes Limitations

Support Hugging Face Transformers #276

New issue



Closed



conker84 opened on Aug 10, 2022

...

Is there any plan to support Hugging Face Transformers? They are the next (maybe current) big thing in Python I'm trying to make it work right now but I'm getting the various errors and one of them is:

com.oracle.truffle.api.CompilerDirectives\$ShouldNotReachHere: writeByte not implemented

Traceback (most recent call last):
File "pip", line 8, in <module 'pip'>



msimacek on Jul 31, 2024

Contributor

...

Many examples with `transformers` now work on GraalPy.

Unfortunately, all the dependencies have to be built from source during the installation, which is slow and requires you to have the right compiler and libraries in the system. Hopefully, in the future we will have prebuilt binary wheels for them.



msimacek closed this as completed on Jul 31, 2024

```
"functools", line 566, in wrapper
File "package_finder.py", line 889, in find_best_candidate
"functools", line 566, in wrapper
File "package_finder.py", line 830, in find_all_candidates
File "sources.py", line 134, in page_candidates
File "package_finder.py", line 790, in process_project_url
File "collector.py", line 577, in fetch_response
File "collector.py", line 481, in _get_index_content
File "collector.py", line 138, in _get_simple_response
File "sessions.py", line 600, in get
File "sessions.py", line 518, in request
```

Assignees

No one assigned

Labels

No labels

Type

No type

Code with agent mode



No branches or pull requests

Notifications

Customize



Subscribe

You're not receiving notifications from this thread.

CTypes Limitations

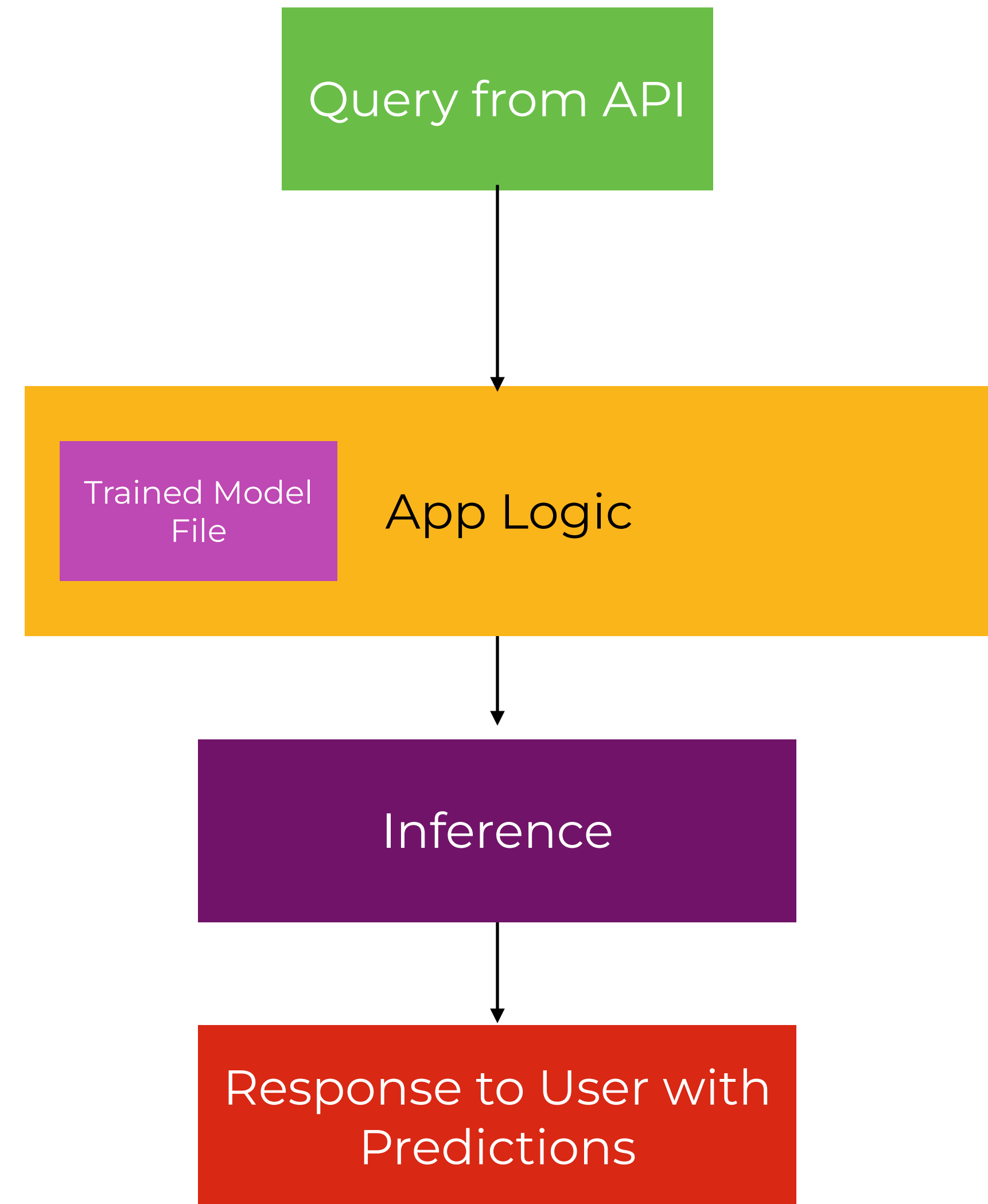


Shortcuts don't help here



Inference of model from Java





Model Inference

HOW STANDARDS PROLIFERATE:
(SEE: A/C CHARGERS, CHARACTER ENCODINGS, INSTANT MESSAGING, ETC)

SITUATION:
THERE ARE
14 COMPETING
STANDARDS.

14?! RIDICULOUS!
WE NEED TO DEVELOP
ONE UNIVERSAL STANDARD
THAT COVERS EVERYONE'S
USE CASES.



YEAH!

SOON:

SITUATION:
THERE ARE
15 COMPETING
STANDARDS.

HOW STANDARDS PROLIFERATE:

(SEE: A/C CHARGERS, CHARACTER ENCODINGS, INSTANT MESSAGING, ETC)

SITUATION:
THERE ARE
14 COMPETING
STANDARDS.

14?! RIDICULOUS!
WE NEED TO DEVELOP
ONE UNIVERSAL STANDARD
THAT COVERS EVERYONE'S
USE CASES.



SOON:

There are
compatible
implementations
of the common
standards



API

JPA
(Java Persistence API)



JDBC
(Java Database Connectivity)



JAX-RS
(Java API for RESTful Web Services)



Servlet API



JSR (Java Specification Request)

- **JSR 381 – Visual Recognition (VisRec) API**
- JSR 376 – Java Platform Module System (Project Jigsaw)
- JSR 383 – Java SE 18.3 (Java 10)
- JSR 315 – Java Servlet 3.0
- JSR 380 – Bean Validation 2.0
- JSR 244 – Java EE 5
- JSR 303 – Bean Validation 1.0
- JSR 907 – JDBC 4.0
- JSR 133 – Java Memory Model and Thread Specification
- JSR 292 – Dynamically Typed Languages Support (invokedynamic)

Interesting JSR



DJL

DeepJavaLibrary

JSR 381 specifically focuses on providing standardized APIs for deep learning frameworks in Java, enabling developers to work with deep learning models in a consistent and interoperable manner. This standardization helps foster innovation, promote compatibility between different deep learning libraries, and streamline the development of deep learning applications within the Java ecosystem.

Key objectives of the JSR 381

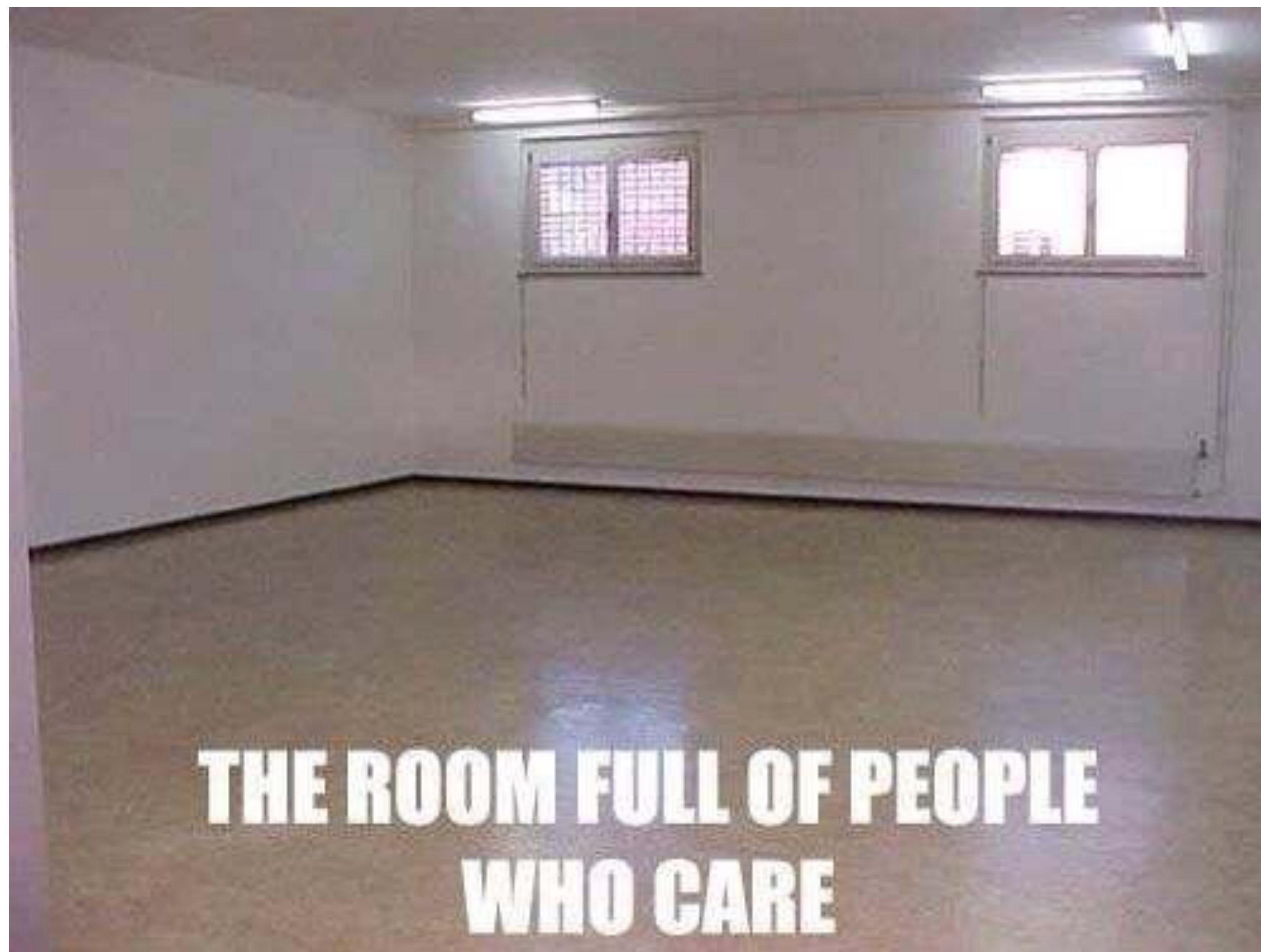
Key objectives of the standard Java API for machine learning include:

- Defining common APIs for tasks such as building, training, evaluating, and deploying deep learning models.
- Ensuring compatibility and interoperability between different deep learning frameworks and tools in Java.
- Providing a consistent programming interface for developers to work with deep learning models, regardless of the underlying framework being used.
- Promoting best practices and standard patterns for deep learning development in Java.
- Facilitating the integration of deep learning capabilities into existing Java applications and frameworks.

The creation of JSR 381 reflects the growing importance of deep learning in Java and the need for standardized APIs to support its adoption and development within the Java community. By establishing a common set of interfaces and specifications, JSR 381 aims to simplify the development and deployment of deep learning solutions in Java, ultimately benefiting developers, organizations, and the broader Java ecosystem.

DeepJavaLibrary

- Examples
 - Example 1 - Hello World
 - Example 2 - Simple Linear Regression
 - Example 3 - Logistic Regression
 - Example 4 - Iris Flower Classification
 - Example 5 - Handwritten Digit Recognition using the MNIST Data Set



Visual Recognition (VisRec) JSR #381

ty w dokumencie

MicroProfile AI

MicroProfile AI minutes

24th March 2025

Contributors [each atte...

Minutes:

17th March 2025

Contributors [each atte...

Minutes:

10th March 2025

Contributors [each atte...

Minutes:

3rd March 2025

Contributors [each atte...

Minutes:

24th Feb 2025

MicroProfile AI minutes

24th March 2025

Video Recording

Contributors [each attendee must add themself to minutes]

- Emily Jiang (IBM)
- Emmanuel Hugonnet (Red Hat)
- Don Bourne (IBM)
- Phil Chung (Oracle)
- Buhake Sindi (Individual)

Minutes:

- Our repo has been created [here](#)
 - Populate the repo
 - Group ID dev.langchain4j.microprofile.jakarta
 - Artifact ID langchain4j-microprofile-jakarta-parent
 - Update the cdi integration
 - Langchain4j-cdi-buildcompatible-extension
 - Langchain4j-cdi-portable-extension
 - Revisit the name at a later time
 - Build infrastructure - deploy and release to maven central
 - Emmanuel and Clement would be able to get the infrastructure done with the help from [Dmytro Liubarskyi](#)
 - Emmanuel will set up OSSH username and password for the Github actions

Korzystasz obecnie z konta articles.pl@gmail.com

Artur Skowroński

Zmień konto

OK

Microprofile AI

Jakarta Agentic Artificial Intelligence

[Project](#)[Website](#)

The Jakarta Agentic AI project provides a set of vendor-neutral APIs that make it easy, consistent, and reliable to build, deploy, and run AI agents on Jakarta EE runtimes.

- **Jakarta Agentic Artificial Intelligence 1.0 (under development)**

The Jakarta Agentic AI project provides a set of ...

Artificial Intelligence (AI) agents are one of the most prominent developments in enterprise and cloud native computing in decades. They promise to fundamentally accelerate innovation, automation, and productivity by leveraging AI in virtually every industry – IT, finance, banking, retail, manufacturing, health care, and so many others. Agents operate by leveraging Neural Networks, Machine Learning (ML), Natural Language Processing (NLP), Large Language Models (LLMs), and many other AI technologies to aim to perform specific tasks autonomously with little or no human intervention. They detect events, gather data, generate self-correcting plans, execute actions, process results, and evolve subsequent decisions. Examples include self-driving cars, security monitors, Site Reliability Engineering (SRE) agents, stock monitors, code/application generators, health monitors, customer service agents, manufacturing robots, and many others.

The Jakarta Agentic AI project provides a set of vendor-neutral APIs that make it easy, consistent, and reliable to build, deploy, and run AI agents on Jakarta EE runtimes. The project aims to do for developing AI agents what Servlet did for HTTP processing, Jakarta REST did for RESTful web services, or perhaps most appropriately, Jakarta Batch did for batch processing.

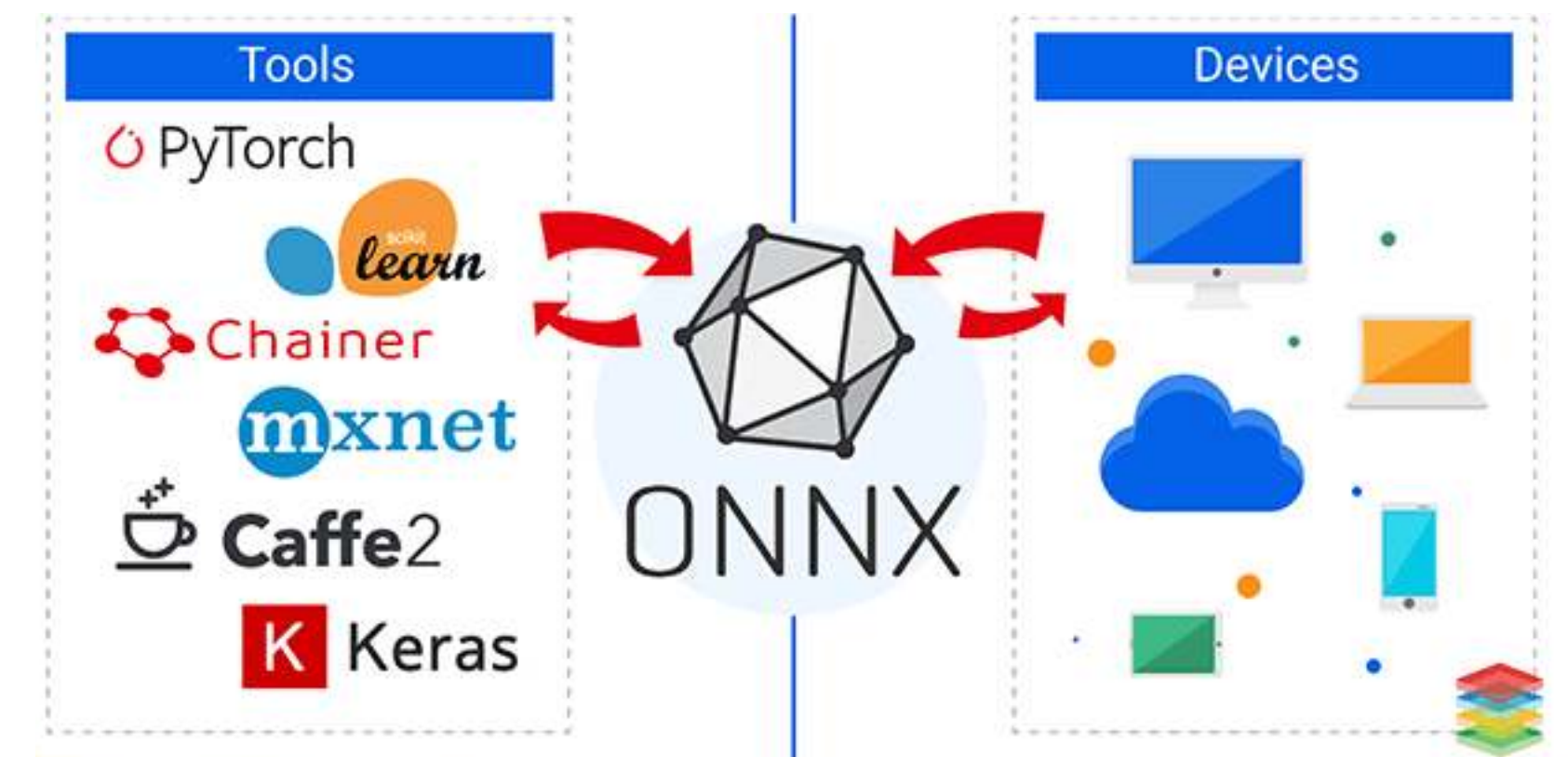
Jakarta Agentic AI

 Keras → .h5 (weights & architecture)

 → .pkl (Pickle)

 PyTorch → .pt

 TensorFlow → .pb

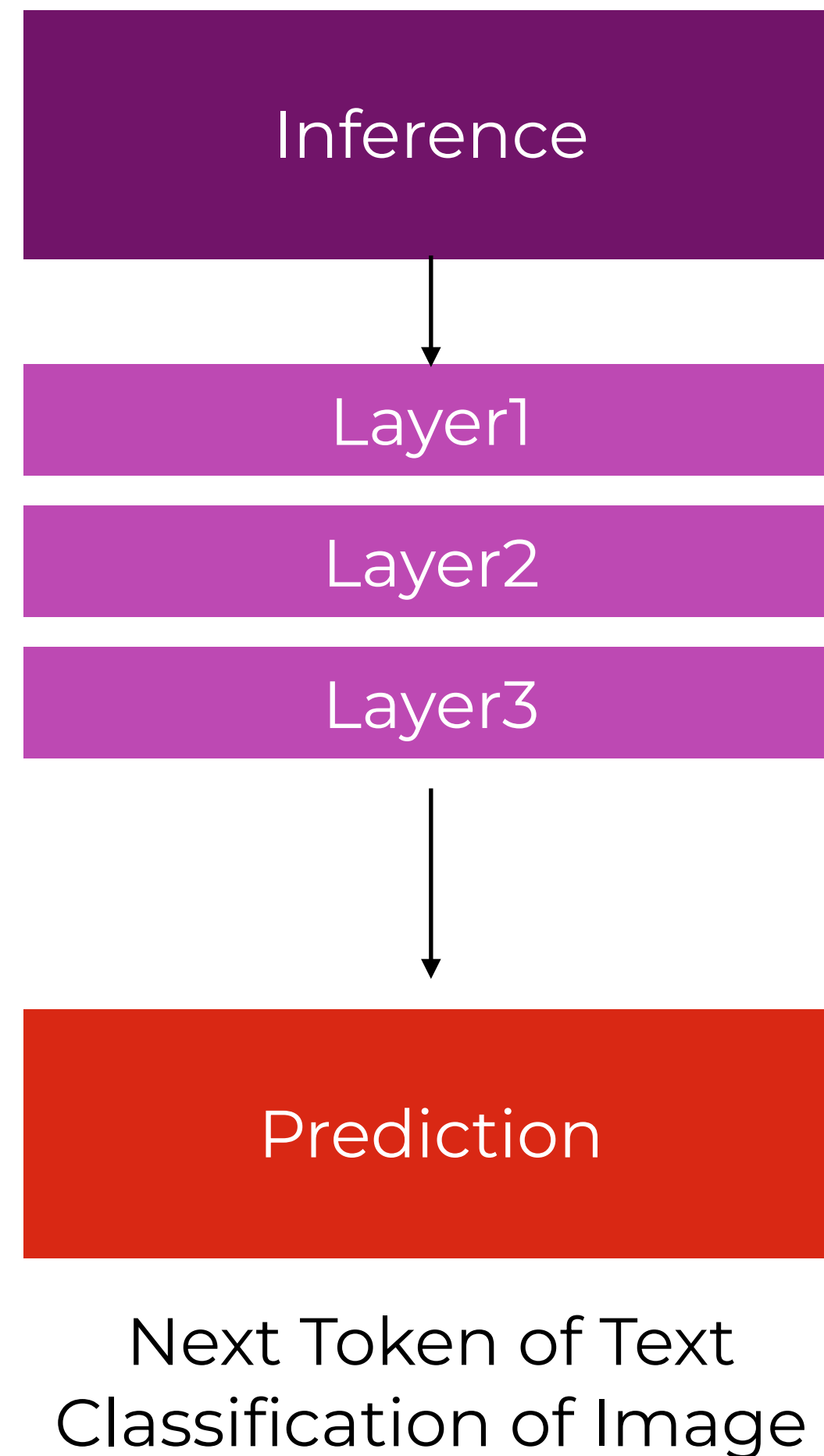


Different Model Formats



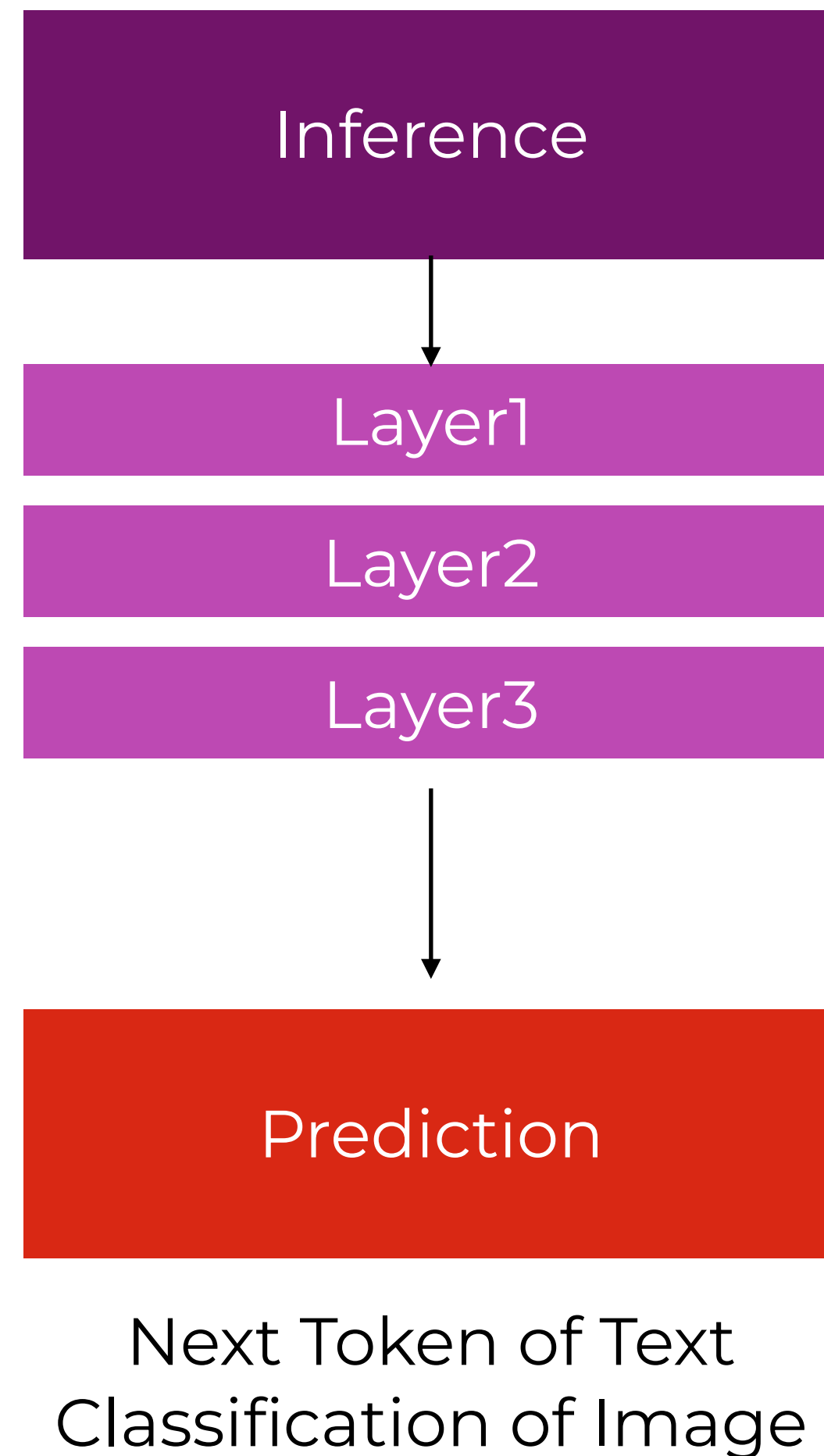
What about performance of the process?





Computational Complexity

Each layer of requires performing a vast number of mathematical operations, such as matrix multiplications. The more layers a model has, the greater the number of such operations.

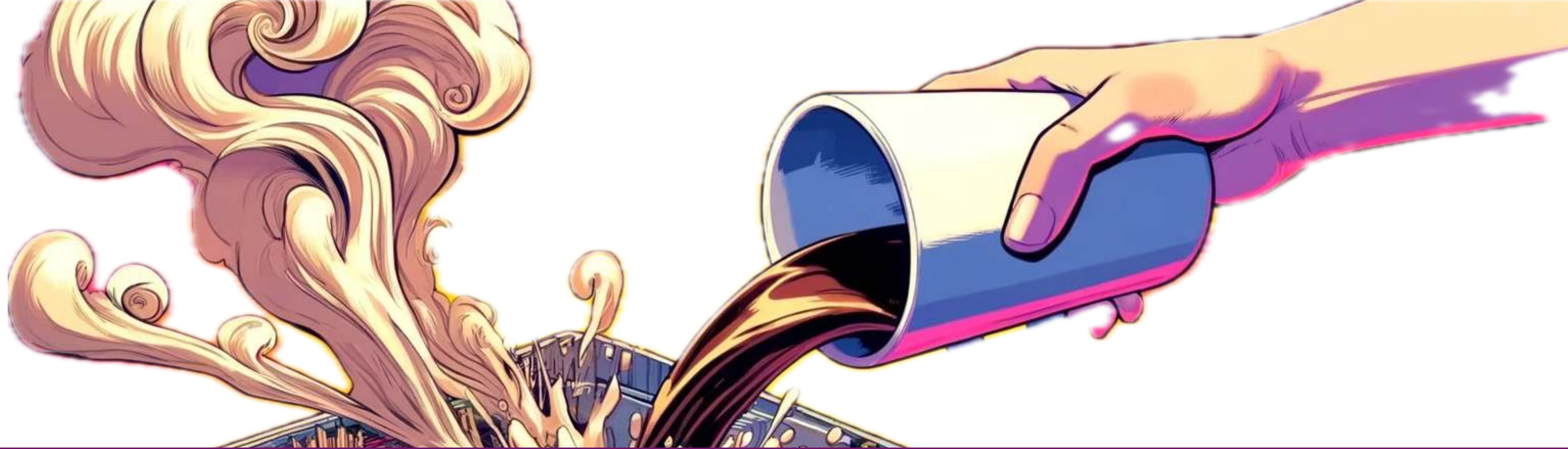


Computational Complexity

Each layer of requires performing a vast number of mathematical operations, such as matrix multiplications. The more layers a model has, the greater the number of such operations.

Model Size

Models with many layers typically have millions, or even billions, of parameters. Storing and processing such a large number of weights requires significant memory resources.

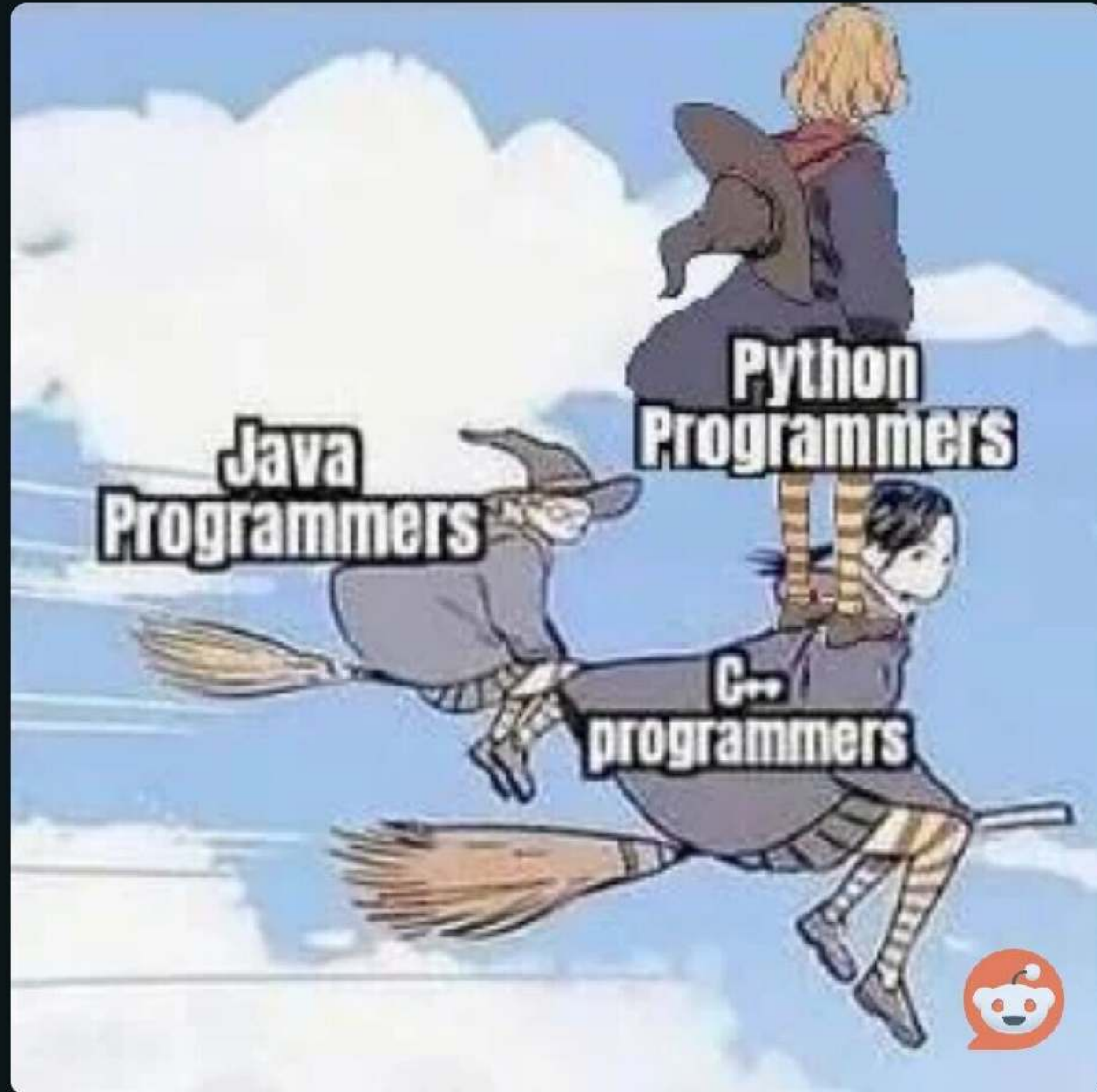


Native Calls





From ProgrammerHumor community on **Reddit**



JVM



imgflip.com

**JVM WHEN YOU REALIZE
IT'S WRITTEN IN C++**





Memory

Setting available Memory/RAM for a DL4J application

Memory Management for ND4J/DL4J: How does it work?

ND4J uses off-heap memory to store NDArrays, to provide better performance while working with NDArrays from native code such as BLAS and CUDA libraries.

"Off-heap" means that the memory is allocated outside of the JVM (Java Virtual Machine) and hence isn't managed by the JVM's garbage collection (GC). On the Java/JVM side, we only hold pointers to the off-heap memory, which can be passed to the underlying C++ code via JNI for use in ND4J operations.

Start Simple - JNI and Calling C Libraries



Memory

Setting available Memory/RAM for a DL4J application

Memory Management for ND4J/DL4J: How does it work?

ND4J uses off-heap memory to store NDArrays, to provide better performance while working with NDArrays from native code such as BLAS and CUDA libraries.

"Off-heap" means that the memory is allocated outside of the JVM (Java Virtual Machine) and hence isn't managed by the JVM's garbage collection (GC). On the Java/JVM side, we only hold pointers to the off-heap memory, which can be passed to the underlying C++ code via JNI for use in ND4J operations.

Start Simple - JNI and Calling C Libraries

JNI stands for Java Native Interface, a framework that allows Java code running in the Java Virtual Machine (JVM) to interact with native applications and libraries written in other programming languages like C or C++.

JNI is (was?) typically used when Java applications need to perform tasks that cannot be efficiently accomplished with pure Java, such as using system-specific libraries or interacting with low-level hardware.



Java 1.1, 1997

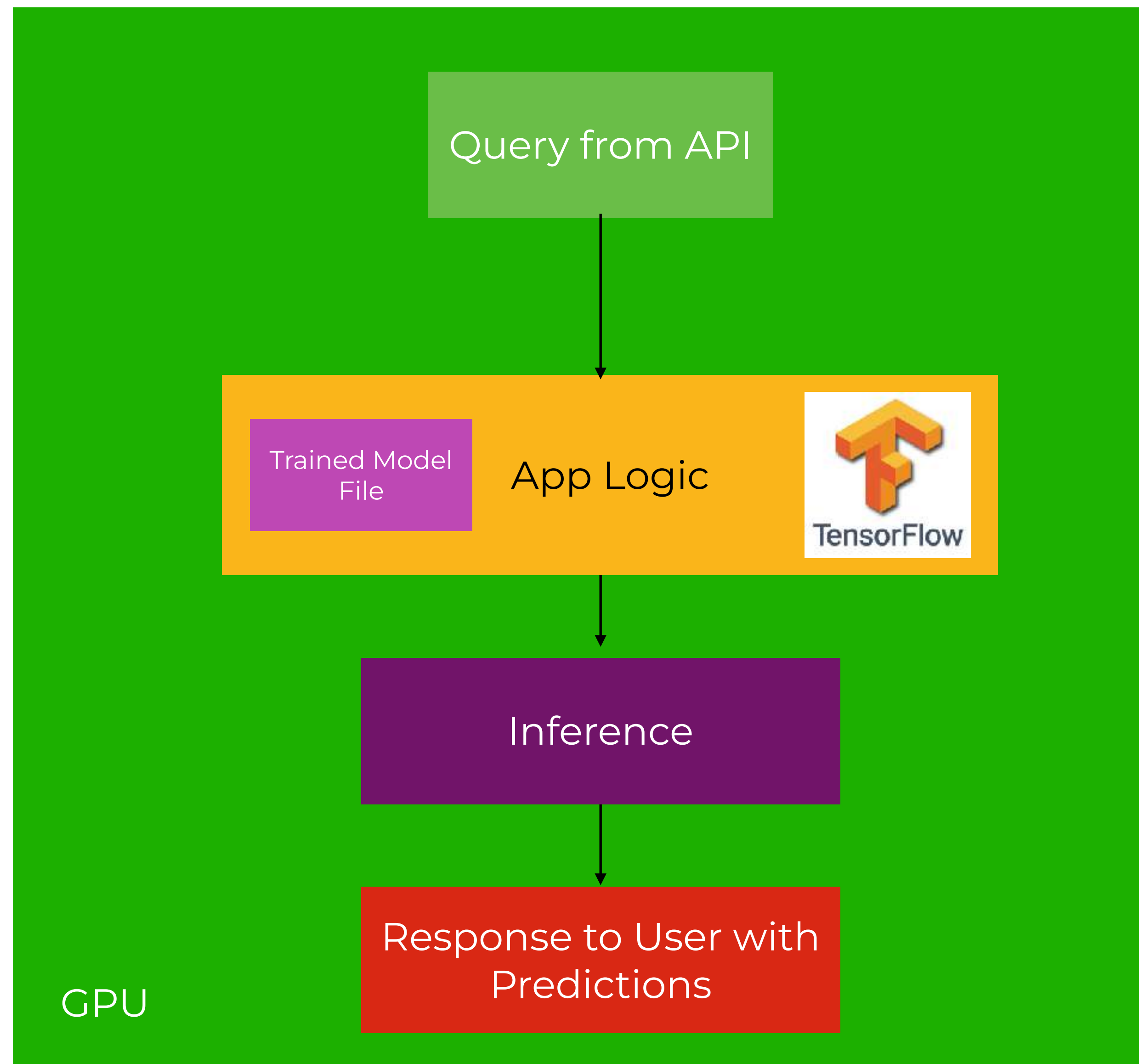


Project Panama



1. Foreign Function & Memory API (FFM API)
2. Linker API
3. Foreign Data Layout API
4. Interconnect with Foreign Languages
5. Improved Performance for Native Calls
6. Vector API

Project Panama



Model Inference run on GPU

NVIDIA Corporation (NVDA)

☆ Follow

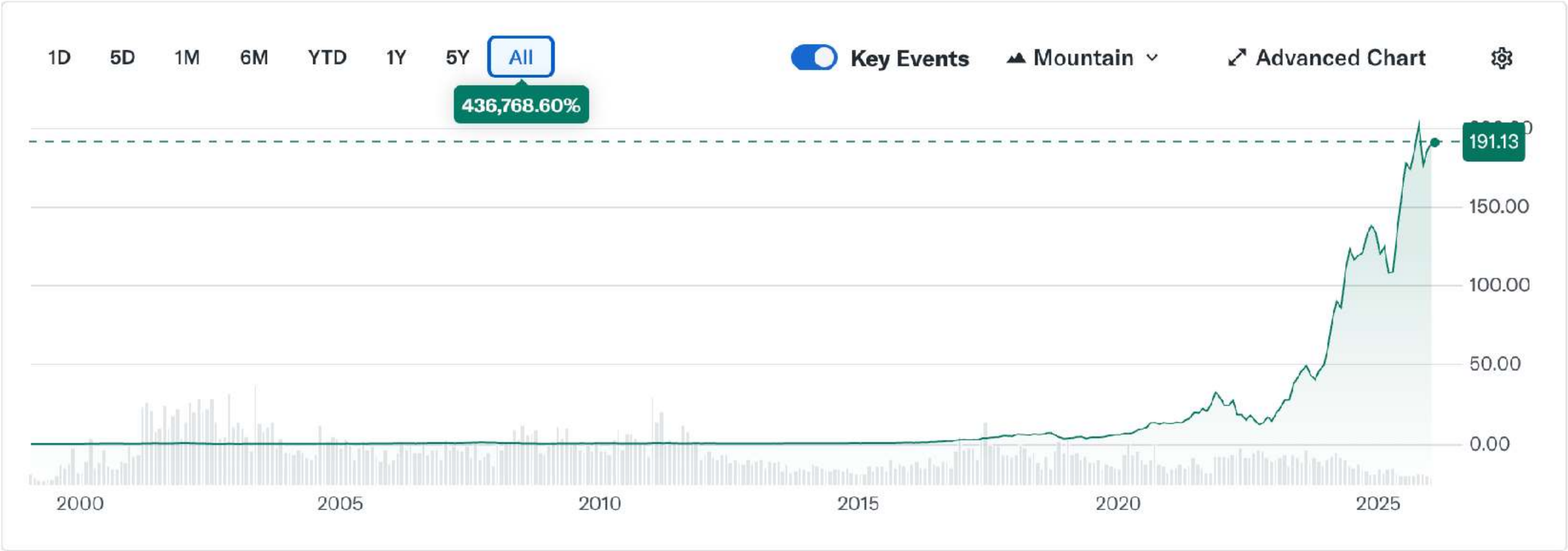
📈 Analyze

🕒 Time to buy NVDA?

191.13 **-1.38 (-0.72%)** **190.25** **-0.88 (-0.46%)**

At close: January 30 at 4:00:01 PM EST

🌙 After hours: January 30 at 7:59:56 PM EST

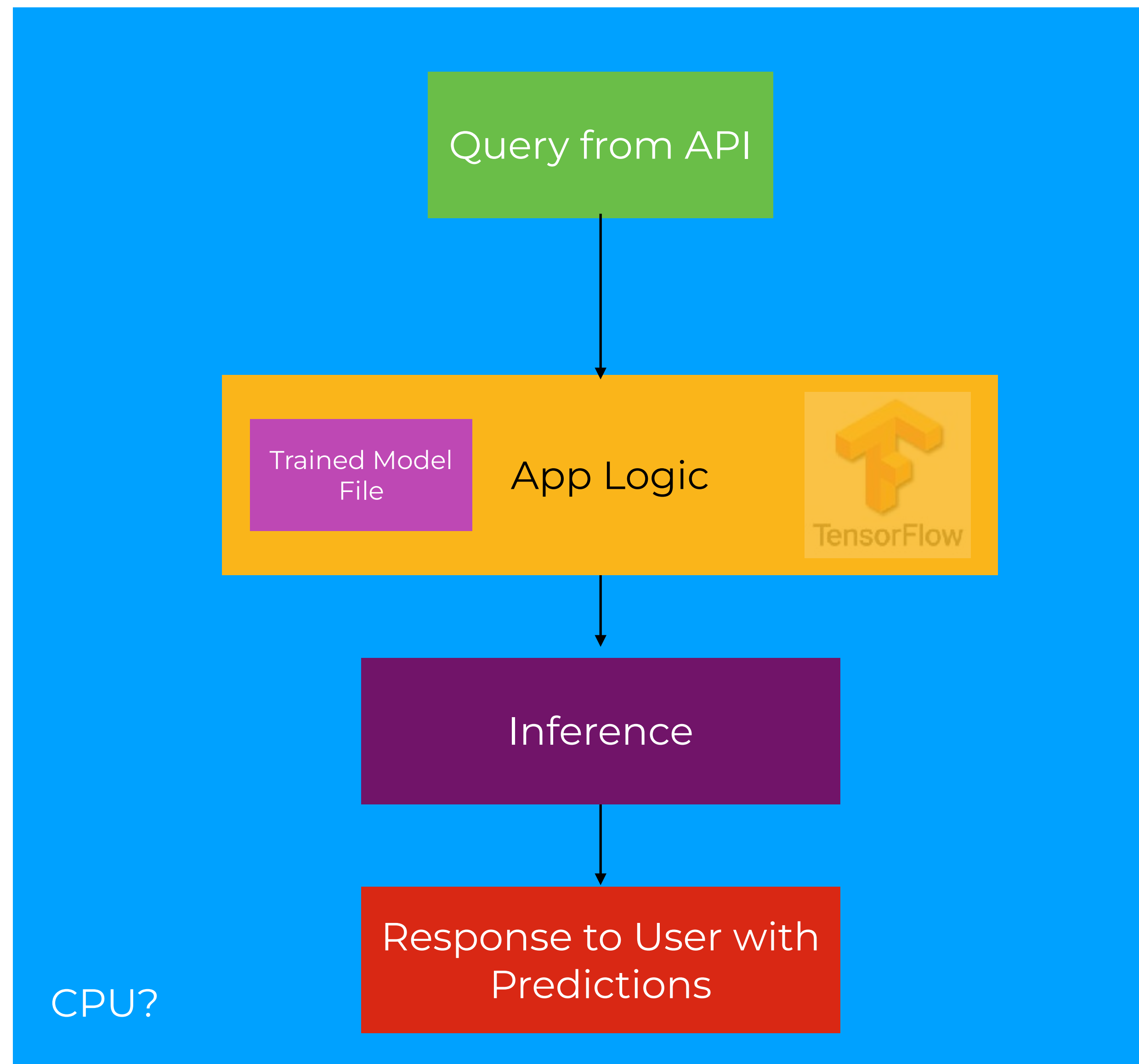


Nvidia Stock

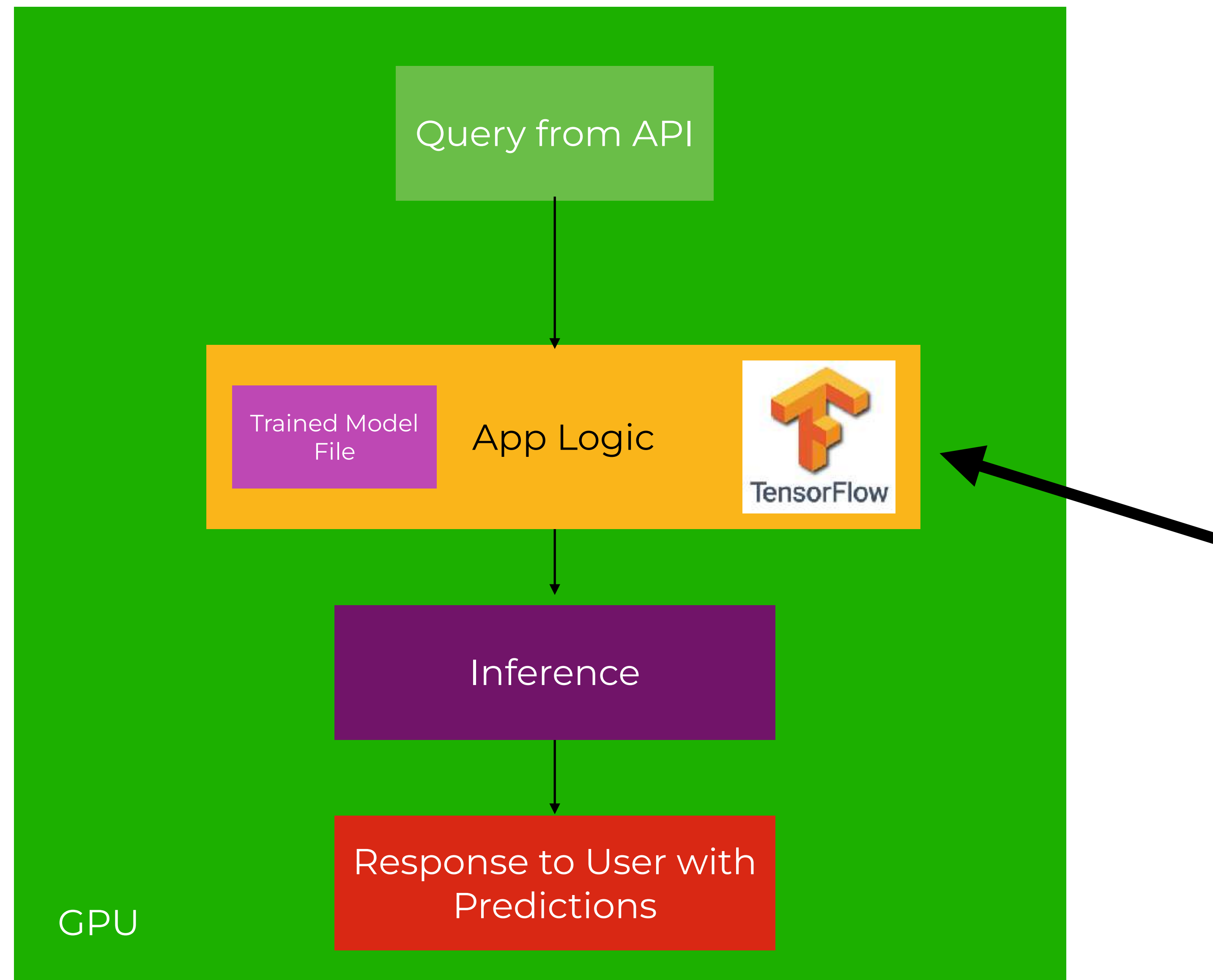
Stopy zwrotu: 1 rok: **+1 500.69%**, YTD: **+142.75%**



PS: Sandisk....



Can it be efficiently run on CPU without frameworks?



Model Inference run on GPU

llama2.c



Have you ever wanted to inference a baby [Llama 2](#) model in pure C? No? Well, now you can!

Train the Llama 2 LLM architecture in PyTorch then inference it with one simple 700-line C file ([run.c](#)). You might think that you need many billion parameter LLMs to do anything useful, but in fact very small LLMs can have surprisingly strong performance if you make the domain narrow enough (ref: [TinyStories](#) paper). This repo is a "fullstack" train + inference solution for Llama 2 LLM, with focus on minimalism and simplicity.

llama.cpp

The logo for LLaMA C++ features the text "LLaMA" in a large, white, sans-serif font, followed by "C++" in a smaller, orange, stylized font. The "C" is a thick, rounded letter, and the "++" are smaller and positioned to the right of the "C".

license [MIT](#) [Server](#) [passing](#) [conan](#) [b3542](#)

[Roadmap](#) / [Project status](#) / [Manifesto](#) / [ggml](#)

Inference of Meta's [LLaMA](#) model (and others) in pure C/C++

llama.cpp is a 700-line, C++-based inference implementation optimized for CPU inference.

It bypasses the traditional GPU-centric frameworks like PyTorch or TensorFlow.

Panama Vector API and Vector Databases

karpathy / llama2.c

Q Type to search

+

<> CodeIssues 117Pull requests 48ActionsProjectsSecurityInsights

⚠

Your recovery codes have not been saved in the past year. Make sure you still have them stored somewhere safe by viewing and downloading them again.

View recovery codes

×

📁 master

📄 llama2.c / run.c

Go to file

t

...

👤 digger-yu

fix some typo

2fbf705 · last year

🕒 History

CodeBlame973 lines (862 loc) · 37.6 KBCode 55% faster with GitHub Copilot

Raw📄📥📝🔍

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

...

/* Inference for Llama-2 Transformer model in pure C */

#include <stdio.h>

#include <stdlib.h>

#include <ctype.h>

#include <time.h>

#include <math.h>

#include <string.h>

#include <fcntl.h>

#if defined _WIN32

#include "win.h"

#else

#include <unistd.h>

#include <sys/mman.h>

#endif

// -----

// Transformer model

typedef struct {

int dim; // transformer dimension

int hidden_dim; // for ffn layers

int n_layers; // number of layers

int n_heads; // number of query heads

int n_kv_heads; // number of key/value heads (can be < query heads because of multiquery)

int vocab_size; // vocabulary size, usually 256 (byte-level)

int seq_len; // max sequence length

} Config;

typedef struct {

// token embedding table

float* token_embedding_table; // (vocab_size, dim)

// weights for rmsnorms

float* rms_att_weight; // (layer, dim) rmsnorm weights

float* rms_ffn_weight; // (layer, dim)

// weights for matmuls. note dim == n_heads * head_size

float* wq; // (layer, dim, n_heads * head_size)

float* wk; // (layer, dim, n_kv_heads * head_size)

float* wv; // (layer, dim, n_kv_heads * head_size)

...

Symbols

Find definitions and references for functions and other symbols in this file by clicking a symbol below or in the code.

Filter symbols

r

type Config

type TransformerWeights

type RunState

type Transformer

func malloc_run_state

func free_run_state

func memory_map_weights

func read_checkpoint

func build_transformer

func free_transformer

func rmsnorm

func softmax

func matmul

func forward

type TokenIndex

type Tokenizer

func compare_tokens

func build_tokenizer

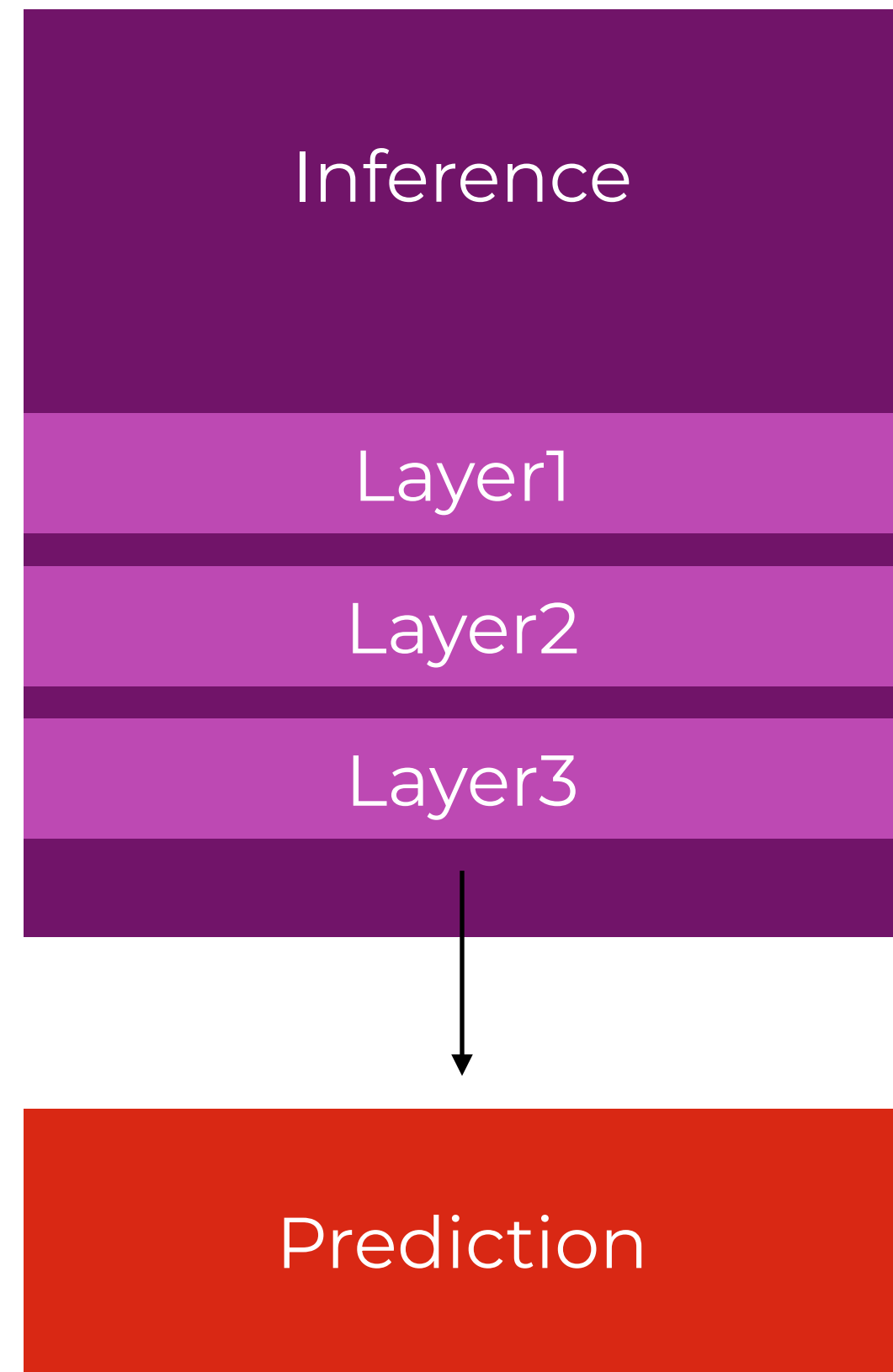
func free_tokenizer

func decode

func safe_printf

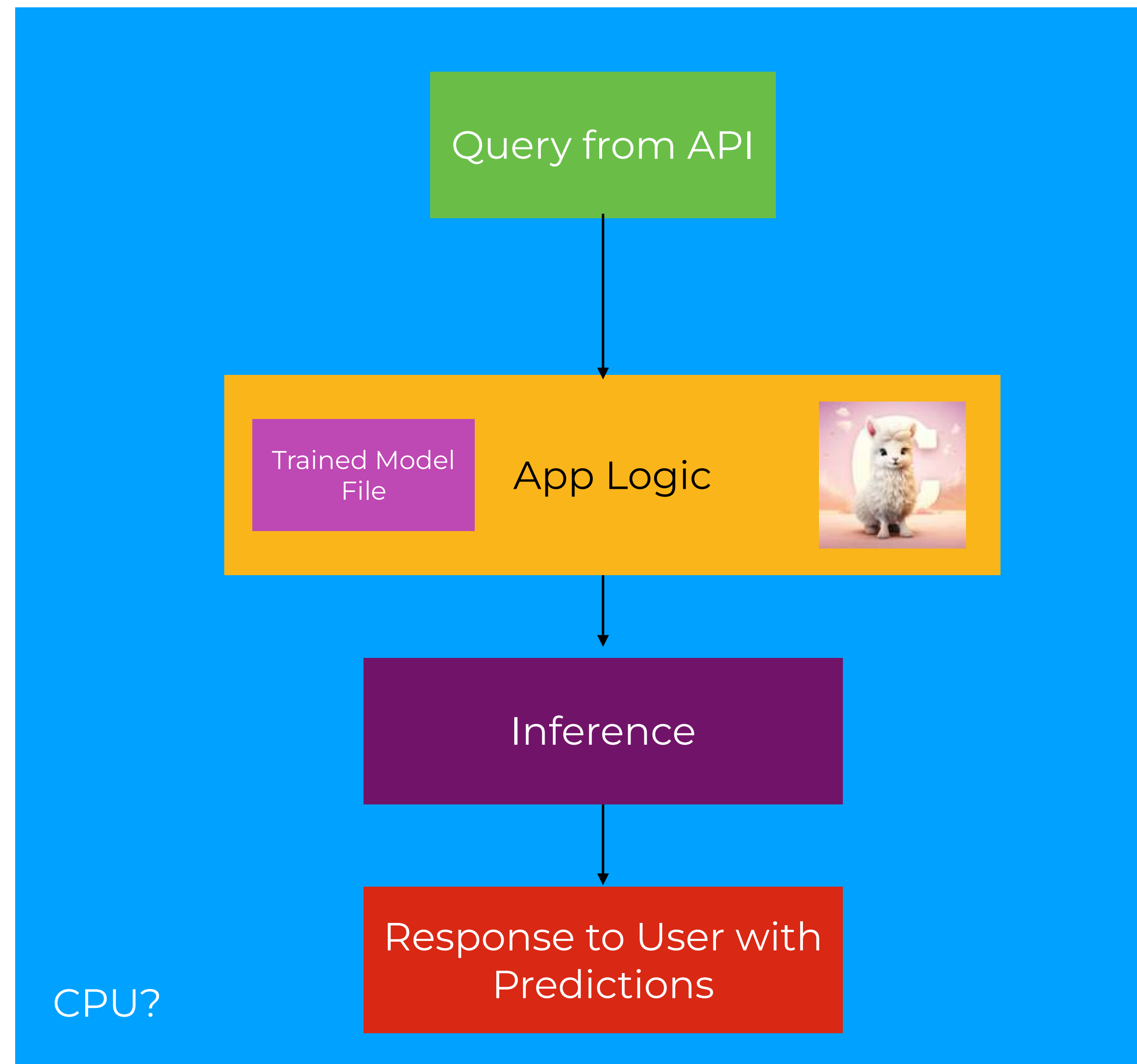
func str_lookup

Panama Vector API and Vector Databases



Next Token of Text
Classification of Image

Model Inference



Can it be efficiently run on CPU without frameworks?

📖 README 📄 MIT license ✎ ☰

Java 11+ llama.cpp #b3534

Java Bindings for llama.cpp

Inference of Meta's LLaMA model (and others) in pure C/C++.

You are welcome to contribute

- [Quick Start](#)
 - [No Setup required](#)
 - [Setup required](#)
- [Documentation](#)
 - [Example](#)
 - [Inference](#)
 - [Infilling](#)
- [Android](#)

📘 Note

Now with support for Llama 3, Phi-3, and flash attention

<https://github.com/kherud/java-llama.cpp> - Java Bindings

```

cat JavaLlamaCppDemo.java
String prompt = args.length > 1 ? args[1] : "tell me a short joke about programming.";

printHeader();
run(modelPath, prompt);
}

private static void printHeader() {
    System.out.println("[java-llama.cpp] =====");
    System.out.println("[java-llama.cpp] java-llama.cpp Inference Demo");
    System.out.println("[java-llama.cpp] =====");
    System.out.println("[java-llama.cpp] Java: " + System.getProperty("java.version"));
    System.out.println("[java-llama.cpp] VM: " + System.getProperty("java.vm.name"));
    System.out.println("[java-llama.cpp] OS: " + System.getProperty("os.name") + " " + System.getProperty("os.arch"));
    System.out.println("[java-llama.cpp] =====");
    System.out.println();
}

public static void run(String modelPath, String prompt) {
    System.out.println("[java-llama.cpp] Checking model file...");
    File modelFile = new File(modelPath);
    if (!modelFile.exists()) {
        System.err.println("[java-llama.cpp] ERROR: Model file not found: " + modelPath);
        System.err.println("[java-llama.cpp] Download the model with:");
        System.err.println("[java-llama.cpp] ./scripts/download-models.sh --fp16");
        System.exit(1);
    }
    System.out.println("[java-llama.cpp] Model file exists: " + modelPath);
    System.out.println("[java-llama.cpp] Model size: " + (modelFile.length() / 1024 / 1024) + " MB");

    System.out.println("[java-llama.cpp] Loading model...");

```

```

./gradlew :demos:java-llama-cpp:run

{%~ elif message.role == "tool" or message.role == "ipython" %}
{{~ "<|start_header_id|>ipython<|end_header_id|>\n\n" }}
{%~ if message.content is mapping or message.content is iterable %}
{{~ message.content | tojson }}
{%~ else %}
{{~ message.content }}
{%~ endif %}
{{~ "<|eot_id|>" }}
{%~ endif %}
{%~ endfor %}
{%~ if add_generation_prompt %}
{{~ '<|start_header_id|>assistant<|end_header_id|>\n\n' }}
{%~ endif %}
, example_format: '<|start_header_id|>system<|end_header_id|>

You are a helpful assistant<|eot_id|><|start_header_id|>user<|end_header_id|>

Hello<|eot_id|><|start_header_id|>assistant<|end_header_id|>

Hi there<|eot_id|><|start_header_id|>user<|end_header_id|>

How are you?<|eot_id|><|start_header_id|>assistant<|end_header_id|>

,

srv update_slots: all slots are idle
2.83s

[java-llama.cpp] Formatting prompt with Llama 3.2 chat template...
[java-llama.cpp] Prompt: Tell me a short joke about programming.

```

DEMO 4

```

} else {
    System.out.println("[java-llama.cpp] GPU offload disabled (set LLAMA_GPU_LAYERS=99 to enable)");
}

try (LlamaModel model = new LlamaModel(modelParams)) {
    long loadTime = System.currentTimeMillis() - loadStart;
    System.out.printf("[java-llama.cpp] Model loaded in %.2fs\n", loadTime / 1000.0);
    System.out.println();

    // Format prompt using Llama 3.2 Instruct chat template
    System.out.println("[java-llama.cpp] Formatting prompt with Llama 3.2 chat template...");
    String formattedPrompt = formatLlama3Prompt(prompt);

    System.out.println("[java-llama.cpp] Prompt: " + prompt);
    System.out.println("[java-llama.cpp] -----");
    System.out.println("[java-llama.cpp] Response:");

    // Configure inference parameters
    System.out.println("[java-llama.cpp] Configuring inference parameters (temp=0.7)...");
    InferenceParameters inferParams = new InferenceParameters(formattedPrompt)
        .setTemperature(0.7f);

    System.out.println("[java-llama.cpp] Starting inference...");
    long inferStart = System.currentTimeMillis();

```

```

arian replies, "
shelf."slot      release: id  0 | task 0 | stop processing: n_past = 78, truncated = 0
slot print_timing: id  0 | task 0 |
prompt eval time =      41.60 ms /   30 tokens (   1.39 ms per token,  721.17 tokens per second)
          eval time =     876.06 ms /   49 tokens (  17.88 ms per token,   55.93 tokens per second)
          total time =     917.66 ms /   79 tokens
srv update_slots: all slots are idle

[java-llama.cpp] -----

[java-llama.cpp] Stats:
[java-llama.cpp]   Model load time: 2.83s
[java-llama.cpp]   Inference time: 0.92s
[java-llama.cpp]   Tokens generated: 50
[java-llama.cpp]   Tokens/sec: 54.41

[java-llama.cpp] Demo completed successfully!

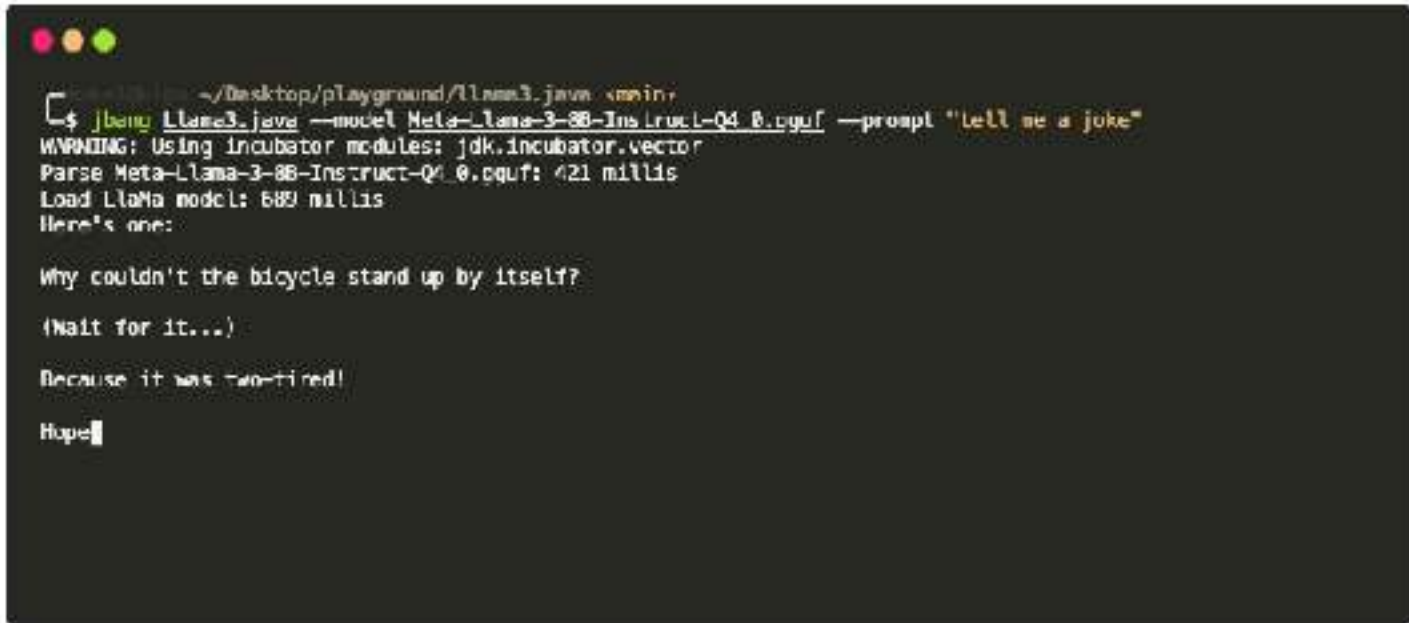
BUILD SUCCESSFUL in 4s
2 actionable tasks: 1 executed, 1 up-to-date
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.3.0/userguide/configuration_cache_enabling.html

```

./gradlew :demos:java-llama-cpp:run

Llama3.java

Practical [Llama 3](#) and [3.1](#) inference implemented in a single Java file.



This project is the successor of [llama2.java](#) based on [llama2.c](#) by [Andrej Karpathy](#) and his [excellent educational videos](#).

Besides the educational value, this project will be used to test and tune compiler optimizations and features on the JVM, particularly for the [Graal compiler](#).

Features

- Single file, no dependencies
- [GGUF format](#) parser
- Llama 3 tokenizer based on [minbpe](#)
- Llama 3 inference with Grouped-Query Attention
- Support Llama 3.1 (ad-hoc RoPE scaling)
- Support for Q8_0 and Q4_0 quantizations
- Fast matrix-vector multiplication routines for quantized tensors using Java's [Vector API](#)
- Simple CLI with `--chat` and `--instruct` modes.

Here's the interactive `--chat` mode in action:



main

llama3.java / Llama3.java



mukel Fix missing <|eom_id|> token in Llama 3 vocabulary.

Code

Blame

Executable File · 2103 lines (1851 loc) · 91 KB

Code 55% faster with GitHub Copilot

```
1  ///usr/bin/env jbang "$0" "$@" ; exit $?
2  //JAVA 21+
3  //PREVIEW
4  //COMPILE_OPTIONS --add-modules=jdk.incubator.vector
5  //RUNTIME_OPTIONS --add-modules=jdk.incubator.vector
6
7  // Practical Llama 3 (and 3.1) inference in a single Java file
8  // Author: Alfonso2 Peterssen
9  // Based on Andrej Karpathy's llama2.c and minbpe projects
10 //
11 // Supports llama.cpp's GGUF format, restricted to Q4_0 and Q8_0 quantized models
12 // Multi-threaded matrix vector multiplication routines implemented using Java's Vector API
13 // Simple CLI with --chat and --instruct mode
14 //
15 // To run just:
16 // jbang Llama3.java --help
17 //
18 // Enjoy!
19 package com.llama4j;
20
```

Llama3.java

Llama3.java

Practical [Llama 3](#) and [3.1](#) inference implemented in a single Java file.

```
~/Desktop/playground/llama3.java <main>
$ jbang llama3.java --model Meta-Llama-3-8B-Instruct-Q4_0.gguf --prompt "Tell me a joke"
WARNING: Using incubator modules: jdk.incubator.vector
Parse Meta-Llama-3-8B-Instruct-Q4_0.gguf: 421 millis
Load LLaMA model: 689 millis
Here's one:

Why couldn't the bicycle stand up by itself?
(Wait for it...)
Because it was two-tired!

Hup!
```

main llama3.java / Llama3.java

mukel Fix missing <|eom_id|> token in Llama 3 vocabulary.

Code Blame Executable File · 2103 lines (1851 loc) · 91 KB Code 55% faster with GitHub Copilot

```
1 //usr/bin/env jbang "$0" "$@" ; exit $?
2 //JAVA 21+
3 //PREVIEW
4 //COMPILE_OPTIONS --add-modules=jdk.incubator.vector
```

DEMO 5

- Single file, no dependencies
- [GGUF format](#) parser
- Llama 3 tokenizer based on [minibpe](#)
- Llama 3 inference with Grouped-Query Attention
- Support Llama 3.1 (ad-hoc RoPE scaling)
- Support for Q8_0 and Q4_0 quantizations
- Fast matrix-vector multiplication routines for quantized tensors using Java's [Vector API](#)
- Simple CLI with `--chat` and `--instruct` modes.

Here's the interactive `--chat` mode in action:

```
13 // Simple CLI with --chat and --instruct mode
14 //
15 // To run just:
16 // jbang llama3.java --help
17 //
18 // Enjoy!
19 package com.llama4j;
20
```

./demos/llama3-java/scripts/run-llama3.sh

1. **Enhanced C2 compiler** - better optimization of vector operations
2. **ARM64 improvements** - better NEON instruction selection on Apple Silicon
3. **Better Vector API intrinsics** - improved SIMD code generation

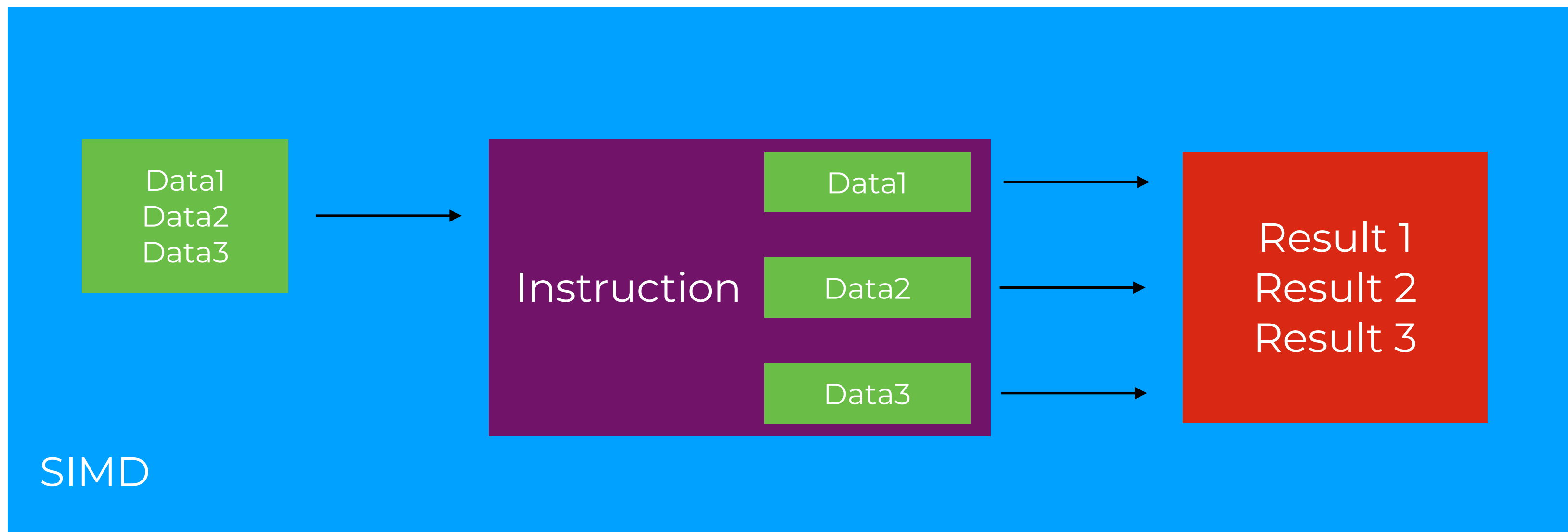
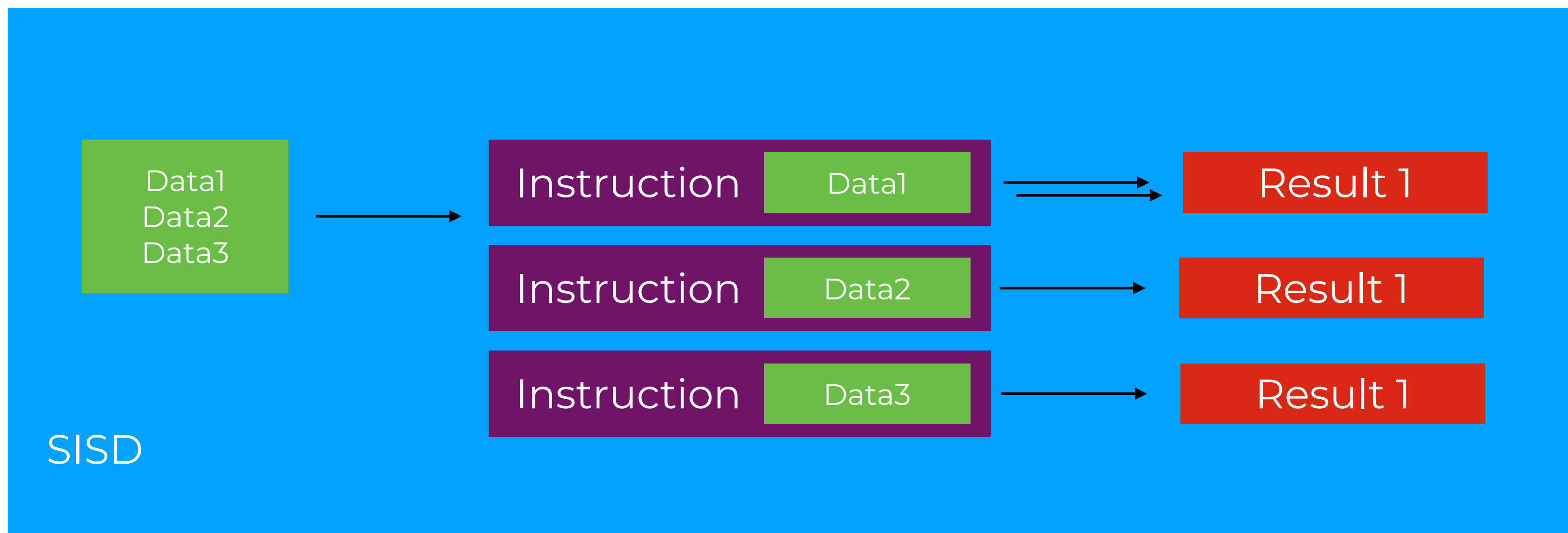


JDK 25 is significantly faster than JDK 21



Project Valhalla & Panama Together: SIMD





Panama Vector API and Vector Databases

```

public class VectorizationExample {
    public static void main(String[] args) {
        int[] arrayA = new int[1000];
        int[] arrayB = new int[1000];
        int[] result = new int[1000];

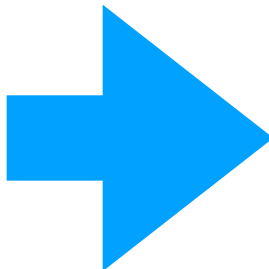
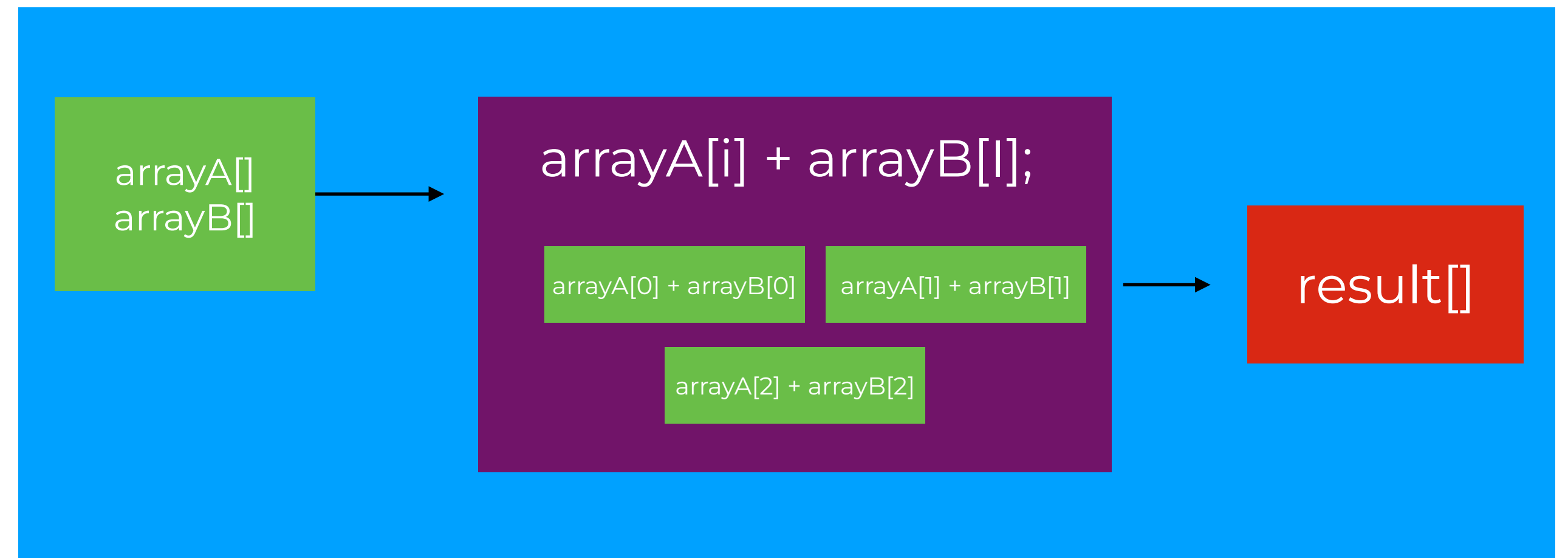
        // Initialize arrays
        for (int i = 0; i < arrayA.length; i++) {
            arrayA[i] = i;
            arrayB[i] = i * 2;
        }

        // Perform element-wise addition (this can be autovectorized)
        for (int i = 0; i < arrayA.length; i++) {
            result[i] = arrayA[i] + arrayB[i];
        }

        // Output some of the results
        for (int i = 0; i < 10; i++) {
            System.out.println("Result[" + i + "] = " + result[i]);
        }
    }
}

```

JIT

Autovectorization

JEP 508: Vector API (Tenth Incubator)

<i>Owner</i>	Ian Graves
<i>Type</i>	Feature
<i>Scope</i>	JDK
<i>Status</i>	Closed / Delivered
<i>Release</i>	25
<i>Component</i>	core-libs
<i>Discussion</i>	panama dash dev at openjdk dot org
<i>Effort</i>	XS
<i>Duration</i>	XS
<i>Relates to</i>	JEP 489: Vector API (Ninth Incubator) JEP 529: Vector API (Eleventh Incubator)
<i>Reviewed by</i>	Jatin Bhateja, Sandhya Viswanathan, Vladimir Ivanov
<i>Endorsed by</i>	Paul Sandoz
<i>Created</i>	2025/03/31 18:19
<i>Updated</i>	2025/10/01 17:55
<i>Issue</i>	8353296

Summary

Introduce an API to express vector computations that reliably compile at runtime to optimal vector instructions on supported CPUs, thus achieving performance superior to equivalent scalar computations.

History

We first proposed the Vector API in [JEP 338](#) and integrated it into JDK 16 as an [incubating API](#). We proposed further rounds of incubation in [JEP 414](#) (integrated into JDK 17), [JEP 417](#) (JDK 18), [JEP 426](#) (JDK 19), [JEP 438](#) (JDK 20), [JEP 448](#) (JDK 21), [JEP 460](#) (JDK 22), [JEP 469](#) (JDK 23), and [JEP 489](#) (JDK 24).

We here propose to re-incubate the Vector API in JDK 25 with one API change and two notable implementation changes:

- VectorShuffle now supports access to and from MemorySegment.
- The implementation now links to native mathematical-function libraries via the Foreign Function & Memory API ([JEP 454](#)) rather than custom C++ code inside the HotSpot JVM, thereby improving maintainability.
- Addition, subtraction, division, multiplication, square root, and fused multiply/add operations on Float16 values are now auto-vectorized on supporting x64 CPUs.

The Vector API will incubate until necessary features of [Project Valhalla](#) become available as preview features. At that time, we will adapt the Vector API and its implementation to use them and then promote the Vector API from incubation to preview.



Panama Vector API and Vector Databases

JVector - The most advanced embedded vector search engine



JVector is a pure Java, zero dependency, embedded vector search engine, used by DataStax Astra DB.

JVector uses state of the art graph algorithms inspired by DiskANN and related research that offer high recall and low latency. It uses the Panama SIMD API to accelerate index build and queries. It is memory efficient, compresses vectors using product quantization so they can stay in memory during searches.

JVector is concurrent, its index can scale linearly to 32 threads.

jvector Public		Watch 29	
main 29 Branches 36 Tags		Go to file t Add file Code	
jbellis 09/24 version of colbert-10M ✓		a95e640 · 2 days ago 669 Commits	
.github/workflows	Enable JDK 22 CI	6 months ago	
.mvn	Add mvn wrapper	last year	
jvector-base	fix getIdUpperBound javadoc	5 days ago	
jvector-examples	09/24 version of colbert-10M	2 days ago	
jvector-multirelease	Use on-heap MemorySegments for native vectors/sequen...	6 months ago	
jvector-native	Remove check for VBMI on CPU. With Fused ADC using s...	last month	
jvector-tests	add support for non-sequential remapped ordinals (#349)	2 months ago	
jvector-twenty	Release 3.0.0-beta.13	3 months ago	
siftsmall	SiftSmall example	last year	
.gitignore	Use precomputed PQ in Bench. Add gitignore for cached ...	6 months ago	
LICENSE.txt	import hnsw + pq	last year	
NOTICE.txt	import hnsw + pq	last year	
README.md	Release 3.0.0-beta.13	3 months ago	
UPGRADING.md	Release 3.0.0-beta.13	3 months ago	

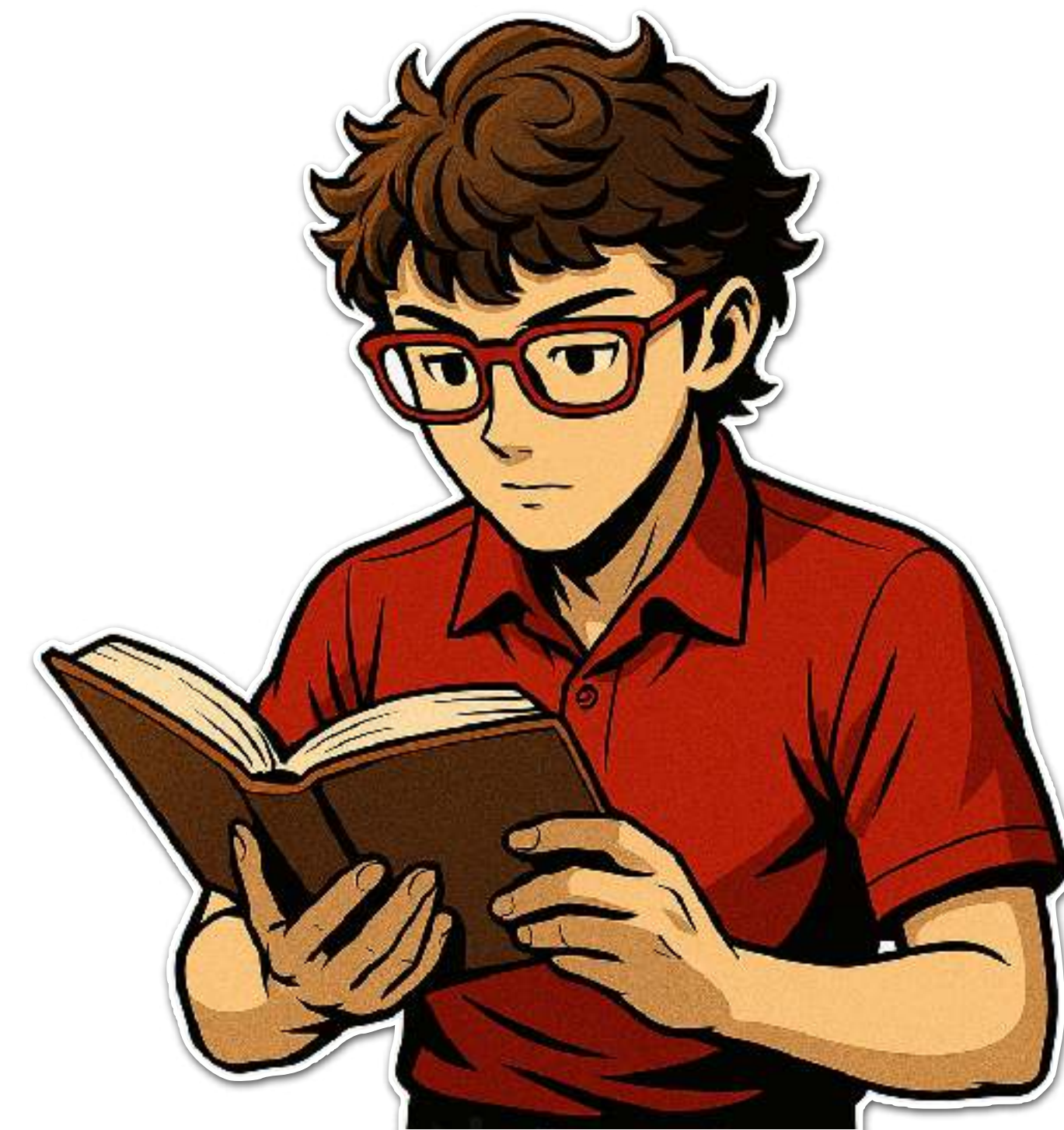


Quantization



Model is ideal for local benchmarks

- Ultra-small size (~2GB): Runs on any hardware
- **FP16 precision: High accuracy without quantization artifacts**
- Fast inference: 1B params = high tokens/second even on consumer CPUs/GPUs
- Instruction-tuned: Realistic benchmark of chat/agent performance (not raw pretraining)
- Standardized GGUF: Works with all major tools (llama.cpp, Ollama, LM Studio, etc.) for apples-to-apples comparison

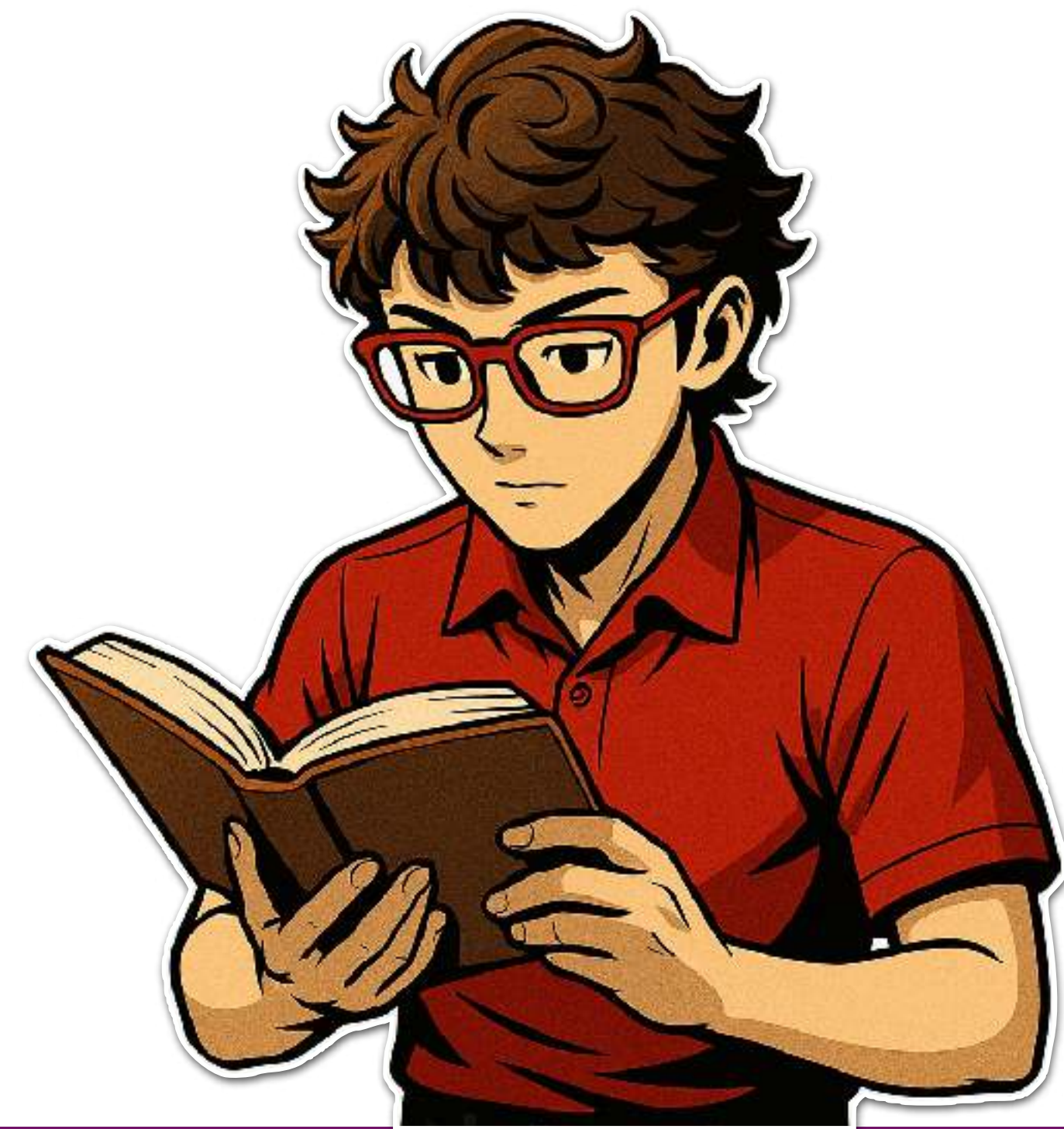


Llama-3.2-1B-Instruct-f16.gguf

Quantization is a compression technique that involves mapping high precision values to a lower precision one.

For an LLM, that means modifying the precision of their weights and activations making it less memory intensive. This surely does have impact on the capabilities of the model including the accuracy.

It is often a trade-off based on the use case to go with model that is quantized. It is found that in some cases its possible to achieve comparable results with significantly lower precision.



Llama-3.2-1B-Instruct-f16.gguf



Llama-3.2-1B-Instruct-f16.gguf

bitnet.cpp

license MIT version 1.0



BitNet b1.58 2B4T on Hugging Face

Try it out via this [demo](#), or build and run it on your own [CPU](#) or [GPU](#).

bitnet.cpp is the official inference framework for 1-bit LLMs (e.g., BitNet b1.58). It offers a suite of optimized kernels, that support **fast** and **lossless** inference of 1.58-bit models on CPU and GPU (NPU support will coming next).

The first release of bitnet.cpp is to support inference on CPUs. bitnet.cpp achieves speedups of 1.37x to 5.07x on ARM CPUs, with larger models experiencing greater performance gains. Additionally, it reduces energy consumption by 55.4% to 70.0%, further boosting overall efficiency. On x86 CPUs, speedups range from 2.37x to 6.17x with energy reductions between 71.9% to 82.2%. Furthermore, bitnet.cpp can run a 100B BitNet b1.58 model on a single CPU, achieving speeds comparable to human reading (5-7 tokens per second), significantly enhancing the potential for running LLMs on local devices. Please refer to the [technical report](#) for more details.

Latest optimization introduces parallel kernel implementations with configurable tiling and embedding quantization support, achieving 1.15x to 2.1x additional speedup over the original implementation across different hardware platforms and workloads. For detailed technical information, see the [optimization guide](#).

The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits

Shuming Ma* Hongyu Wang* Lingxiao Ma Lei Wang Wenhui Wang
Shaohan Huang Li Dong Ruiping Wang Jilong Xue Furu Wei[◊]
<https://aka.ms/GeneralAI>

Abstract

Recent research, such as BitNet [WMD⁺23], is paving the way for a new era of 1-bit Large Language Models (LLMs). In this work, we introduce a 1-bit LLM variant, namely **BitNet b1.58**, in which every single parameter (or weight) of the LLM is ternary $\{-1, 0, 1\}$. It matches the full-precision (i.e., FP16 or BF16) Transformer LLM with the same model size and training tokens in terms of both perplexity and end-task performance, while being significantly more cost-effective in terms

BitNet



The image shows a YouTube video player interface. The video frame displays a woman with long, wavy brown hair, wearing a red sweater, smiling at the camera. The background is a dimly lit room with a blue sofa, a white pillow, a framed abstract painting on the wall, and a dark wooden shelving unit with various items and a warm light source. Below the video frame, the title "The myth of 1-bit LLMs | Quantization-Aware Training" is visible. The channel name "Julia Turc" and subscriber count "56,9 tys. subskrybentów" are shown. A row of interactive buttons includes "Wesprzyj", a notification bell, a thumbs-up icon with "3,7 tys.", a thumbs-down icon, "Udostępnij", "Zapisz", "Klip", "Pobierz", and a three-dot menu.

The myth of 1-bit LLMs | Quantization-Aware Training

Julia Turc
56,9 tys. subskrybentów

Wesprzyj

3,7 tys.

Udostępnij

Zapisz

Klip

Pobierz

BitNet



Quantization improves performance by reducing memory bandwidth requirement and increasing cache utilization, but can have negative impact on model quality.

Llama-3.2-1B-Instruct-f16.gguf



Optimization: Half Floats and Project Valhalla

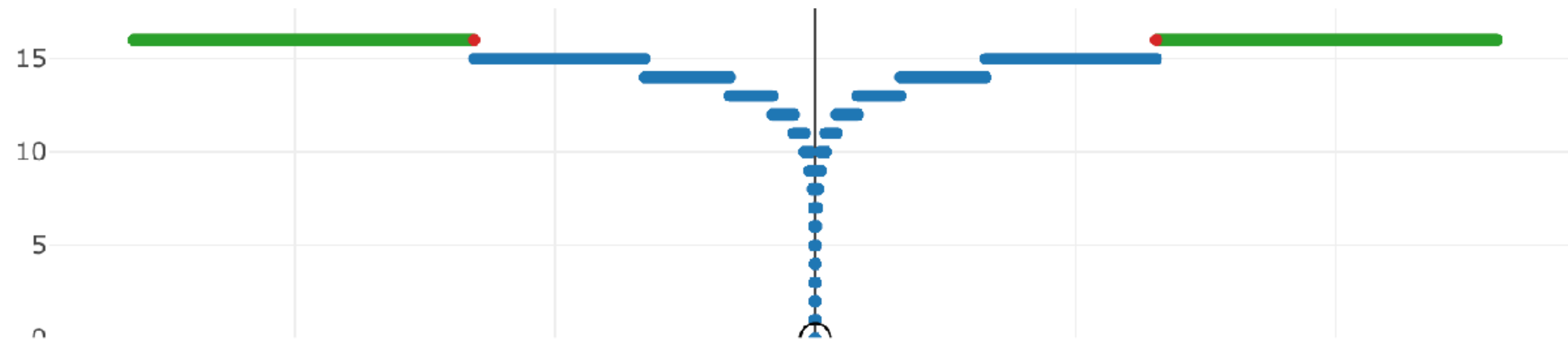


In standard floating-point numbers (commonly known as floats), we have 32 bits, with some allocated for the sign of the number, some for the exponent, and some for the mantissa, which determines precision.

1 bit for the sign (positive or negative)

8 bits for the exponent (range)

23 bits for the mantissa (precision)



Half Floats

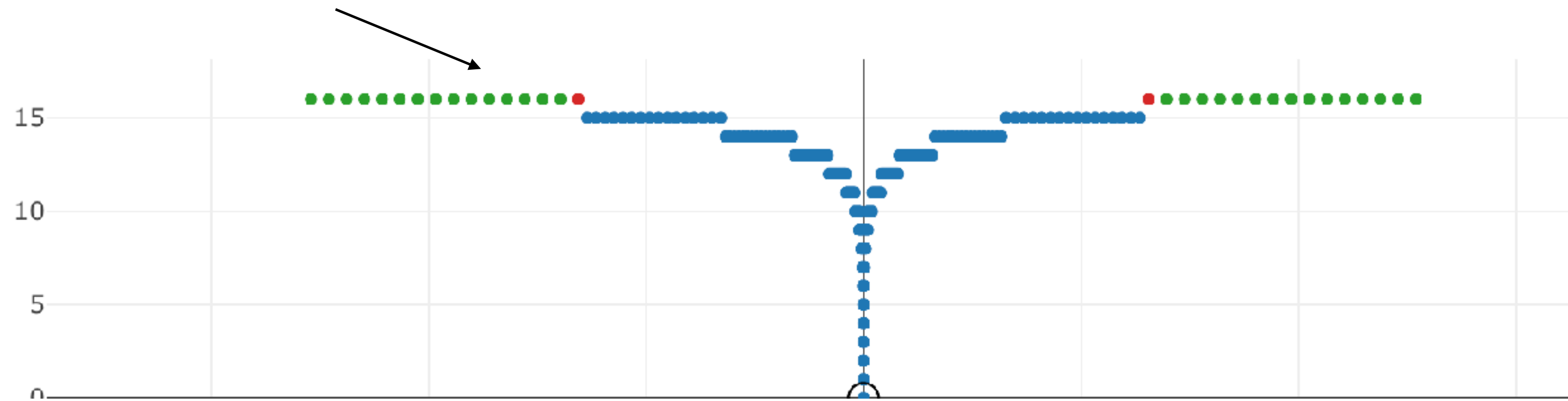
In half-floats, we only have 16 bits, meaning they can store smaller and **less precise numbers**.

1 bit for the sign (positive or negative)

5 bits for the exponent (range)

10 bits for the mantissa (precision)

Narrower range with less numbers in given range



Half Floats

jdk.incubator.vector > jdk.incubator.vector > **Float16**

Search

Contents

Filter

Description

Floating-point Equality, Equivalence, and Comparison

Decimal ↔ Binary Conversion Issues

Field Summary

Method Summary

Field Details

POSITIVE_INFINITY

NEGATIVE_INFINITY

NaN

MAX_VALUE

MIN_NORMAL

MIN_VALUE

SIZE

PRECISION

MAX_EXPONENT

MIN_EXPONENT

Class Float16

java.lang.Object

java.lang.Number

jdk.incubator.vector.Float16

All Implemented Interfaces:

Serializable, Comparable<Float16>

public final class **Float16**

extends [Number](#)

implements [Comparable<Float16>](#)

The Float16 is a class holding 16-bit data in IEEE 754 binary16 format.

Binary16 Format:
S EEEEE MMMMMMMMMM
Sign - 1 bit
Exponent - 5 bits
Significand - 10 bits (does not include the *implicit bit* inferred from the exponent, see [PRECISION](#))

Unless otherwise specified, the methods in this class use a *rounding policy* (JLS 15.4^u) of [round to nearest](#).

This is a [value-based](#) class; programmers should treat instances that are [equal](#) as interchangeable and should not use instances for synchronization, or unpredictable behavior may occur. For example, in a future release, synchronization may fail.

Floating-point Equality, Equivalence, and Comparison

The class `java.lang.Double` has a [discussion of equality, equivalence, and comparison of floating-point values](#) that is equally applicable to `Float16` values.

jdk.incubator.vector > jdk.incubator.v

Contents

Description

- Floating-point Equality, Equivalence Comparison
- Decimal ↔ Binary Conversion Issues
- Field Summary
- Method Summary
- Field Details
 - POSITIVE_INFINITY
 - NEGATIVE_INFINITY
 - NaN
 - MAX_VALUE
 - MIN_NORMAL
 - MIN_VALUE
 - SIZE
 - PRECISION
 - MAX_EXPONENT
 - MIN_EXPONENT

```
import jdk.incubator.vector.Float16;

Float16 a = Float16.valueOf(3.5f);
Float16 b = Float16.valueOf(2.0f);

// Scalar arithmetic
Float16 sum = Float16.add(a, b);           // 5.5
Float16 product = Float16.multiply(a, b);  // 7.0
Float16 fma = Float16.fma(a, b, c);        // a * b + c
Float16 sqrt = Float16.sqrt(a);            // 1.8711

// Conversions
float f = a.floatValue();
double d = a.doubleValue();
```

stances for synchronization, or

ually applicable to Float16

In JDK 24+

jdk.incubator.vector > jdk.incubator.v

Contents

Description

- Floating-point Equality, Equivalence Comparison
- Decimal ↔ Binary Conversion Issues
- Field Summary
- Method Summary
- Field Details
- POSITIVE_INFINITY

```
import jdk.incubator.vector.Float16;

Float16 a = Float16.value0f(3.5f);
Float16 b = Float16.value0f(2.0f);

// Scalar arithmetic
Float16 sum = Float16.add(a, b);           // 5.5
```

DEMO 6

- SIZE
- PRECISION
- MAX_EXPONENT
- MIN_EXPONENT

```
float f = a.floatValue();
double d = a.doubleValue();
```

stances for synchronization, or

ually applicable to Float16

./run-fp16-vector.sh

JEP 508: Vector API (Tenth Incubator)

Owner	Ian Graves
Type	Feature
Scope	JDK
Status	Closed / Delivered
Release	25
Component	core-libs
Discussion	panama dash dev at openjdk dot org
Effort	XS
Duration	XS
Relates to	JEP 489: Vector API (Ninth Incubator) JEP 529: Vector API (Eleventh Incubator)
Reviewed by	Jatin Bhateja, Sandhya Viswanathan, Vladimir Ivanov
Endorsed by	Paul Sandoz
Created	2025/03/31 18:19
Updated	2025/10/01 17:55
Issue	8353296

Summary

Introduce an API to express vector computations that reliably compile at runtime to optimal vector instructions on supported CPUs, thus achieving performance superior to equivalent scalar computations.

History

We first proposed the Vector API in [JEP 338](#) and integrated it into JDK 16 as an [incubating API](#). We proposed further rounds of incubation in [JEP 414](#) (integrated into JDK 17), [JEP 417](#) (JDK 18), [JEP 426](#) (JDK 19), [JEP 438](#) (JDK 20), [JEP 448](#) (JDK 21), [JEP 460](#) (JDK 22), [JEP 469](#) (JDK 23), and [JEP 489](#) (JDK 24).

We here propose to re-incubate the Vector API in JDK 25 with one API change and two notable implementation changes:

- VectorShuffle now supports access to and from MemorySegment.
- The implementation now links to native mathematical-function libraries via the Foreign Function & Memory API ([JEP 454](#)) rather than custom C++ code inside the HotSpot JVM, thereby improving maintainability.
- Addition, subtraction, division, multiplication, square root, and fused multiply/add operations on Float16 values are now auto-vectorized on supporting x64 CPUs.

The Vector API will incubate until necessary features of [Project Valhalla](#) become available as preview features. At that time, we will adapt the Vector API and its implementation to use them and then promote the Vector API from incubation to preview.



- Addition, subtraction, division, multiplication, square root, and fused multiply/add operations on Float16 values are now auto-vectorized on supporting x64 CPUs.

In JDK 25+

JEP 508: Vector API (Tenth Incubator)

Owner Ian Graves
Type Feature
Scope JDK
Status Closed / Delivered
Release 25
Component core-libs
Discussion [panama dash dev at openjdk dot org](#)
Effort XS
Duration XS
Relates to [JEP 489: Vector API \(Ninth Incubator\)](#)
[JEP 529: Vector API \(Eleventh Incubator\)](#)
Reviewed by Jatin Bhateja, Sandhya Viswanathan, Vladimir Ivanov
Endorsed by Paul Sandoz
Created 2025/03/31 18:19
Updated 2025/10/01 17:55
Issue [8353296](#)



Summary

Introduce an API to express vector optimal vector instructions on supported hardware, which are superior to equivalent scalar computations.

History

We first proposed the Vector API in [JEP 417](#) (JDK 17), [JEP 417](#) (JDK 18), [JEP 417](#) (JDK 19), [JEP 417](#) (JDK 20), [JEP 417](#) (JDK 21), [JEP 417](#) (JDK 22), [JEP 417](#) (JDK 23), [JEP 417](#) (JDK 24), and [JEP 417](#) (JDK 25).

We here propose to re-incubate the Vector API with two notable implementation changes:

- VectorShuffle now supports access to and from MemorySegment.
- The implementation now links to native mathematical-function libraries via the Foreign Function & Memory API ([JEP 454](#)) rather than custom C++ code inside the HotSpot JVM, thereby improving maintainability.
- Addition, subtraction, division, multiplication, square root, and fused multiply/add operations on Float16 values are now auto-vectorized on supporting x64 CPUs.

The Vector API will incubate until necessary features of [Project Valhalla](#) become available as preview features. At that time, we will adapt the Vector API and its implementation to use them and then promote the Vector API from incubation to preview.

Auto-vectorization - JIT compiler automatically generates SIMD instructions for scalar Float16 loops, without requiring explicit Vector API calls.

- Addition, subtraction, division, multiplication, square root, and fused multiply/add operations on Float16 values are now auto-vectorized on supporting x64 CPUs.

In JDK 25+



JDK / JDK-8370691

Add new Float16Vector type and enable intrinsification of vector operations supported by auto-vectorizer

Open ▾

Export ▾

Details

Type:	Enhancement	Resolution:	Unresolved
Priority:	P4	Fix Version/s:	tbd
Affects Version/s:	26		
Component/s:	hotspot		
Labels:	intrinsic	vectorAPI	
Subcomponent:	compiler		
CPU:	generic		
OS:	generic		

Description

- Add a new HalfFloatVector type in addition to existing primitive vector types, maintaining o FloatVector type.
- Add necessary inline expander support.
 - Enable intrinsification for a few vector operations, namely ADD/SUB/MUL/DIV/MAX/MIN/F
- Use existing Float16 vector IR and backend support.

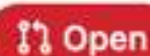
The idea here is to first be on par with Float16 auto-vectorization support.

People

Assignee:	Jatin Bhateja
Reporter:	Jatin Bhateja
Votes:	0 Vote for this issue
Watchers:	3 Start watching this issue

8370691: Add new Float16Vector type and enable intrinsification of vector operations supported by auto-vectorizer #28002

Code ▾



jatin-bhateja wants to merge 35 commits into `openjdk:master` from `jatin-bhateja:JDK-8370691`

Conversation 81

Commits 35

Checks 19

Files changed 157

+80,545 -28,604



jatin-bhateja commented on Oct 27, 2025 • edited by openjdk bot

Member

Add a new Float16Vector type and corresponding concrete vector classes, in addition to existing primitive vector types, maintaining operation parity with the FloatVector type.

- Add necessary inline expander support.
 - Enable intrinsification for a few vector operations, namely ADD/SUB/MUL/DIV/MAX/MIN/FMA.
- Use existing Float16 vector IR and backend support.
- Extended the existing VectorAPI JTREG test suite for the newly added Float16Vector operations.

The idea here is to first be at par with Float16 auto-vectorization support before intrinsifying new operations (conversions, reduction, etc).

The following are the performance numbers for some of the selected Float16Vector benchmarking kernels compared to equivalent auto-vectorized Float16OperationsBenchmark kernels.

Reviewers

4 more reviewers

PaulSandoz

eme64

RealFYang

Bhavana-Kilambi

Still in progress? Learn about draft PRs

Assignees

No one assigned

Labels

In JDK 26(?)+

Cost: FP16→FP32→FP16 conversion on every operation of Vector API!

```
// We have data in Float16 (50% memory)
Float16[] data = new Float16[1000];

// But to do SIMD, we have to convert:
for (int i = 0; i < data.length; i += 4) {
    // 1. Float16 → Float32 (conversion!)
    float[] chunk = new float[4];
    chunk[0] = data[i].floatValue(); // conversion
    chunk[1] = data[i+1].floatValue(); // conversion
    chunk[2] = data[i+2].floatValue(); // conversion
    chunk[3] = data[i+3].floatValue(); // conversion

    // 2. SIMD in Float32 (4 elements at once)
    FloatVector v = FloatVector.fromArray(SPECIES_128, chunk, 0);
    v = v.mul(2.0f).add(1.0f);

    // 3. Float32 → Float16 (conversion!)
    v.toArray(chunk, 0);
    data[i] = Float16.valueOf(chunk[0]); // conversion
    // ...
}
```



In JDK 26(?) +



Special Hardware!



March 27, 2006

Azul Systems® First to Deliver 48-Way Multicore Chip, Redefining Standard in Enterprise Computing



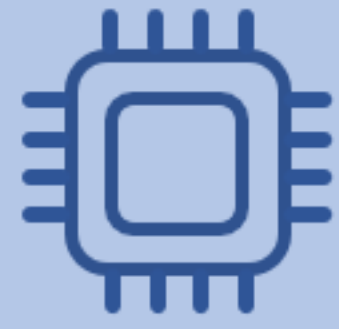
Mountain View, CA - March 27, 2006 - Azul Systems, Inc., the pioneer of the industry's first network attached processing solution designed to deliver compute and memory resources as a shared network service for transaction-intensive applications, today announced their next generation Vega™ 2 processor, the world's first and only single-chip 64-bit processor with 48 cache-coherent processor cores for use in the company's industry-leading, award winning compute appliances.

Designed by Azul and fabricated by Taiwan Semiconductor Manufacturing Company (TSMC,) the Vega 2 processor utilizes an advanced 90nm process, a 9-layer copper/low-k interconnect, and multi-threshold transistors for optimal performance-power efficiency. This revolutionary processor is poised to spur more energy-efficient, higher density, and larger capacity compute appliances that meet critical data center needs such as higher asset utilization, improved security, and lower-cost per transaction, all while reducing real-estate, electricity and associated management costs.

The Vega 2 processor integrates 48 cores and consists of 812 million transistors to enable future generations of Azul Compute Appliances to scale up to 768-way symmetric multiprocessing (SMP) systems with up to 768 GBytes of memory. By comparison, the world's largest and most power efficient SMP system built today is the Azul Compute Appliance 3840, which is based on the current generation Azul Vega 1 processor (a 24-way multicore chip) and provides 384 processor cores and 256 GBytes of memory in an 11U rack-mountable enclosure and consumes only 2700 watts of electricity.

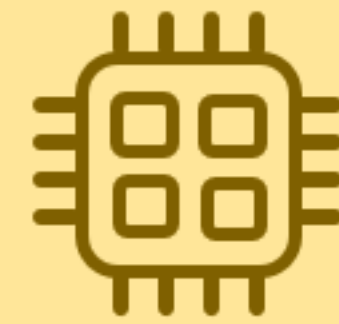
"Multicore chip architectures are quickly taking their place as the industry de facto standard for enterprise computing," said Vernon Turner, vice president and general manager, enterprise computing, IDC. "The Azul Vega chip is revolutionary in its sheer multicore scale and power and density characteristics. This design criterion will be critical as more and more next generation enterprise systems come to market and deliver compute resources as a shared network service across all types of heterogeneous workloads."





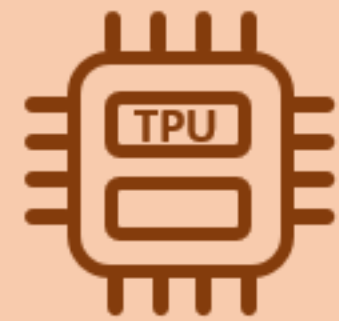
CPU

- Small models
- Small datasets
- Useful for design space exploration



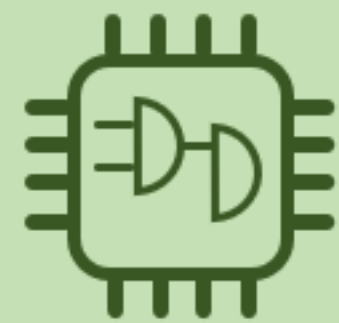
GPU

- Medium-to-large models, datasets
- Image, video processing
- Application on CUDA or OpenCL



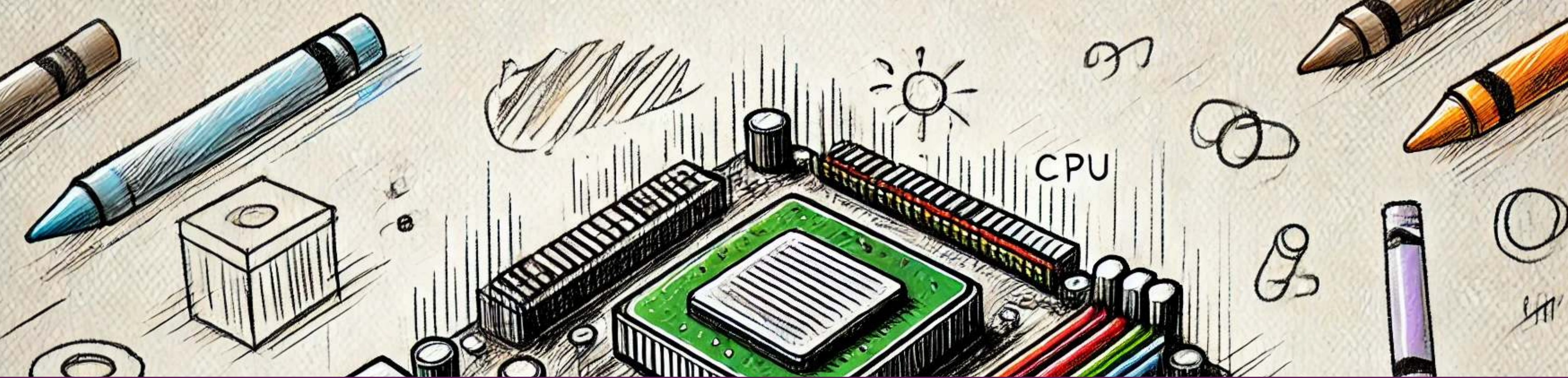
TPU

- Matrix computations
- Dense vector processing
- No custom TensorFlow operations

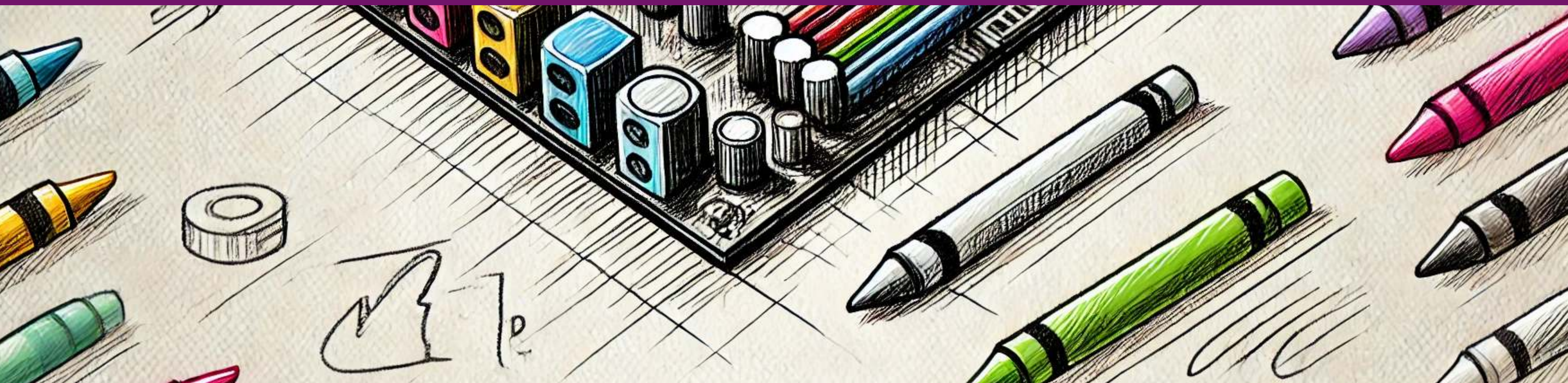


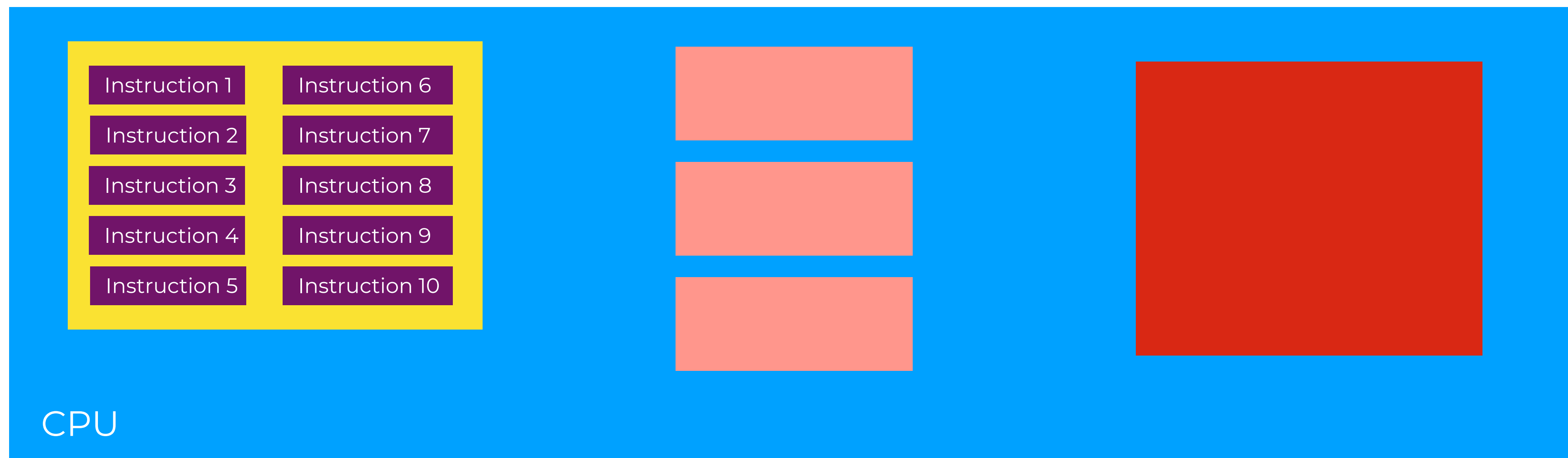
FPGA

- Large datasets, models
- Compute intensive applications
- High performance, high perf./cost ratio

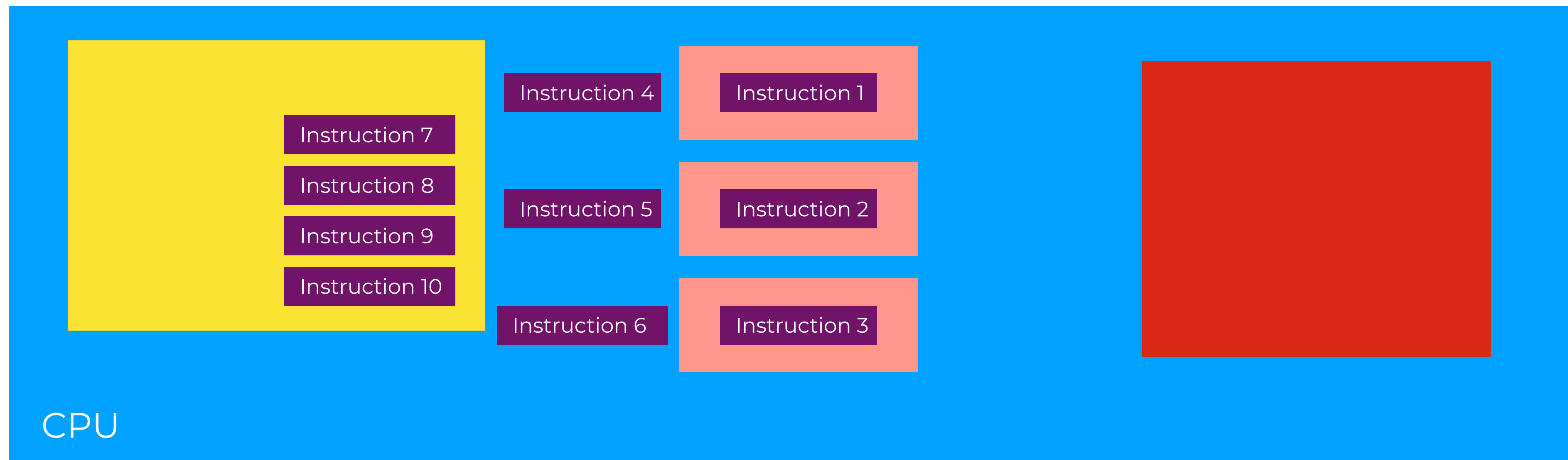


OVERLY SIMPLIFIED EXAMPLE ALERT!

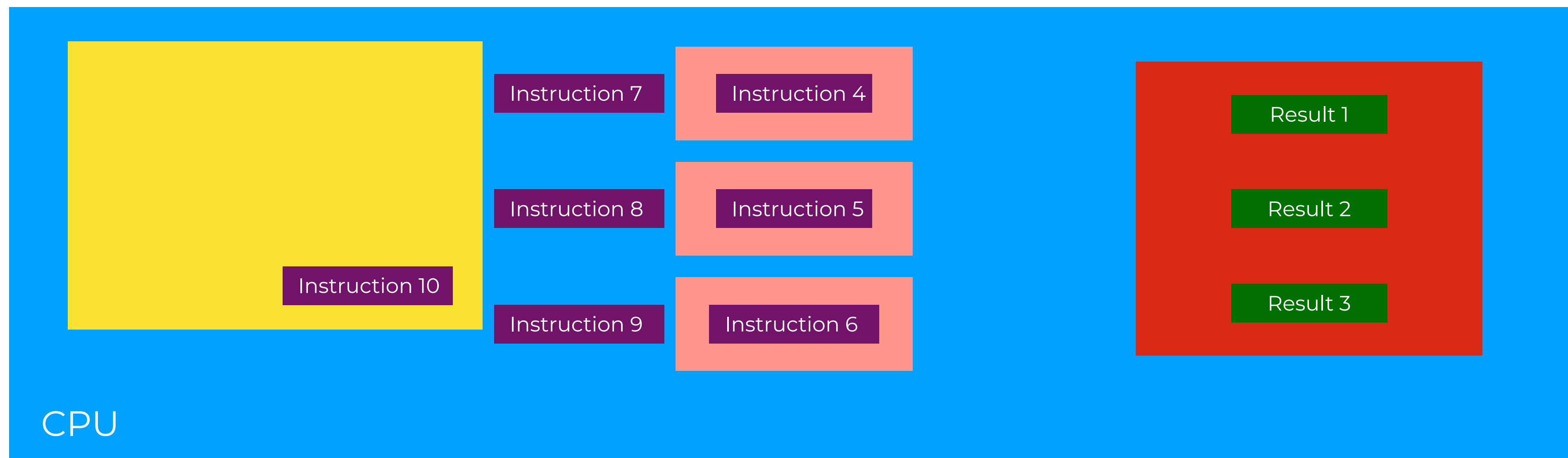




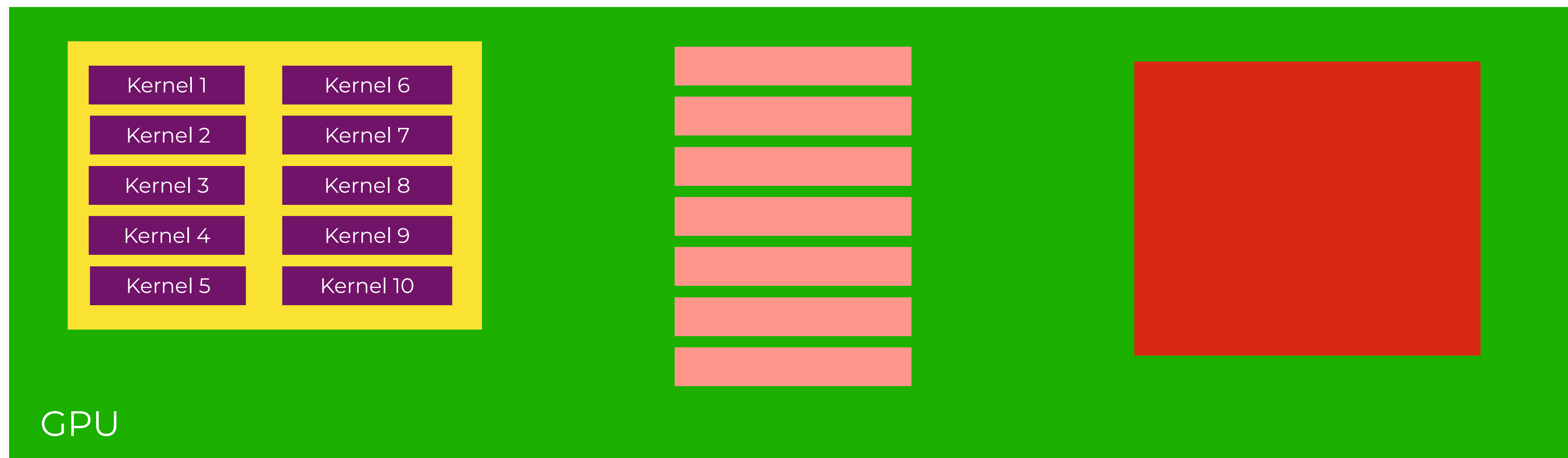
Model Inference run on GPU



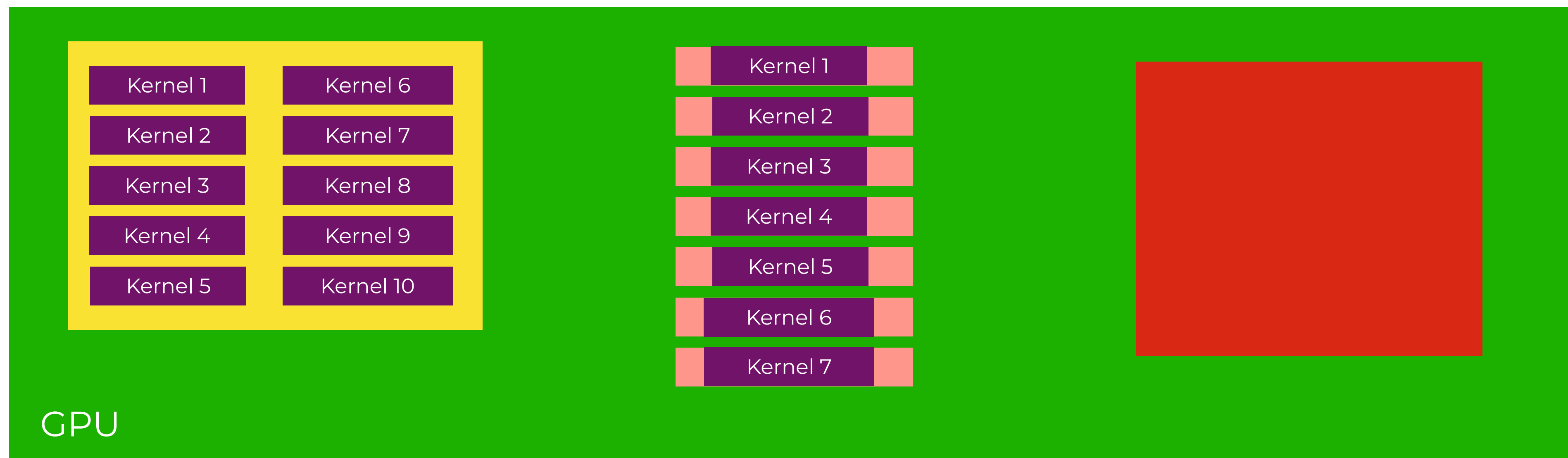
Model Inference run on GPU



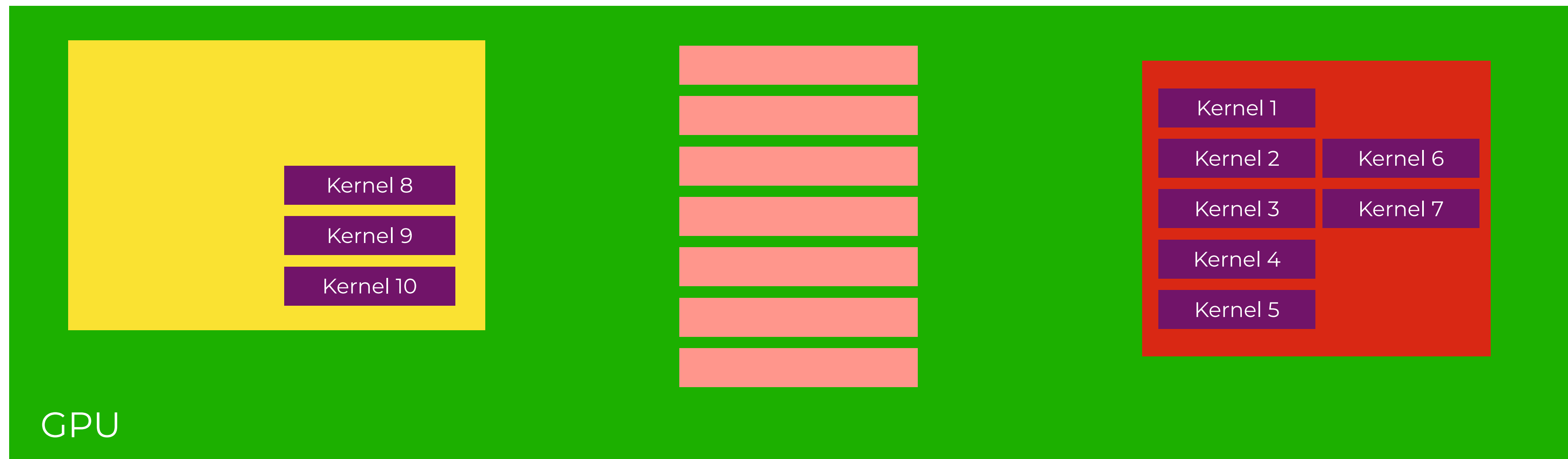
Model Inference run on GPU



Model Inference run on GPU



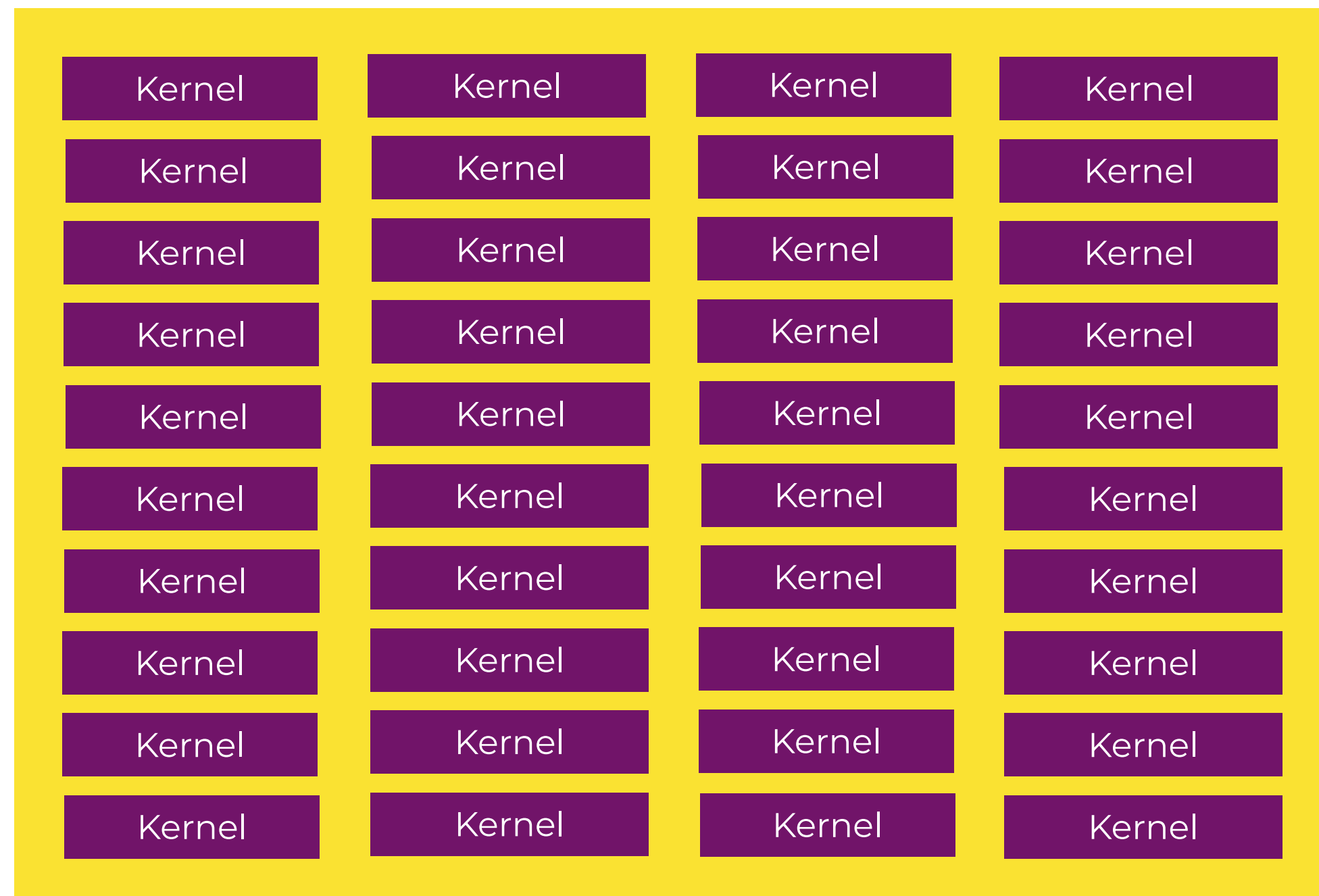
Model Inference run on GPU



Model Inference run on GPU



NVIDIA.COM



NDGrid (n-dimensional grid) refers to a multi-dimensional grid of threads used for parallel computation on a GPU.

The GPU runs threads in grids with multiple dimensions, allowing kernel processing in different dimensions (e.g., 1D, 2D, 3D) simultaneously.

What is Kernel and NDGrid?

macOS Mojave 10.14 Release Notes

Update your apps to use new features, and test your apps against API changes.

Open GL and Open CL

Deprecations

- The APIs in the OpenGL and OpenCL frameworks are deprecated and remain present for compatibility purposes. Transition to Metal if your app is using OpenGL or OpenCL.

MacOS has Metal

You write	Compile to	Consumed by	Runs on
CUDA C++	PTX (and/or cubin)	CUDA driver	NVIDIA GPUs
OpenCL C	vendor IR / ISA or SPIR-V	OpenCL driver	many vendors
GLSL/HLSL (Vulkan compute)	SPIR-V	Vulkan driver	many vendors
SPIR-V (direct)	(driver compiles to ISA)	Vulkan/OpenCL driver	many vendors

Different representations

macOS Mojave 10.14 Release Notes

Update your apps to use new features, and test your apps against API changes.

Open GL and Open CL

Deprecations

- The APIs in the OpenGL and OpenCL frameworks are deprecated and remain present for compatibility purposes. Transition to Metal if your app is using OpenGL or OpenCL.

MacOS has Metal

The first steps in GPU programming in Java began with projects that exposed CUDA and OpenCL APIs through **JNI**.

Projects such as JCuda, and JOpenCL allows the use of native GPU libraries from within Java, but require the programmer to manage GPU tasks in a rather low-level manner.

Calculating the square of each element in an array

```
cudaMalloc(deviceInput, n * Sizeof.FLOAT);
cudaMalloc(deviceOutput, n * Sizeof.FLOAT);

// Copyin data from CPU to GPU (CPU -> GPU)
cudaMemcpy(deviceInput, Pointer.to(hostInput), n * Sizeof.FLOAT, cudaMemcpyHostToDevice);

// Defining CUDA kernel as String
String cudaKernel =
    "extern \"C\" " + "\n" +
    "__global__ void squareKernel(float *input, float *output) {" + "\n" +
    "    int idx = threadIdx.x;" + "\n" +
    "    output[idx] = input[idx] * input[idx];" + "\n" +
    "}";

// Kernel Compiling
CUmodule module = new CUmodule();
CUfunction function = new CUfunction();
JCudaDriver.cuModuleLoadData(module, cudaKernel);
JCudaDriver.cuModuleGetFunction(function, module, "squareKernel");
```

Kernels are embedded as strings in the code, and programmers had to manually manage data movement and task execution.

```
[JCuda] =====  
[JCuda] JCuda Device Info Demo  
[JCuda] =====  
[JCuda] os.name=Linux  
[JCuda] os.arch=amd64  
[JCuda] java.version=21.0.2  
[JCuda] java.vm.name=OpenJDK 64-Bit Server VM  
[JCuda]
```

DEMO 7

```
[JCuda] Searching for CUDA devices...  
[JCuda] cuda.deviceCount=1  
[JCuda] Querying device 0...  
[JCuda] cuda.device[0].name=Tesla T4  
[JCuda] cuda.device[0].computeCapability=7.5  
  
[JCuda] Demo completed successfully!
```

`./gradlew :demos:jcuda:run`

```
[JCuda] =====  
[JCuda] JCuda Device Info Demo  
[JCuda] =====  
[JCuda] os.name=Linux  
[JCuda] os.arch=amd64  
[JCuda] java.version=21.0.2  
[JCuda] java.vm.name=OpenJDK 64-Bit Server VM  
[JCuda] =====  
  
[JCuda] Enabling JCuda exceptions...  
[JCuda] Initializing CUDA driver (cuInit)...  
[JCuda] CUDA driver initialized in 245ms  
[JCuda] Getting driver version...  
[JCuda] cuda.driverVersion=12080  
[JCuda] Counting CUDA devices...  
[JCuda] cuda.deviceCount=1  
[JCuda] Querying device 0...  
[JCuda] cuda.device[0].name=Tesla T4  
[JCuda] cuda.device[0].computeCapability=7.5  
  
[JCuda] Demo completed successfully!
```

Works on Linux

```
[JCuda] =====
[JCuda] JCuda Device Info Demo
[JCuda] =====
[JCuda] os.name=Linux
[JCuda] os.arch=amd64
[JCuda] java.version=21.0.2
[JCuda] java.vm.name=OpenJDK 64-Bit Server VM
[JCuda] =====

[JCuda] Enabling JCuda exceptions...
[JCuda] Initializing CUDA driver (cuInit)...
[JCuda] CUDA driver initialized in 245ms
[JCuda] Getting driver version...
[JCuda] cuda.driverVersion=12080
[JCuda] Counting CUDA devices...
[JCuda] cuda.deviceCount=1
[JCuda] Querying device 0...
[JCuda] cuda.device[0].name=Tesla T4
[JCuda] cuda.device[0].computeCapability=7.5

[JCuda] Demo completed successfully!
```

```
./gradlew :demos:jcuda:run

> Task :demos:jcuda:run FAILED
[JCuda] =====
[JCuda] JCuda Device Info Demo
[JCuda] =====
[JCuda] os.name=Mac OS X
[JCuda] os.arch=aarch64
[JCuda] java.version=25
[JCuda] java.vm.name=OpenJDK 64-Bit Server VM
[JCuda] =====

[JCuda] Enabling JCuda exceptions...
[JCuda] ERROR: jcuda.CudaException: This CUDA version is not available on MacOS

[JCuda] Requirements:
[JCuda] - NVIDIA CUDA driver installed (nvidia-smi should work)
[JCuda] - Matching native JCuda binaries for your OS/arch
Exception in thread "main" java.lang.RuntimeException: JCuda initialization failed
    at conf.jvm.jcuda.JCudaInfoDemo.run(JCudaInfoDemo.java:79)
    at conf.jvm.jcuda.JCudaInfoDemo.main(JCudaInfoDemo.java:18)
Caused by: java.lang.ExceptionInInitializerError
    at conf.jvm.jcuda.JCudaInfoDemo.run(JCudaInfoDemo.java:34)
    ... 1 more
Caused by: jcuda.CudaException: This CUDA version is not available on MacOS
    at jcuda.LibUtilsCuda.loadLibrary(LibUtilsCuda.java:65)
    at jcuda.driver.JCudaDriver.<clinit>(JCudaDriver.java:350)
    ... 2 more

FAILURE: Build failed with an exception.

* What went wrong:
Execution failed for task ':demos:jcuda:run'.
> Process 'command '/Users/askowronski/.sdkman/candidates/java/25-tem/bin/java'' finished with non-zero exit value 1

* Try:
> Run with --stacktrace option to get the stack trace.
> Run with --info or --debug option to get more log output.
> Run with --scan to get full insights from a Build Scan (powered by Develocity).
> Get more help at https://help.gradle.org.

BUILD FAILED in 907ms
2 actionable tasks: 1 executed, 1 up-to-date
```

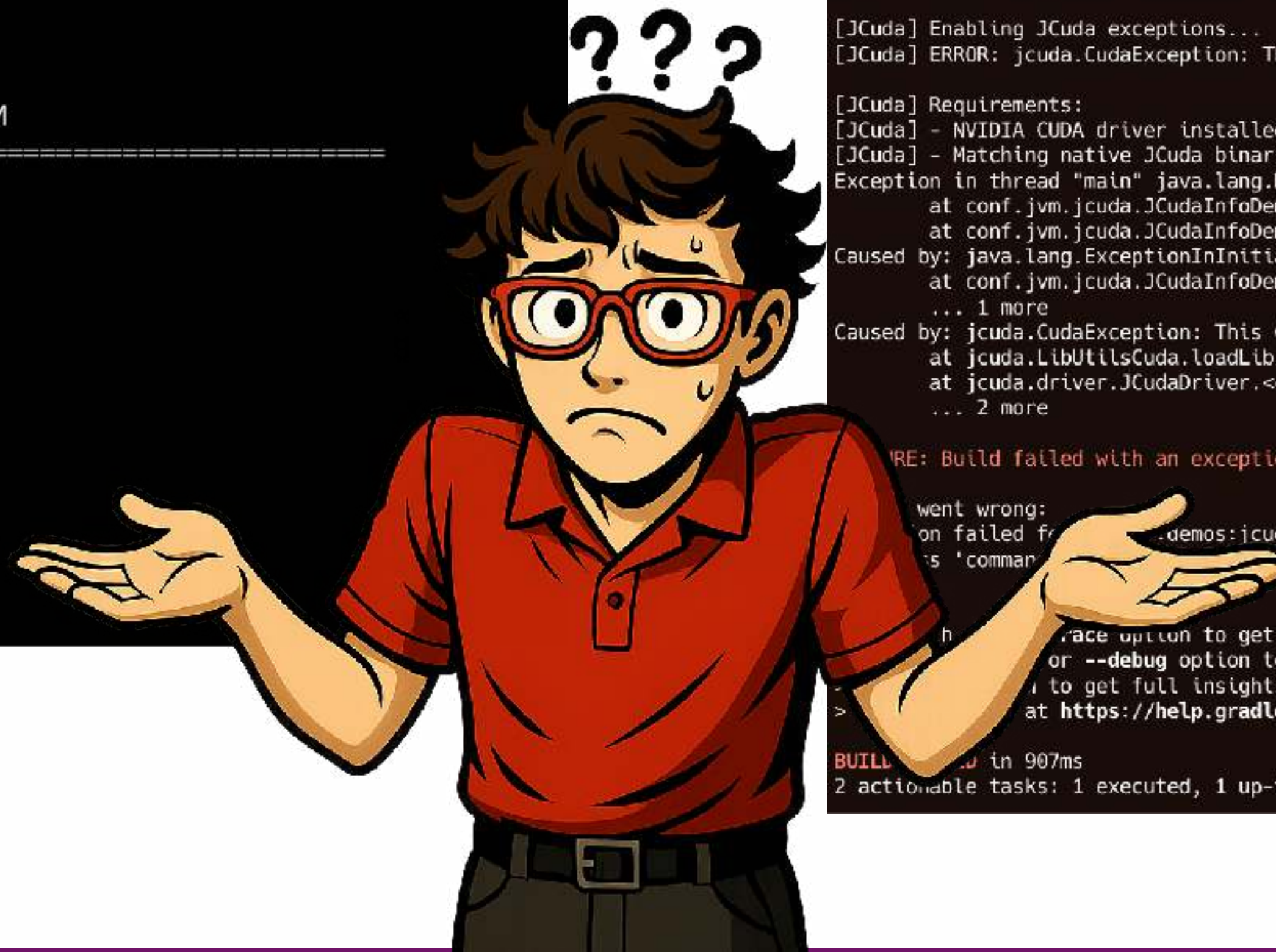
Do not work on Mac

NVIDIA stopped supporting CUDA on macOS after 10.13 High Sierra

```
[JCuda] =====
[JCuda] JCuda Device Info Demo
[JCuda] =====
[JCuda] os.name=Linux
[JCuda] os.arch=amd64
[JCuda] java.version=21.0.2
[JCuda] java.vm.name=OpenJDK 64-Bit Server VM
[JCuda] =====

[JCuda] Enabling JCuda exceptions...
[JCuda] Initializing CUDA driver (cuInit)...
[JCuda] CUDA driver initialized in 245ms
[JCuda] Getting driver version...
[JCuda] cuda.driverVersion=12080
[JCuda] Counting CUDA devices...
[JCuda] cuda.deviceCount=1
[JCuda] Querying device 0...
[JCuda] cuda.device[0].name=Tesla T4
[JCuda] cuda.device[0].computeCapability=7.5

[JCuda] Demo completed successfully!
```



```
[JCuda] =====
[JCuda] os.name=Mac OS X
[JCuda] os.arch=aarch64
[JCuda] java.version=25
[JCuda] java.vm.name=OpenJDK 64-Bit Server VM
[JCuda] =====

[JCuda] Enabling JCuda exceptions...
[JCuda] ERROR: jcuda.CudaException: This CUDA version is not available on MacOS

[JCuda] Requirements:
[JCuda] - NVIDIA CUDA driver installed (nvidia-smi should work)
[JCuda] - Matching native JCuda binaries for your OS/arch
Exception in thread "main" java.lang.RuntimeException: JCuda initialization failed
    at conf.jvm.jcuda.JCudaInfoDemo.run(JCudaInfoDemo.java:79)
    at conf.jvm.jcuda.JCudaInfoDemo.main(JCudaInfoDemo.java:18)
Caused by: java.lang.ExceptionInInitializerError
    at conf.jvm.jcuda.JCudaInfoDemo.run(JCudaInfoDemo.java:34)
    ... 1 more
Caused by: jcuda.CudaException: This CUDA version is not available on MacOS
    at jcuda.LibUtilsCuda.loadLibrary(LibUtilsCuda.java:65)
    at jcuda.driver.JCudaDriver.<clinit>(JCudaDriver.java:350)
    ... 2 more

FAILURE: Build failed with an exception.

What went wrong:
Execution failed for task ':demos:jcuda:run'.
> Command-line tool 'java' from SDK candidate '/usr/local/sdkman/candidates/java/25-tem/bin/java' finished with non-zero exit value 1

To see the full stack trace of the errors, use --stacktrace option to get the stack trace.
To get the full log of the build, use --debug option to get more log output.
To get full insights from a Build Scan (powered by Develocity),
enable Build Scan in your IDE or at https://help.gradle.org.

BUILD FAILED in 907ms
2 actionable tasks: 1 executed, 1 up-to-date
```

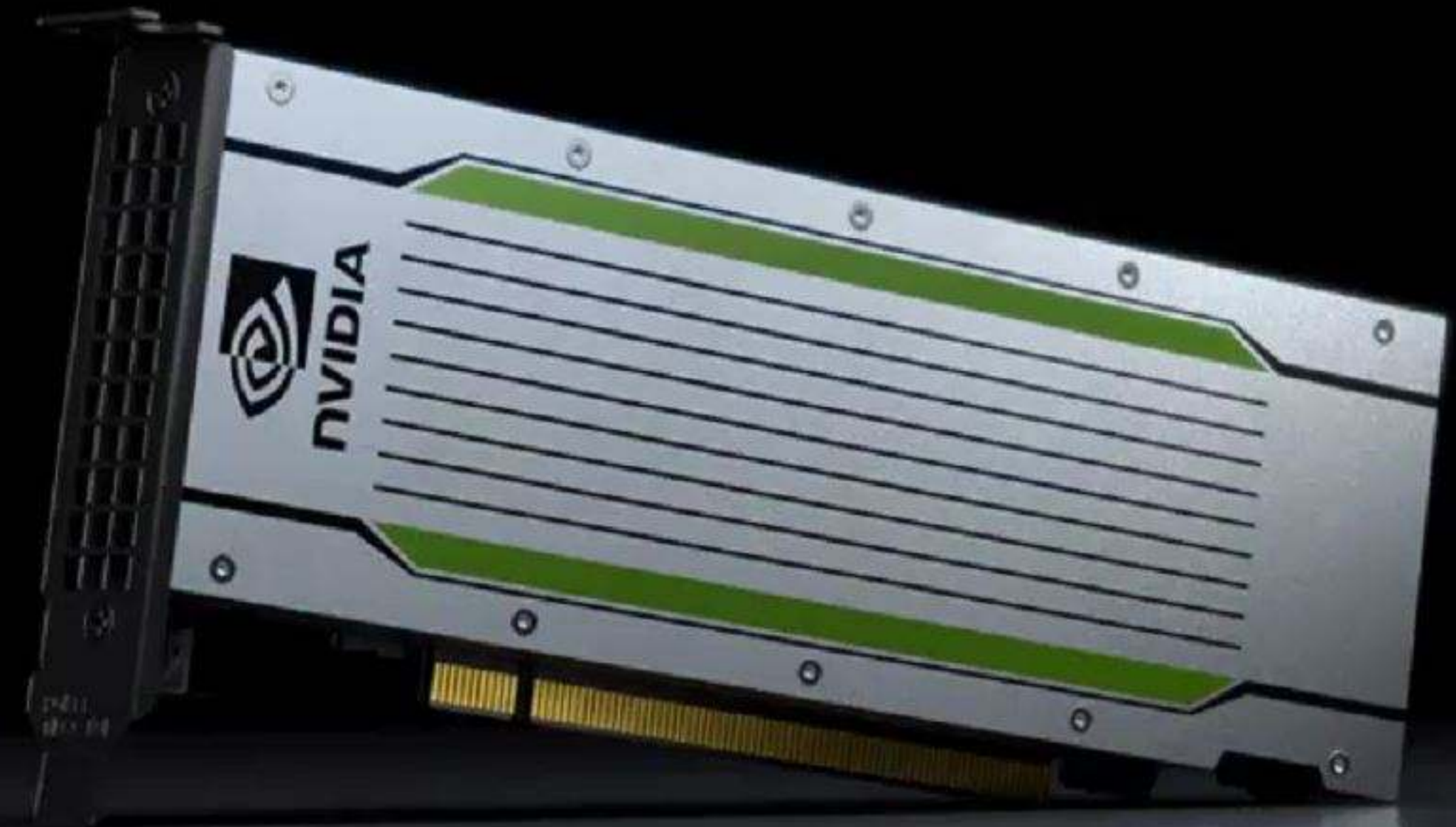
CUDA is not available on macOS

ANNOUNCING TESLA T4 ON GCP

THE CLOUD SCALE-OUT ACCELERATOR

320 Turing Tensor Cores
2,560 CUDA Cores
65 FP16 TFLOPS | 130 INT8 TOPS | 260 INT4 TOPS
16GB | 320GB/s
70 W

Now Available Early Access



In repo there are benchmarks to run on GCP

Aparapi and Rootbeer are projects that allowed the expression of kernels **directly in Java code**, eliminating the need to manually write CUDA or OpenCL code.

They don't provide a direct API for defining **ndgrid** in the way CUDA or OpenCL does

```
import com.aparapi.Kernel;
import com.aparapi.Range;

public class AparapiExample {

    public static void main(String[] args) {
        // Array size
        final int size = 10;
        final float[] input = new float[size];
        final float[] output = new float[size];

        // Fill the input array with sample data
        for (int i = 0; i < size; i++) {
            input[i] = i;
        }

        // Define the kernel in Java, which will be processed on the GPU
        Kernel kernel = new Kernel() {
            @Override
            public void run() {
                int id = getGlobalId();
                output[id] = input[id] * input[id];
            }
        };

        // Define the range (number of threads)
        Range range = Range.create(size);

        kernel.execute(range);

        System.out.println("Results:");
        for (int i = 0; i < size; i++) {
            System.out.println(input[i] + " squared = " + output[i]);
        }

        // Clean up the kernel resources
        kernel.dispose();
    }
}
```

Kernels expressed in Java

1. Lack of support for proper data types
2. Lack of support for diverse hardware
3. In-efficient transpiled code...
4. ...or manual hacks

```
import com.aparapi.Kernel;
import com.aparapi.Range;

public class AparapiExample {

    public static void main(String[] args) {
        // Array size
        final int size = 10;
        final float[] input = new float[size];
        final float[] output = new float[size];

        // Fill the input array with sample data
        for (int i = 0; i < size; i++) {
            input[i] = i;
        }

        // Define the kernel in Java, which will be processed on the GPU
        Kernel kernel = new Kernel() {
            @Override
            public void run() {
                int id = getGlobalId();
                output[id] = input[id] * input[id];
            }
        };

        // Define the range (number of threads)
        Range range = Range.create(size);

        kernel.execute(range);

        System.out.println("Results:");
        for (int i = 0; i < size; i++) {
            System.out.println(input[i] + " squared = " + output[i]);
        }

        // Clean up the kernel resources
        kernel.dispose();
    }
}
```

Kernels expressed in Java

Library provides a way to compile
Scala 3 DSL to SPIR-V and to run it
with Vulkan runtime on GPUs.

It is multiplatform.

In practice, it means almost all
moderately modern devices from
most manufacturers including
Nvidia, AMD, Intel, Apple.

Cyfra

Library provides a way to compile Scala 3 DSL to SPIR-V and to run it with Vulkan runtime on GPUs.

It is multiplatform. It works on:

- Linux, Windows, and Mac (for Mac requires installation of moltenvk).
- Any dedicated or integrated GPUs that support Vulkan. In practice, it means almost all moderately modern devices from most manufacturers including Nvidia, AMD, Intel, Apple.

Animations

Included Foton library provides a clean and fun way to animate functions and ray traced scenes.

Examples

Ray traced animation



[code](#) (this is API usage, to see ray tracing implementation look at [RtRenderer](#))

Cyfra - Kernels expressed in Scala!

What makes Cyfra interesting:

- Pure Scala/JVM - No JNI, no native code
- GPU DSL - Write GPU kernels in Scala
- SPIR-V - Compiled to portable GPU bytecode
- Cross-platform - Works on NVIDIA, AMD, Intel, Apple (Vulkan/MoltenVK)

Cyfra

Library provides a way to compile Scala 3 DSL to SPIR-V and to run it with Vulkan runtime on GPUs.

It is multiplatform. It works on:

- Linux, Windows, and Mac (for Mac requires installation of moltenvk).
- Any dedicated or integrated GPUs that support Vulkan. In practice, it means almost all moderately modern devices from most manufacturers including Nvidia, AMD, Intel, Apple.

Animations

Included Foton library provides a clean and fun way to animate functions and ray traced scenes.

Examples

Ray traced animation



[code](#) (this is API usage, to see ray tracing implementation look at [RtRenderer](#))

Cyfra - Kernels expressed in Scala!

LLama inference implementation in Scala #82

[Open](#) **szymon-rd** wants to merge 4 commits into `main` from `llm.scala`

Conversation 0 Commits 4 Checks 0 Files changed 76

 **szymon-rd** commented 2 days ago • edited Member ...

WIP (shows weird conflicts because to be rebased)
DSL to be cleaned up, GShared is weird now.
Some sources are to be removed.



 **LLama GPU Implementation** d200629

 **szymon-rd** force-pushed the `llm.scala` branch from `8b5671a` to `d200629` yesterday Compare

 **szymon-rd** added 3 commits 5 hours ago

 **Fix SPIR-V dominance bug in loop compilation and add Float16 support** efb8ca1

 **Cache interpret results in ExecutionHandler** baec55c

 **Cache full execution: command buffer, descriptor sets, result** 1e42040



Cyfra - Kernels expressed in Scala!

LLama inference implementation in Scala #82

 Open **szymon-rd** wants to merge 4 commits into `main` from `llm.scala` 

 Conversation 0  Commits 4  Checks 0  Files changed 76



szymon-rd commented 2 days ago · edited ▾

Member ...

WIP (shows weird conflicts because to be rebased)
DSL to be cleaned up, GShared is weird now.
Some sources are to be removed.



DEMO 8

 **szymon-rd** added 3 commits 5 hours ago

-   Fix SPIR-V dominance bug in loop compilation and add Float16 support ... efb8ca1
-   Cache interpret results in ExecutionHandler ... baec55c
-   Cache full execution: command buffer, descriptor sets, result ... 1e42040



`./cyfra-demo/scripts/run-cyfra-llama.sh`



Project Sumatra

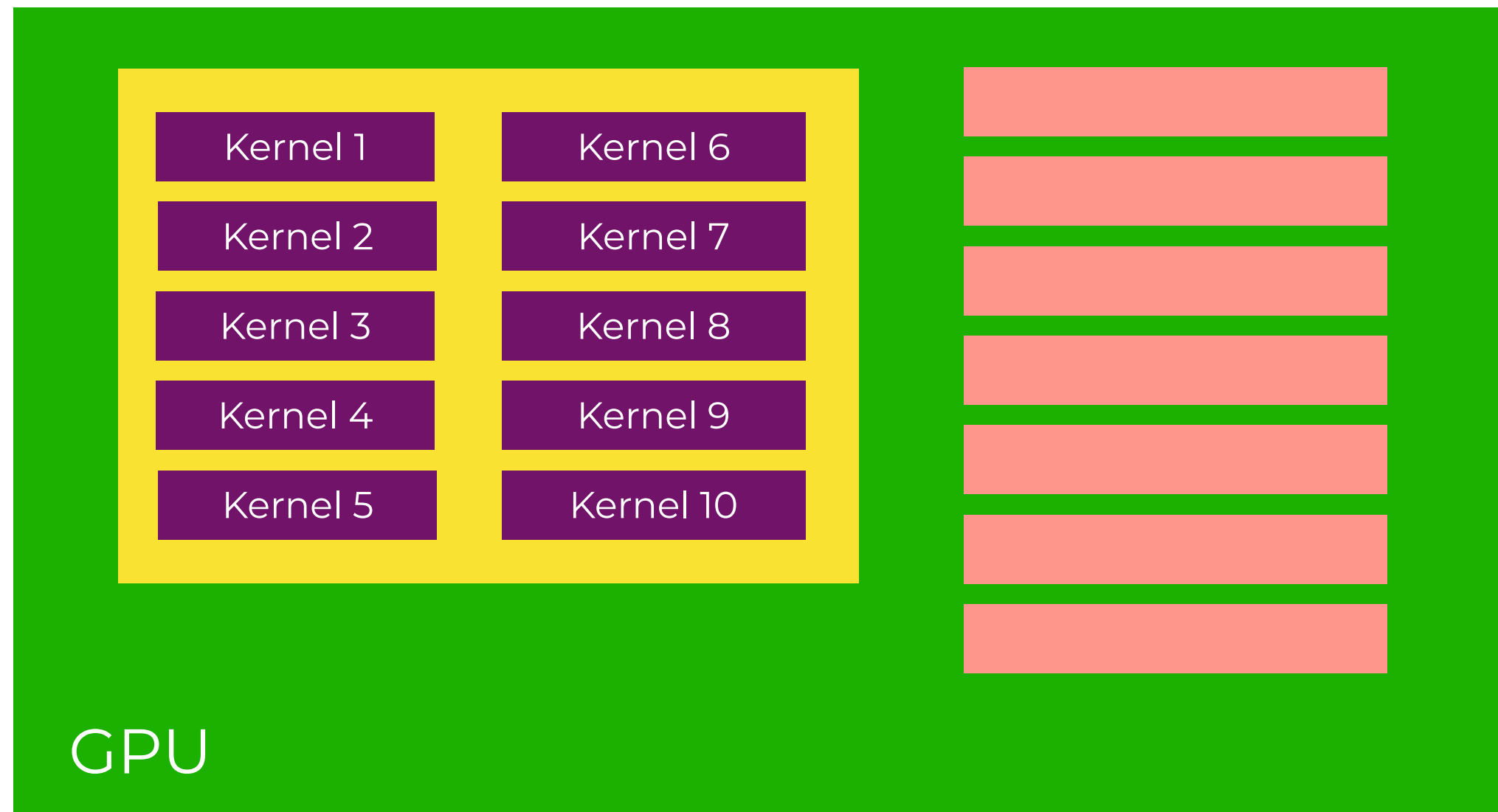




Parallel processing



Project Sumatra

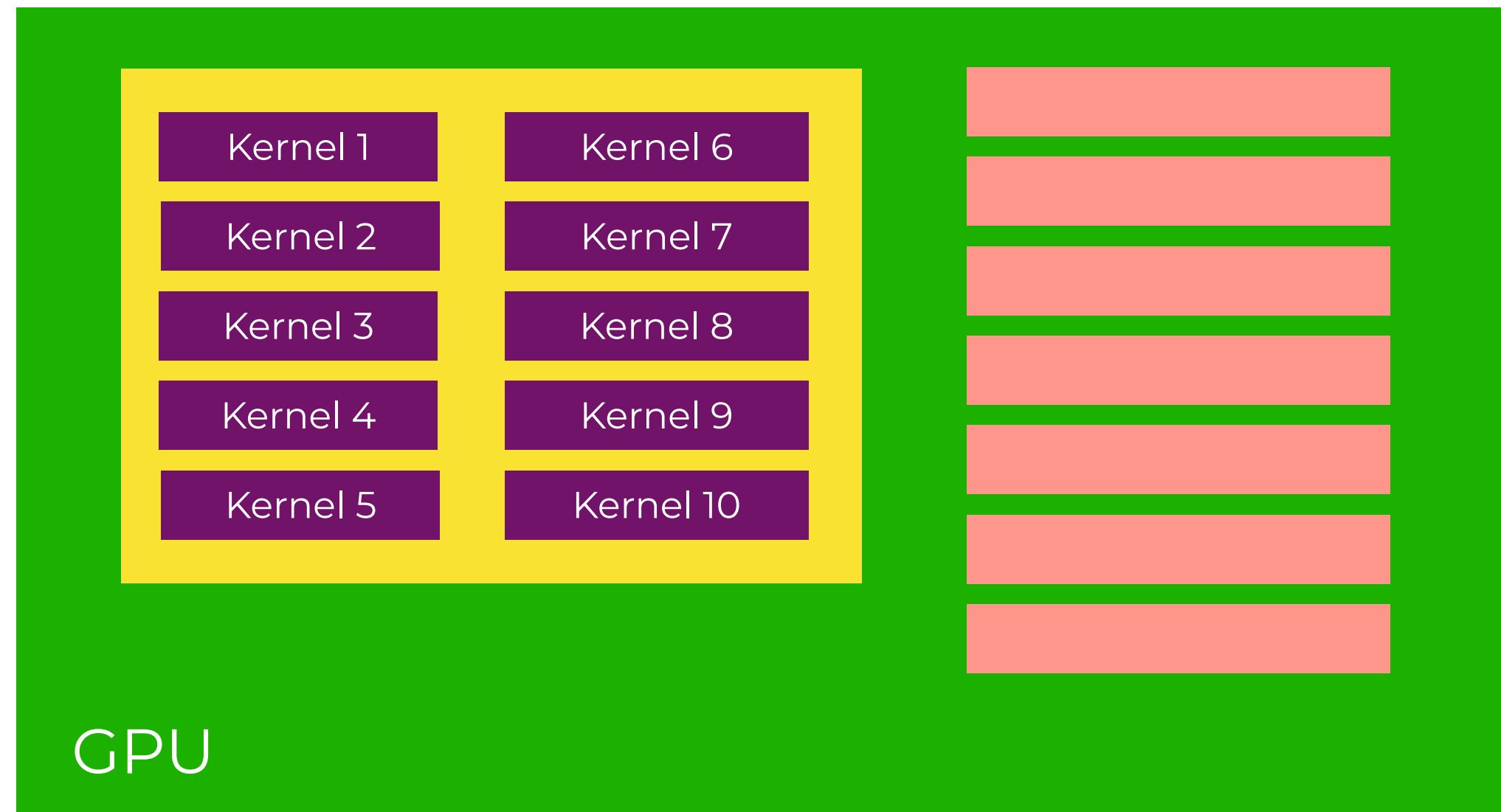


Streams.parallel()



```
public class ParallelStreamExample {  
    public static void main(String[] args) {  
  
        int[] arr = new int[10000];  
        for (int i = 0; i < arr.length; i++)  
        {  
            arr[i] = i;  
        }  
  
        int sum = Arrays.stream(arr)  
                        .parallel()  
                        .sum();  
  
        System.out.println("Sum: " + sum);  
    }  
}
```

Project Sumatra

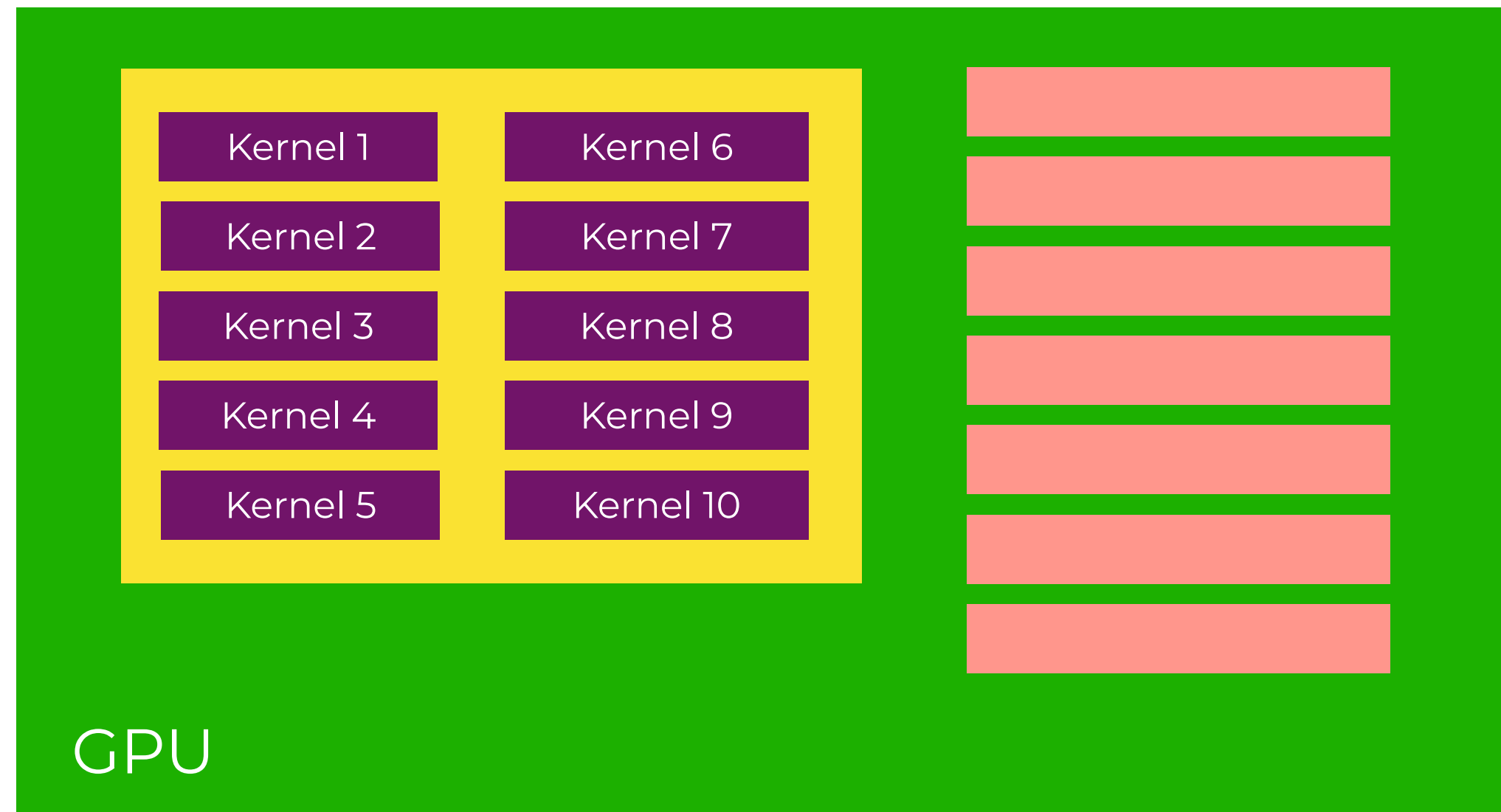


Streams.parallel()

```
public class ParallelStreamExample {  
    public static void main(String[] args) {  
  
        int[] arr = new int[10000];  
        for (int i = 0; i < arr.length; i++)  
        {  
            arr[i] = i;  
        }  
  
        int sum = Arrays.stream(arr)  
            .parallel()  
            .sum();  
  
        System.out.println("Sum: " + sum);  
    }  
}
```

GraalVM™

Project Sumatra



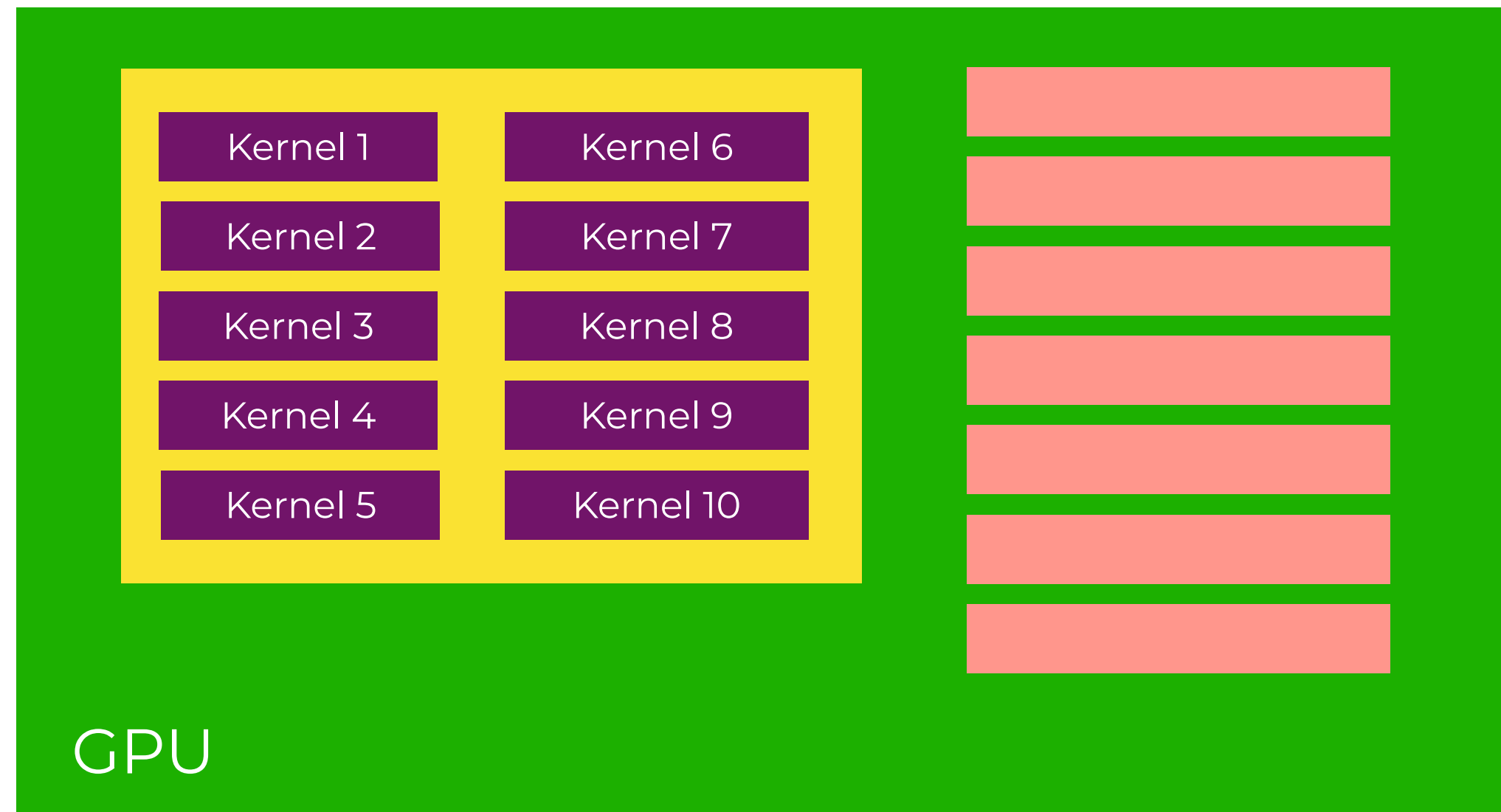
Streams.parallel()

```
public class ParallelStreamExample {  
    public static void main(String[] args) {  
  
        int[] arr = new int[10000];  
        for (int i = 0; i < arr.length; i++)  
        {  
            arr[i] = i;  
        }  
  
        int sum = Arrays.stream(arr)  
            .parallel()  
            .sum();  
  
        System.out.println("Sum: " + sum);  
    }  
}
```

GraalVM™



Project Sumatra



`Streams.parallel()`

Data transfer between JVM and GPU can reduce performance benefits, especially for small tasks.



JDK Sumatra's approach to GPU acceleration has been largely superseded by **GraalVM**'s capabilities, which provide broader optimization features (e.g., native image and polyglot execution) and better performance across various platforms



Project Sumatra

This primary goal of this project is to enable Java applications to take advantage of graphics processing units (GPUs) and accelerated processing units (APUs)--whether they are discrete devices or integrated with a CPU--to improve performance.

We expect the initial focus to be on the Hotspot JVM, enabling code generation, runtime support and garbage collection on GPUs. Once basic capabilities are in place in the JVM, we will also examine how to best expose GPU support to application and/or library developers, initially leveraging the new Java 8 Lambda language and library features. As the project progress, we may identify challenges with the Java API and constructs which may lead to new language, JVM or library extensions that will need standardization under the JCP process. While the Java language is an obvious focus, we also anticipate that this project will provide guidance for other JVM-hosted languages such as JavaScript/Nashorn, Scala and JRuby.

You can follow the project and get involved on the [sumatra-dev](#) mailing list.

Related projects and material

- [Aparapi](#) - API for data parallel Java
- [Rootbeer](#) - a compiler that makes it easier to program GPUs with Java
- [Arrays 2.0](#) - a [JVM Language Summit](#) talk by John Rose

This Project is sponsored by the [HotSpot Group](#).

Project Sumatra



Next Level: TornadoVM



API

Task

Annotations

Task Graph



```
Task task = new Task("matrixMultiply", this::multiplyMatrices);|
```

Next Level: TornadoVM

API

Task

Annotations

Task Graph

```
@Parallel
public void multiplyMatrices(float[][] A, float[][] B, float[][] C)
{
    for (int i = 0; i < A.length; i++) {
        for (int j = 0; j < B[0].length; j++) {
            for (int k = 0; k < B.length; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}
```

Next Level: TornadoVM

API

Task

Annotations

Task Graph



```
TaskGraph graph = new TaskGraph("exampleGraph")
    .task("task1", this::computation1, input1, output1)
    .task("task2", this::computation2, input2, output2)
    .task("task3", this::computation3, output1, output3)
    .streamOut(output3);
```

Next Level: TornadoVM

API

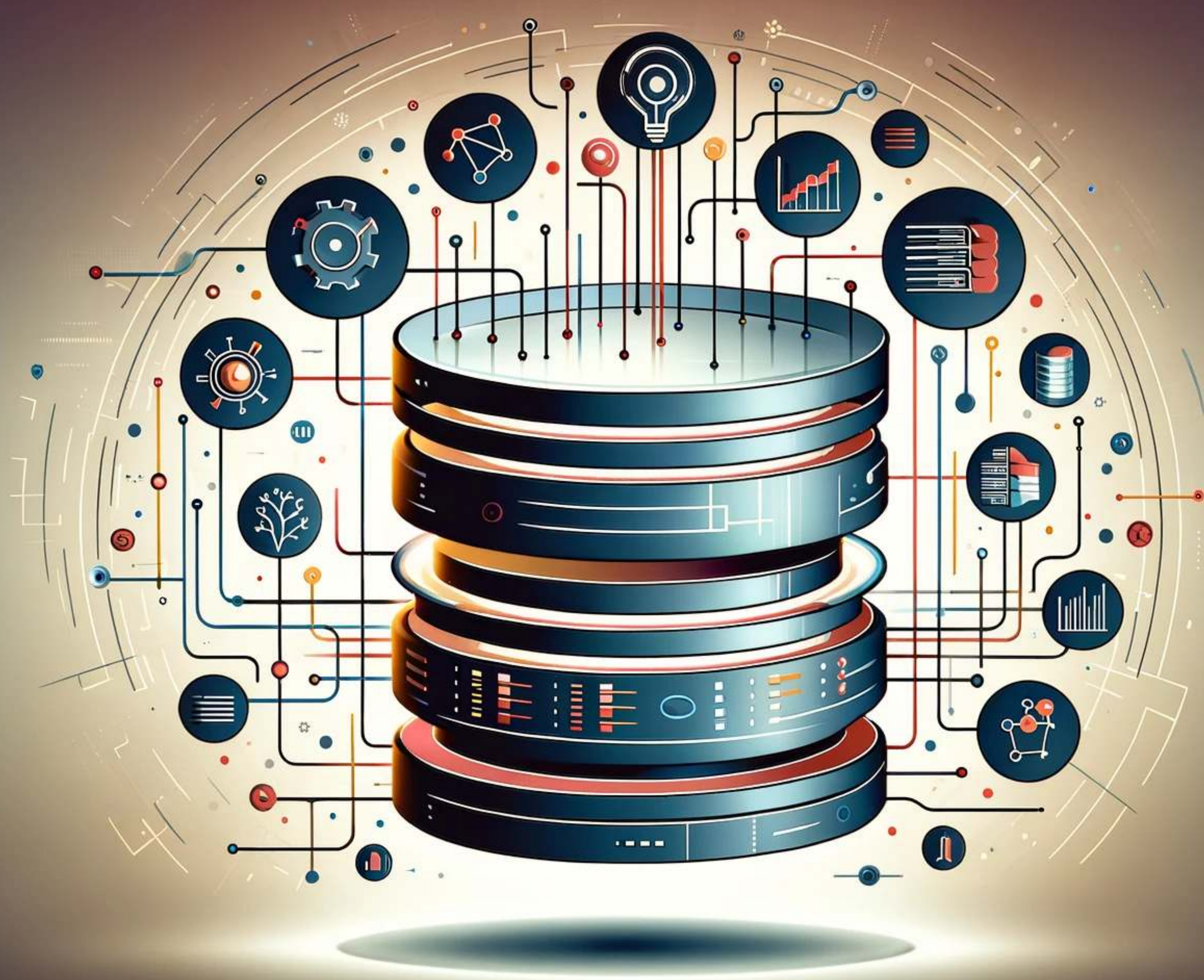
Task

Annotations

Task Graph

Graph Optimizer

Next Level: TornadoVM



API

Task

Annotations

Task Graph

Graph Optimizer

Execution Engine

Next Level: TornadoVM

API

Task

Annotations

Task Graph

Graph Optimizer

Execution Engine

CPU

GPU

FPGA

Next Level: TornadoVM

API

Task

Annotations

Task Graph

Graph Optimizer

Execution Engine

CPU

GPU

FPGA

Tornado JIT Compiler

Next Level: TornadoVM



GraalVM

API

Task

Annotations

Task Graph

Graph Optimizer

Execution Engine

CPU

GPU

FPGA

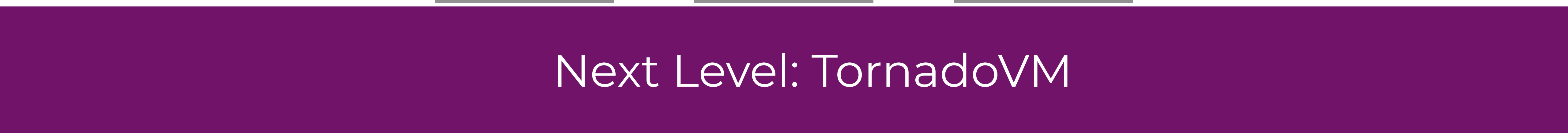
Tornado JIT Compiler

CUDA

OpenCL

PTX

Next Level: TornadoVM



- Task Graphs - Direct abstraction for NDGrid
- A lot of different backends (not only CUDA)
- Managed Memory
- JIT Support through GraalVM

```
import uk.ac.manchester.tornado.api.TaskSchedule;
import uk.ac.manchester.tornado.api.collections.types.FloatArray;

public class TornadoSquareExample {

    public static void main(String[] args) {
        // Rozmiar tablicy
        final int size = 1024;
        float[] input = new float[size];
        float[] output = new float[size];

        // Inicjalizacja tablicy wejściowej
        for (int i = 0; i < size; i++) {
            input[i] = i;
        }

        // Zadanie obliczające kwadrat każdego elementu w tablicy
        TaskSchedule task = new TaskSchedule("s0")
            .task("t0", (float[] input, float[] output) -> {
                for (int i = 0; i < input.length; i++) {
                    output[i] = input[i] * input[i];
                }
            }, input, output);

        // Wykonanie zadania na GPU lub innym urządzeniu
        task.execute();

        // Wyświetlenie wyników
        for (int i = 0; i < size; i++) {
            System.out.println(input[i] + " squared = " + output[i]);
        }
    }
}
```

Next Level: TornadoVM

- Task Graphs - Direct abstraction for NDGrid

- A lot of different backends (not

DEMO 9

- JIT Support through GraalVM

```
import uk.ac.manchester.tornado.api.TaskSchedule;
import uk.ac.manchester.tornado.api.collections.types.FloatArray;

public class TornadoSquareExample {

    public static void main(String[] args) {
        // Rozmiar tablicy
        final int size = 1024;
        float[] input = new float[size];
        float[] output = new float[size];

        // Inicjalizacja tablicy wejściowej
        for (int i = 0; i < size; i++) {
            input[i] = i;
        }
    }
}
```

```
// Wykonanie zadania na GPU lub innym urządzeniu
task.execute();

// Wyświetlenie wyników
for (int i = 0; i < size; i++) {
    System.out.println(input[i] + " squared = " + output[i]);
}
}
```

Next Level: TornadoVM



Juan Fumero • Following

Research Fellow at The University of Manchester, Intel oneAPI Innovator

1w • Edited • 🌐



Java [#Llama2](#) fork extended with GPU support using Level Zero JNI library to run on Intel ARC and Integrated GPUs. The initial version from my colleague [Michalis Papadimitriou](#) included a port for [#TornadoVM](#). We perform this study to analyze the performance impact of shared memory for LLMs applications such as Llama.

The Level Zero JNI library (<https://lnkd.in/g8jaZBVY>) is an open source framework to program and manage GPUs from Java, and we implemented this library to dispatch SPIR-V kernels in TornadoVM. However, currently TornadoVM does not implement shared memory.

👉 We see that the new level-zero port achieves higher tok/s vs TornadoVM on Integrated GPUs.

So, what's the catch? The Level-Zero JNI contains hand-written kernels with explicit runtime calls to manage and dispatch the code on GPUs, while in TornadoVM, well, it's parallelized and automatically accelerated from the Java sequential code. Besides, in near future, TornadoVM could implement shared memory buffers, and get extra performance on shared memory systems.

GPULlama3.java powered by TornadoVM 🔗 GPULlama3 Build & Run passing



Llama3 models written in **native Java** automatically accelerated on GPUs with [TornadoVM](#). Runs Llama3 inference efficiently using TornadoVM's GPU acceleration.

Currently, supports Llama3, Mistral, Qwen2.5, Qwen3 and Phi3 models in the GGUF format.

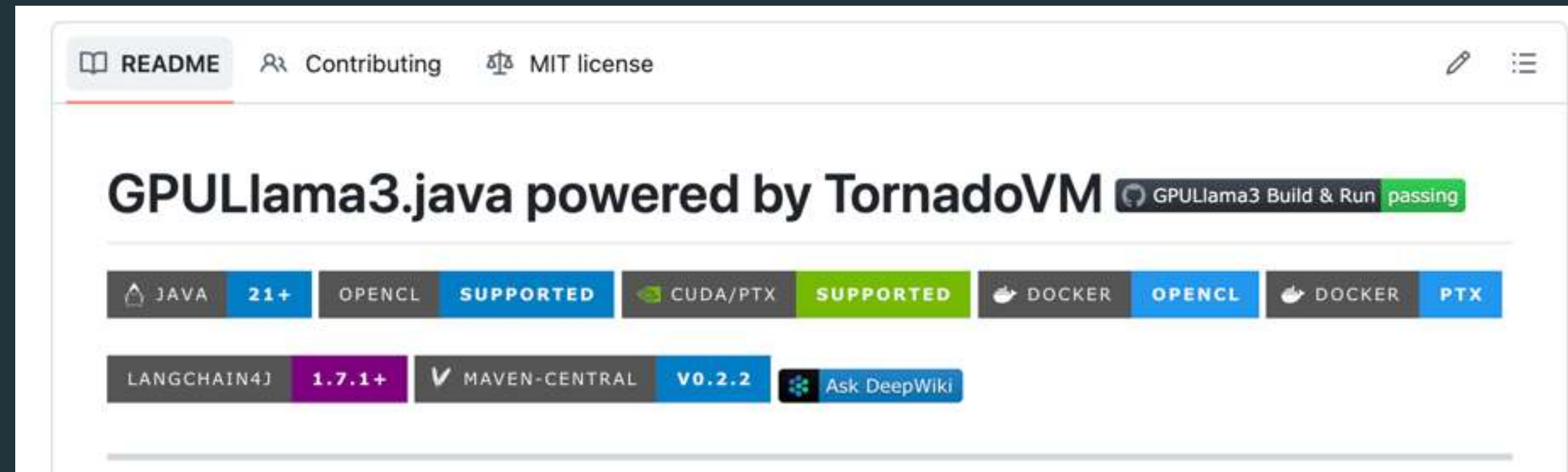
Builds on [Llama3.java](#) by [Alfonso² Peterssen](#). Previous integration of TornadoVM and Llama2 it can be found in [llama2.tornadovm](#).



Integration with LangChain4j

Since **LangChain4j v1.7.1**, `GPULlama3.java` is officially supported as a **model provider**.

This means you can directly use `GPULlama3.java` inside your LangChain4j applications without extra glue code, just run on your GPU.



DEMO 10



This models in the GGUF format.

Builds on [Llama3.java](#) by [Alfonso² Peterssen](#). Previous integration of TornadoVM and Llama2 it can be found in [llama2.tornadovm](#).



Integration with LangChain4j

Since **LangChain4j v1.7.1**, `GPULlama3.java` is officially supported as a **model provider**.

This means you can directly use `GPULlama3.java` inside your LangChain4j applications without extra glue code, just run on your GPU.



Juan Fumero · 1st

Consulting Member of Technical Staff @[oracle](#) Java Platform Gr...

4mo · Edited ·



I am happy to share that I am starting a new position at [Oracle](#), joining the Java Platform Group!



214

42 comments · 2 reposts

The TornadoVM Programming Model Explained with Juan Fumero - JVM Weekly vol. 152

Today, Juan Fumero explains, how TornadoVM works under the hood.



ARTUR SKOWRONSKI

DEC 04, 2025



Share

...





Future: HAT & Project Babylon



Java and GPU

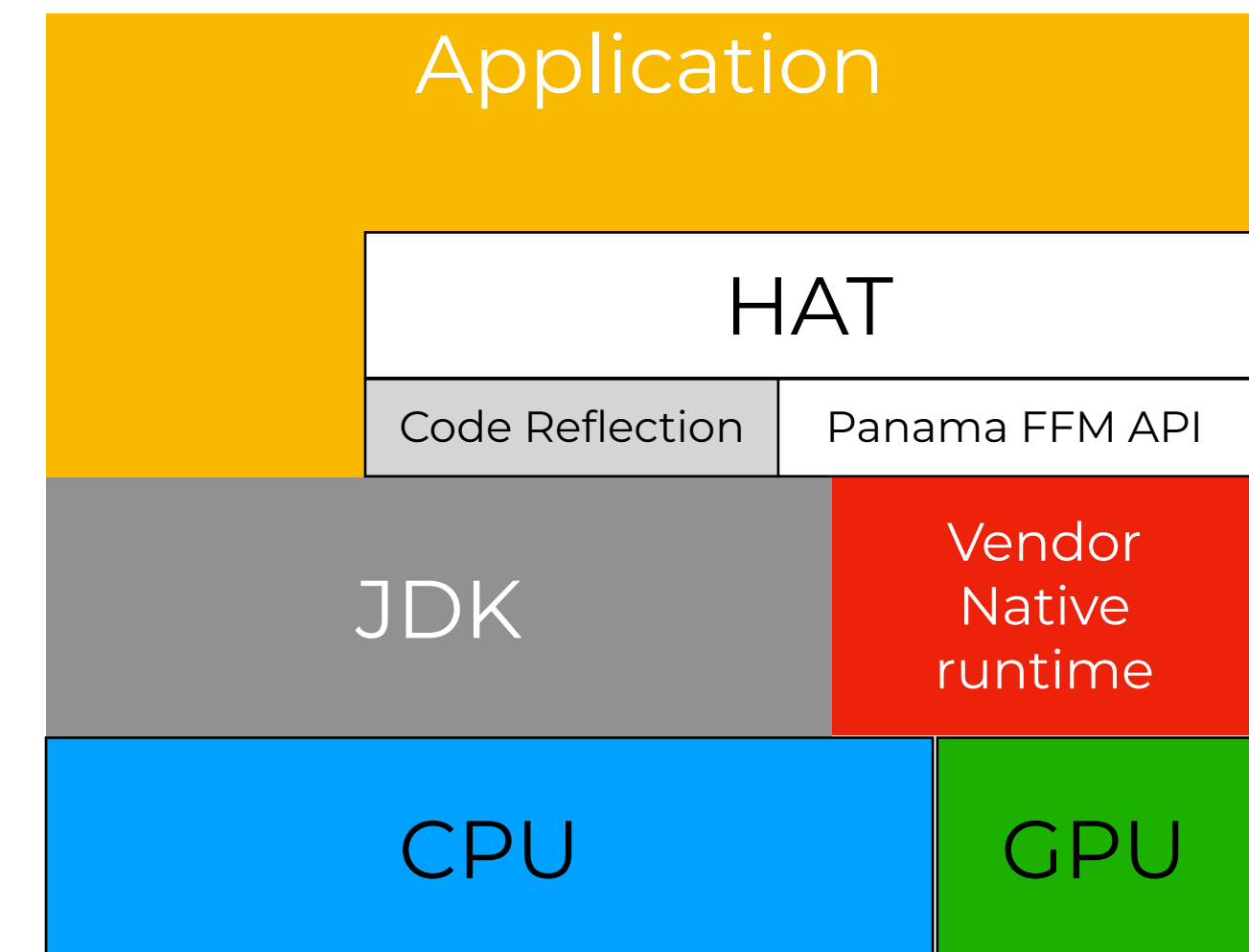
Are we nearly there yet?

JVM Language Summit
2023



Heterogenous Accelerator Toolkit (HAT)

- NDRange API
- FFM data wrapping patterns
- Support for Code Reflection from Project Babylon
- Support for FFM API



Heterogenous Accelerator Toolkit (HAT)

Code reflection is an extension of Java reflection that allows access to symbolic representations of Java code, such as method bodies and lambda expressions, at runtime.

This makes it possible to programmatically manipulate Java code.

Code Reflection



```
@CodeReflection  
public static void kernel(KernelContext kc, S32Arr s32Arr) {  
    s32Arr.array(kc.x, s32Arr.array(kc.x) * s32Arr.array(kc.x));  
}
```

Code Reflection

Abstract Syntax Tree

Full Syntactic Details

bytecode

Type Flattening

Code Models

Paul Sandoz

June 2024

Code reflection, an enhancement to Java reflection, enables access to symbolic representations of Java code in method bodies and lambda bodies. “Symbolic representations of Java code” may seem like a fancy term, but its easily demystified. It’s a model of Java code where the code of a method or lambda body is represented as instances of specific Java classes arranged in an appropriate structure. Thereby it is possible to write Java programs that manipulate Java programs. Before we get into the details of how code reflection models Java code we should talk about two existing approaches to modeling Java code.

Code Models

Abstract Syntax Tree

Full Syntactic Details

Code Models

Information about types and Structures

bytecode

Type Flattening

Code Models

Paul Sandoz

June 2024

Code reflection, an enhancement to Java reflection, enables access to symbolic representations of Java code in method bodies and lambda bodies. “Symbolic representations of Java code” may seem like a fancy term, but its easily demystified. It’s a model of Java code where the code of a method or lambda body is represented as instances of specific Java classes arranged in an appropriate structure. Thereby it is possible to write Java programs that manipulate Java programs. Before we get into the details of how code reflection models Java code we should talk about two existing approaches to modeling Java code.

```
// Depth-first search, reporting elements in pre-order
model.traverse(null, (acc, codeElement) -> {
    // Count the depth of the code element by
    // traversing up the tree from child to parent
    int depth = 0;
    CodeElement<?, ?> parent = codeElement;
    while ((parent = parent.parent()) != null) depth++;
    // Print out code element class
    System.out.println("  ".repeat(depth) + codeElement.getClass());
    return acc;
});
```

Code Models

JDK

Code Reflection

Code Models

Information about types and Structures

GPU

Vendor Native Runtime Code

Code Models

July 28, 2021

Introducing Triton: Open-source GPU programming for neural networks

[View code ↗](#)

[Read documentation ↗](#)



Exploring Triton GPU programming for neural networks in Java

Paul Sandoz
February 2024

In this article we will explain how we can use Code Reflection to implement the [Triton](#) programming model in Java as an alternative to Python.

Code Reflection is a Java platform feature being researched and developed under OpenJDK Project Babylon.

We will introduce Code Reflection concepts and APIs as we explain the problem and present a solution. The explanations are neither exhaustive nor very detailed, they are designed to give the reader an intuitive sense and understanding of Code Reflection and its capabilities.

Triton

[Triton](#) is a domain-specific programming model and compiler developers can use to write programs in Python that compile to GPU code.

Triton enables developers with little or no experience of GPU hardware and GPU-specific programming languages, such as CUDA, to write very efficient parallel programs.

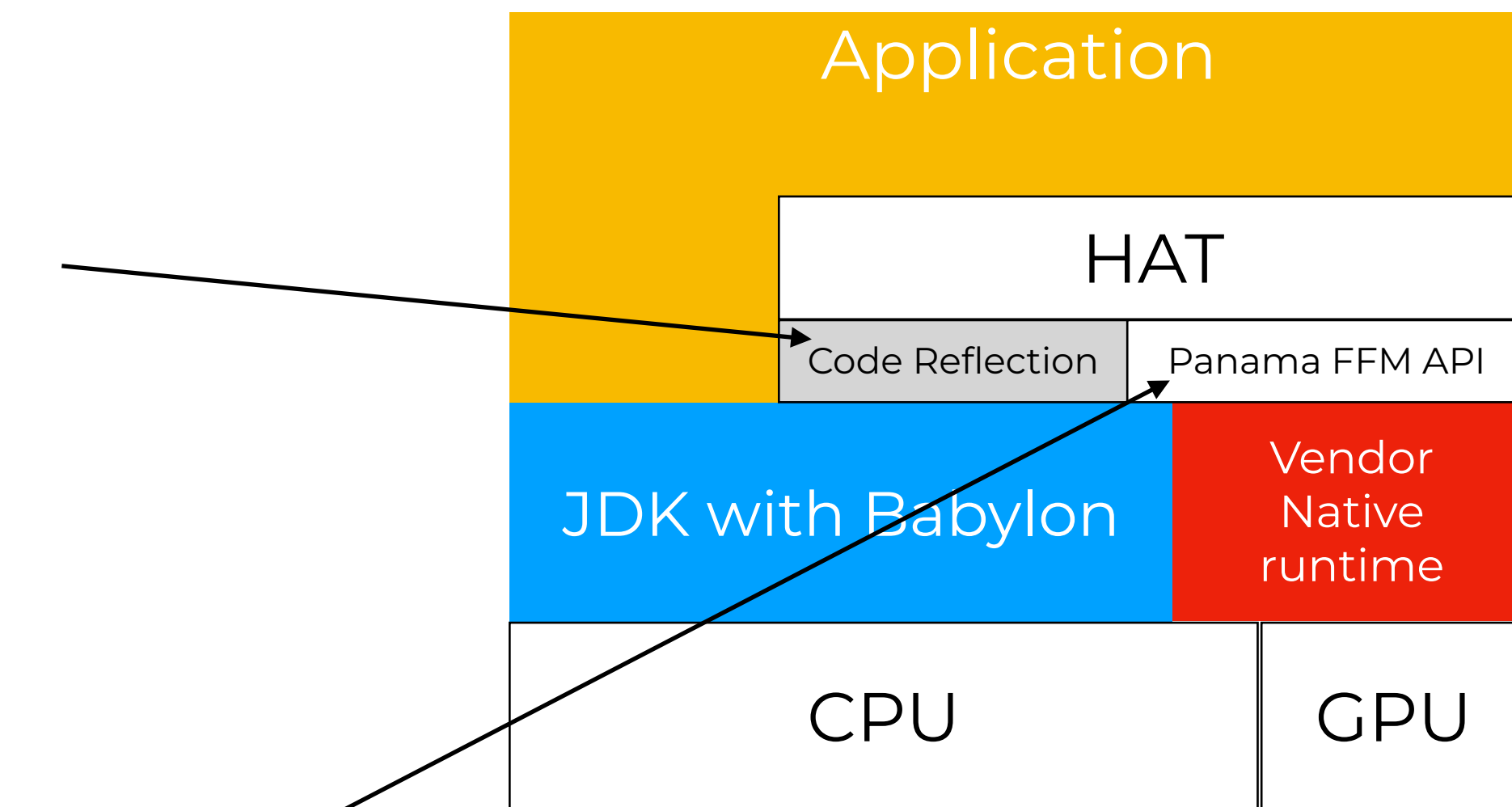
The release announcement for Triton [states](#):

Triton makes it possible to reach peak hardware performance with relatively little effort; for example, it can be used to write FP16 matrix multiplication kernels that match the performance of cuBLAS—something that many GPU programmers can't do—in under 25 lines of code. Our researchers have already used it to produce kernels that are up to 2x more efficient than equivalent Torch implementations, and we're excited to work with the community to make GPU programming more accessible to everyone.

Code Models

Babylon can use code reflection to dynamically generate GPU code (e.g., OpenCL, CUDA) by analyzing the code model and converting fragments of Java code into corresponding GPU instructions.

Panama FFM API and off-heap **MemorySegments** eliminate the need for manual memory management between the JVM and GPU, speeding up data exchange and increasing performance.



Exploring Triton GPU programming for neural networks in Java

Paul Sandoz
February 2024

In this article we will explain how we can use Code Reflection to implement the [Triton](#) programming model in Java as an alternative to Python.

Code Reflection is a Java platform feature being researched and developed under OpenJDK Project Babylon.

We will introduce Code Reflection concepts and APIs as we explain the problem and present a solution. The explanations are neither exhaustive nor very detailed, they are designed to give the reader an intuitive sense and understanding of Code Reflection and its capabilities.

Code Models

Heterogeneous Accelerator Toolkit

JVM Language Summit
2024



Heterogeneous Accelerator Toolkit (HAT) Update #JVMLS

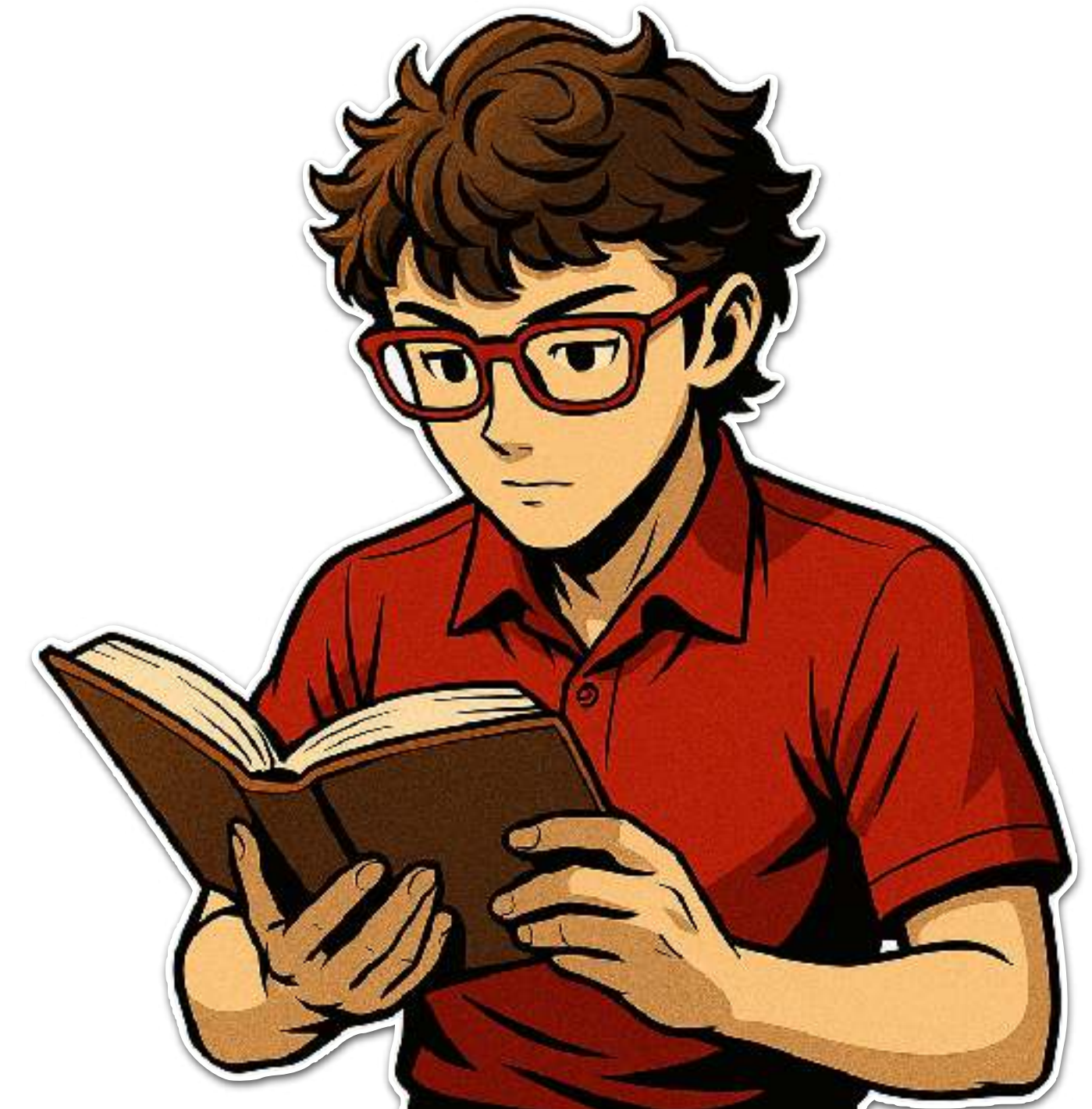
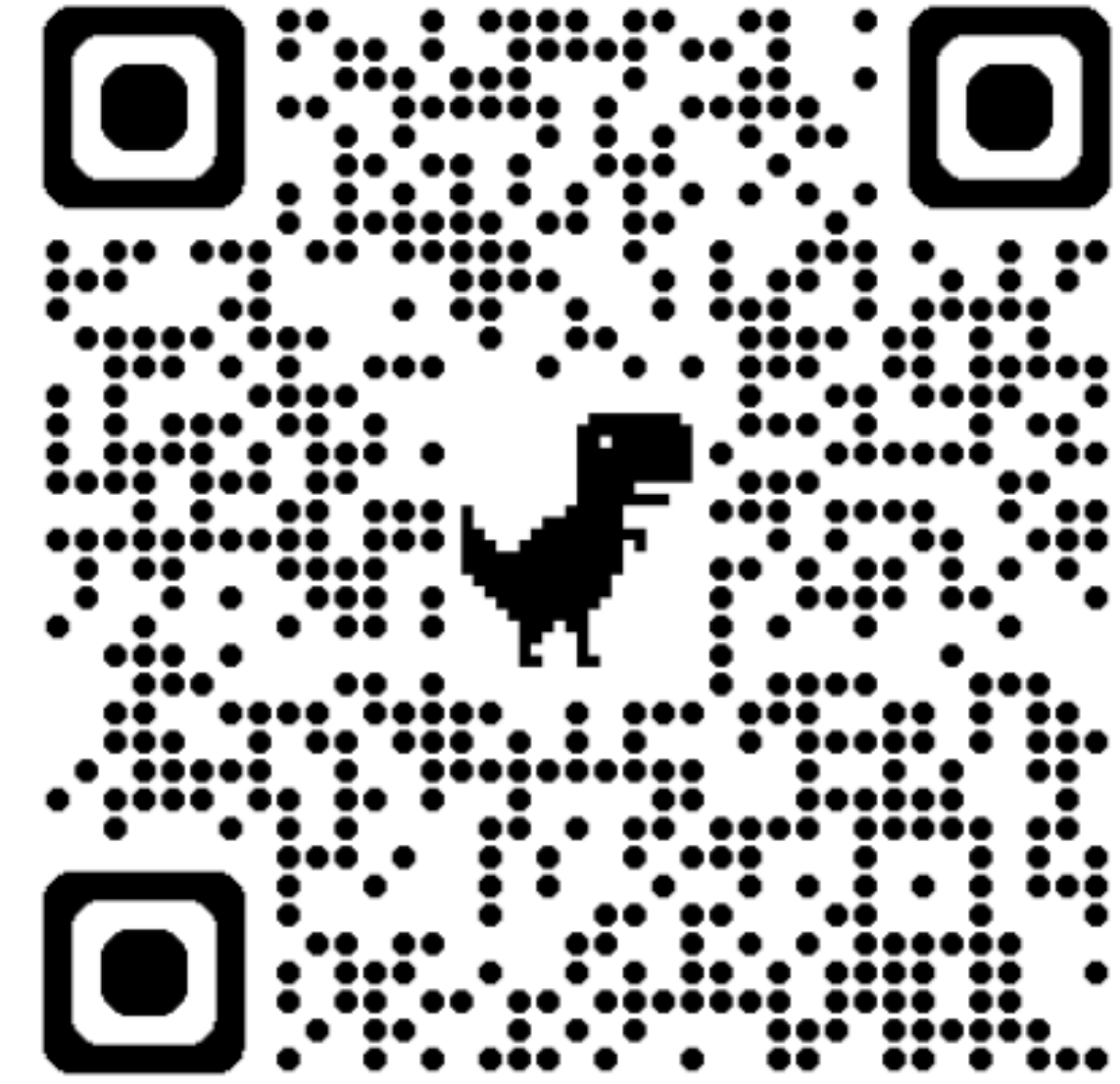
Optimizing GPU Programs from Java using Babylon and HAT

Juan Fumero

January 2026

Key Takeaways

- [The Heterogeneous Accelerator Toolkit \(HAT\)](#) is a parallel programming framework that allows Java developers to offload Java code and dispatch the generated code on modern hardware accelerators, such as Graphics Processing Units (GPUs).
- HAT can be used to speed up massive parallel workloads such as Deep Learning, AI, big data analytics and physic simulations, just to name a few, by automatically offloading and running these workloads on specialized hardware.
- HAT provides programming abstractions to facilitate GPU programming from Java: some key abstractions are an ND-Range API, a compute-layer and a kernel layer which allow Java developers to write explicit parallel code that can be offloaded to GPUs. Besides HAT provides memory abstractions that facilitate the usage of custom data structures and efficiently map them to the different memory regions of the GPUs.
- This article provides an overview of the HAT programming model: using the matrix-multiplication as an example, we demonstrate how Java developers can tune GPU workloads from the Java side to achieve performance close to native cuBLAS, scaling from 7 GFLOP/s on CPUs to 14 TFLOP/s on an NVIDIA A10 GPU.



Optimizing GPU Programs from Java using Babylon and HAT

Juan Fumero

January 2026

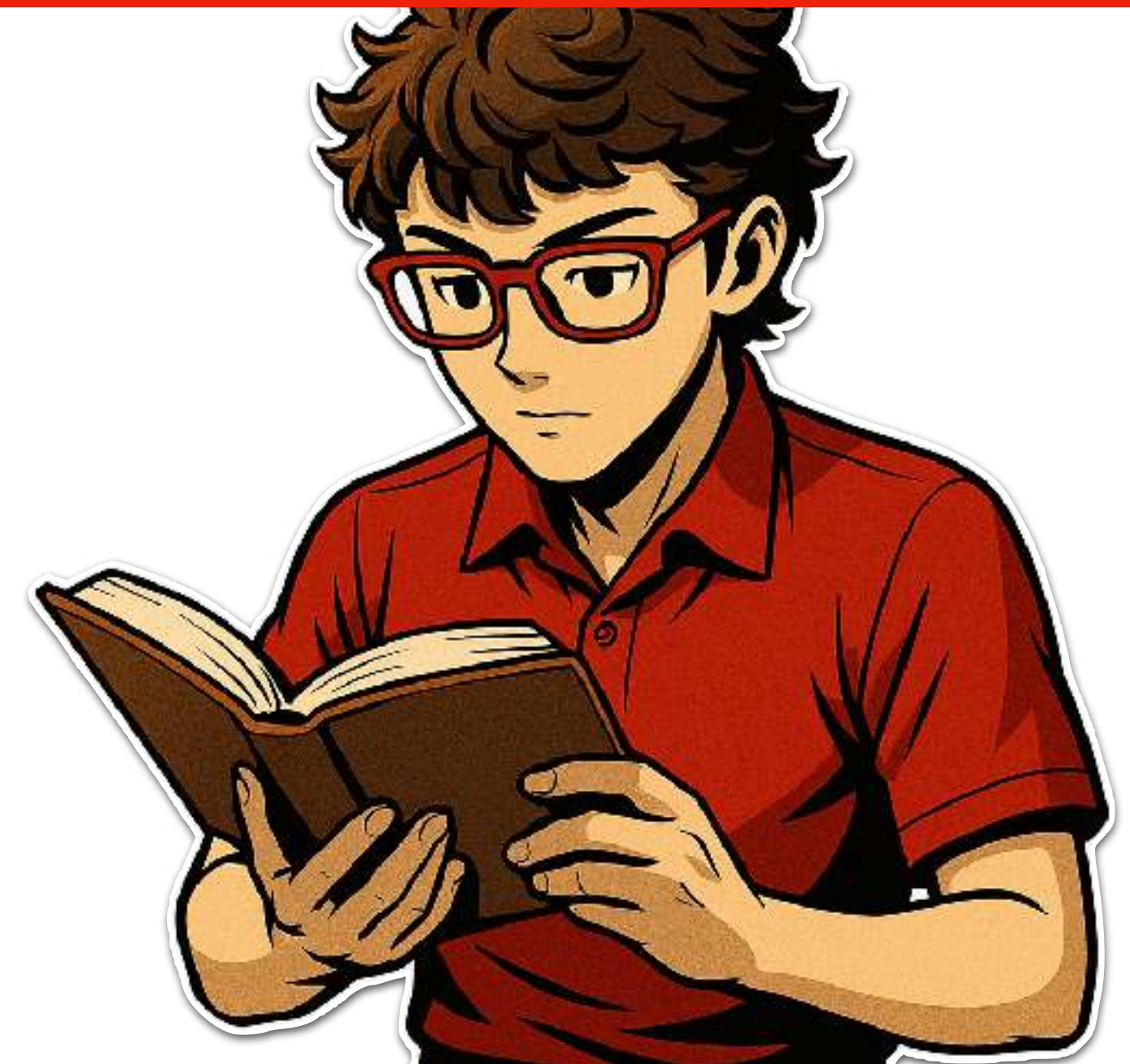
Key Takeaways

- The [Heterogeneous Accelerator Toolkit \(HAT\)](#) is a parallel programming framework that allows Java developers to offload Java code and dispatch the generated code on modern hardware accelerators, such as



DEMO 11

- HAT provides programming abstractions to facilitate GPU programming from Java: some key abstractions are an ND-Range API, a compute-layer and a kernel layer which allow Java developers to write explicit parallel code that can be offloaded to GPUs. Besides HAT provides memory abstractions that facilitate the usage of custom data structures and efficiently map them to the different memory regions of the GPUs.
- This article provides an overview of the HAT programming model: using the matrix-multiplication as an example, we demonstrate how Java developers can tune GPU workloads from the Java side to achieve performance close to native cuBLAS, scaling from 7 GFLOP/s on CPUs to 14 TFLOP/s on an NVIDIA A10 GPU.



LLM Inference Performance

Rank	Approach	Speed	Change from Previous Run
	java-llama.cpp	44.62 tok/s	-1.41 tok/s (-3%)
	Cyfra	27.3 tok/s	+2.0 tok/s (+8%)

DEMO 12

5	TornadoVM GPULLama3	6.74 tok/s	+0.11 tok/s (+2%)
6	Llama3.java (JDK 21)	0.27 tok/s	-0.03 tok/s (-10%)

LLM Inference Performance

Rank	Approach	Speed	Change from Previous Run
	java-llama.cpp	44.62 tok/s	-1.41 tok/s (-3%)
	Cyfra	27.3 tok/s	+2.0 tok/s (+8%)
	Llama3.java (GraalVM 25)	10.94 tok/s	-2.81 tok/s (-20%)
4	Llama3.java (JDK 25 Temurin)	8.74 tok/s	-3.82 tok/s (-30%)
5	TornadoVM GPULLama3	6.74 tok/s	+0.11 tok/s (+2%)
6	Llama3.java (JDK 21)	0.27 tok/s	-0.03 tok/s (-10%)



Bare-Metal Results:

- Llama3.java (JDK 21): 0.52 tok/s
- Llama3.java (JDK 25): 5.15 tok/s (9.9x speedup)
- **java-llama.cpp CUDA: 72.89 tok/s** (all 17/17 layers offloaded to GPU!)
- JCuda: OK (Tesla T4 detected)
- TornadoVM VectorAdd: 10.91 GB/s
- TornadoVM GPULlama3: 18.89 tok/s

Docker Results:

- Llama3.java (JDK 21): 0.52 tok/s
- Llama3.java (JDK 25): 4.76 tok/s
- **java-llama.cpp CUDA: 65.48 tok/s**
- JCuda: OK
- TornadoVM VectorAdd: OK
- TornadoVM GPULlama3: OK
- TensorFlow FFM: OK
- GraalPy Java Host: OK

ANNOUNCING TESLA T4 ON GCP

THE CLOUD SCALE-OUT ACCELERATOR

320 Turing Tensor Cores
2,560 CUDA Cores
65 FP16 TFLOPS | 130 INT8 TOPS | 260 INT4 TOPS
16GB | 320GB/s
70 W

Now Available Early Access





Conclusions: What the future holds



Panama & Valhalla SIMD:



Where are we?

Panama & Valhalla SIMD:

Valhalla Half-Float:



Where are we?

Panama & Valhalla SIMD:

Valhalla Half-Float:

GraalPy:



Where are we?

Panama & Valhalla SIMD:

Valhalla Half-Float:

GraalPy:

JCuda:



Where are we?

Panama & Valhalla SIMD:

Valhalla Half-Float:

GraalPy:

JCuda:

Sumatra:



Where are we?

Panama & Valhalla SIMD:

Valhalla Half-Float:

GraalPy:

JCuda:

Sumatra:

TornadoVM:



Where are we?

Panama & Valhalla SIMD:

Valhalla Half-Float:

GraalPy:

JCuda:

Sumatra:

TornadoVM:

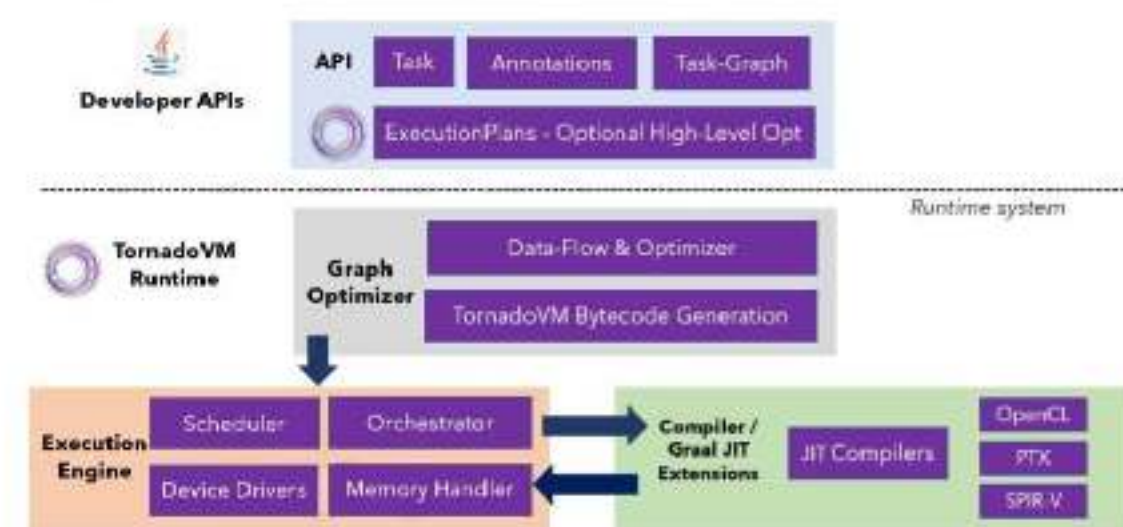
HAT & Babylon:



Where are we?



TornadoVM Software Stack



⚡ FastAPI

Look what they need to
mimic a fraction of our power

ArturSkowronski / conference-jvm-in-age-ai-2026

Type to search

<> Code

Issues

Pull requests

Actions

Projects

Security

Insights

Settings

conference-jvm-in-age-ai-2026

Public

Pin

Watch 0

Fork 0

Star 0

main 10 Branches 0 Tags

Go to file

Code

About

ArturSkowronski and claude Update Babylon demo to run HA... ffc62d8 · 2 minutes ago 102 Commits

.claude	New Gradle Wrapper	last week
.github/workflows	Add --enable-preview for TornadoVM Java 21 previ...	3 weeks ago
benchmark-results	Update GCP benchmarks with CUDA-enabled resul...	3 days ago
cyfra-demo	Add Cyfra (Scala/Vulkan GPU) LLM inference demo	yesterday
demos	Update Babylon demo to run HAT MatMul GPU exa...	2 minutes ago
docker	Add Cyfra (Scala/Vulkan GPU) LLM inference demo	yesterday
gradle/wrapper	New Gradle Wrapper	last week
scripts	Add Cyfra (Scala/Vulkan GPU) LLM inference demo	yesterday
tornadovm-demo	Fix bash unbound variable error in TornadoVM script	1 hour ago

About

No description, website, or topics provided.

Readme

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package



<https://github.com/ArturSkowronski/conference-jvm-in-age-ai-2026>



Thank you 🙏



@ArturSkowronski



jvm weekly